
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 18:41:37
Файлов исходного кода: 87
Суммарный объём кода: 1.23 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs-bfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасика [id: aho-corasick]

24. src/data/vizStepBus.ts

25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts

27. src/data/tickets/ticket14_20.ts

28. src/data/tickets/ticket21_24.ts

29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx

31. src/components/BellmanFordViz.tsx

32. src/components/BfsViz.tsx

33. src/components/ChapterImage.tsx

34. src/components/ChapterNav.tsx

35. src/components/ChapterView.tsx

36. src/components/DfsBridgesSimulator.tsx

37. src/components/DijkstraViz.tsx

38. src/components/DPVisualizer.tsx

39. src/components/EulerSimulator.tsx

40. src/components/ExpressQuiz.tsx

41. src/components/FloydViz.tsx

42. src/components/GraphTraversalViz.tsx

43. src/components/HeapViz.tsx

44. src/components/hints/demoKit.tsx

45. src/components/hints/demosExtra.tsx

46. src/components/hints/demosGraph.tsx

47. src/components/hints/demosStruct.tsx

48. src/components/hints/MiniDemo.tsx

49. src/components/hints/SimulatorModal.tsx

50. src/components/hints/TermPopover.tsx

51. src/components/JohnsonViz.tsx

52. src/components/KosarajuViz.tsx

53. src/components/KruskalSimulator.tsx

54. src/components/MnemonicCards.tsx

55. src/components/MobileToc.tsx

56. src/components/Navbar.tsx

57. src/components/PlanarityDemo.tsx

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
 push:
```

```
 branches:
```

```
- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
```



```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
# На Ubuntu политика ImageMagick иногда запре
# разрешаем то, что нужно пайплайну (text ->
for policy in /etc/ImageMagick-6/policy.xml /
    if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```

- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.densi
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`\${public}/export/full_code_all_pages
 echo "| \`\${public}/export/full_code_all_pages

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```
-----  
    <body class="bg-slate-950 text-slate-100 min-h-screen  
....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg  
new file mode 100644  
index 00000000..3bbbba0  
--- /dev/null  
+++ b/public/favicon.svg  
@@ -0,0 +1,5 @@  
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
+</svg>  
....
```

```
## `src/App.tsx`
```

```
### Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from
import { chapters } from "../data/content";
```



```

 }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
 {/* Сайдбар только на десктопе — на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId}>
 </div>

 <main id="main-content" className="flex-1 min-w-0">
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index}>
 </div>
 </main>
 </div>
 </>
) : null}
 </Navbar>
 </div>
);

```

```

 </footer>
 </div>
);
}
....

Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
  </>
-  ) : (
+  ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
      </>
    </main>
  ) : (
    <main>
      <ChapterNav />
      <ChapterView />
    </main>
  )
}

```

```

<div className="flex items-center gap-3 w-full" >
  <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
    <Sparkles className="w-6 h-6" />
  </div>
  <div className="min-w-0 flex-1" >
    <h1 className="text-xl font-extrabold text-slate-900" >
      Универсальное пособие
    </h1>
    <p className="text-xs text-indigo-400 font-medium" >
      Подготовка к экзаменам: Алгоритмы, Структуры данных
    </p>
  </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
      activeTab === 'guide'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
      activeTab === 'simulator'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <CodeIcon className="w-4 h-4" /> Симулятор
  </button>
</div>

```

```

        </a>
        <button
          id="download-html-btn"
          onClick={saveBook}
          disabled={busy}
          className="flex items-center justify-center
            er:bg-amber-600/20 border border-amber-500/
          title="Скачать всё пособие одним автономным
        >
          {busy ? <Loader2 className="w-4 h-4 animate
        </button>
      </div>
    </div>
  </header>
);
};
....

```

Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

```

-----  
### Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.asm.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в интерпретатор
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}
```

```

        const captured = [...stdout, ...stderr].join("");
        setOutput(`${captured}${captured && !captured.end`);
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >
                        Как это работает <ExternalLink className="w-4 h-4" />
                    </a>
                </div>
            </div>
        </div>
    </main>

```

```

    }}
    spellCheck={false}
    aria-label="Код Python"
    className="block w-full min-h-[360px] res
-none focus:ring-2 focus:ring-inset focus:r
    placeholder="Напишите Python-код..."
  />

<div className="border-t border-slate-800 b
  <label className="block px-4 pt-3 text-[1
    Ввод для input() • по одной строке на к
  </label>
  <textarea
    id="python-stdin"
    value={stdin}
    onChange={(event) => setStdin(event.tar
    spellCheck={false}
    rows={2}
    placeholder={'Например:\nАлиса'}
    className="block w-full resize-y bg-tra
eholder:text-slate-700"
  />
</div>
</section>

<section className="rounded-2xl border border
  <div className="flex items-center justify-b
    <div className="flex items-center gap-2 t
      <Terminal className="w-4 h-4 text-emera
    </div>
    <span className={`inline-flex items-cente
      {runtimeReady && <CheckCircle2 classNam
      {runtimeReady ? "готов" : "не загружен"}
    </span>
  </div>
  <pre className="min-h-[360px] max-h-[560px]
x] leading-6 text-slate-300">
    {output}
  </pre>

```

Универсальное пособие • полный исходный код

```
-----  
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`  
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`  
+  
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код  
+name = "алгоритмы"  
+  
+for number in range(1, 4):  
+    print(f"{number}. Учим {name}!")  
+`;  
+  
+type PyodideRuntime = {  
+  runPythonAsync: (source: string) => Promise<unknown>  
+  setStdout: (options: { batched: (message: string) => void }) => void  
+  setStderr: (options: { batched: (message: string) => void }) => void  
+  setStdin: (options: { stdin: () => string | number | null }) => void  
+};  
+  
+type PyodideModule = {  
+  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>  
+};  
+  
+let runtimePromise: Promise<PyodideRuntime> | null = null;  
+  
+function getPythonRuntime() {  
+  if (!runtimePromise) {  
+    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {  
+      (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL })  
+    });  
+  }  
+  return runtimePromise;  
+}  
+  
+function errorText(error: unknown) {  
+  if (error instanceof Error) return error.message;  
+  return String(error);  
+}  
+  
+export function PythonCompiler() {
```



```

+
+  const reset = () => {
+    setCode(STARTER_CODE);
+    setStdin("");
+    setOutput("Нажмите «Запустить», чтобы увидеть резу.
+  };
+
+  return (
+    <main className="max-w-screen-2xl mx-auto px-4 sm:p
+      <div className="max-w-6xl mx-auto">
+        <div className="flex flex-col sm:flex-row sm:i
+          <div>
+            <div className="inline-flex items-center g
+              <Terminal className="w-4 h-4" /> Боковая
+            </div>
+            <h2 className="text-2xl sm:text-3xl font-e
+            <p className="text-slate-400 mt-2 max-w-2x
+              Пишите небольшой код и запускайте его пр
+              который переводит CPython в WebAssembly.
+            </p>
+          </div>
+          <a
+            href="https://pyodide.org/"
+            target="_blank"
+            rel="noreferrer"
+            className="inline-flex items-center gap-1.5
+          >
+            Как это работает <ExternalLink className="
+          </a>
+        </div>
+
+        <div className="grid xl:grid-cols-[minmax(0,1.
+          <section className="rounded-2xl border borde
+            <div className="flex flex-wrap items-cente
+              <div className="flex items-center gap-2"
+                <span className="w-2.5 h-2.5 rounded-f
+                <span className="text-sm font-bold tex
+                <span className="text-[11px] text-slate

```

```

+
+         <div className="border-t border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+             <label className="block px-4 pt-3 text-slate-600 font-medium">Ввод для input() • по одной строке на строку</label>
+             <textarea
+                 id="python-stdin"
+                 value={stdin}
+                 onChange={(event) => setStdin(event.target.value)}
+                 spellCheck={false}
+                 rows={2}
+                 placeholder={'Например:\nАлиса'}
+                 className="block w-full resize-y bg-transparent border border-slate-800 rounded-md px-4 py-3 text-slate-700"
+             />
+         </div>
+     </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 20px; margin: 10px 0;}}>
+         <div className="flex items-center justify-between" style={{border: 1px solid #333; border-radius: 10px; padding: 5px; margin-bottom: 10px;}}>
+             <div className="flex items-center gap-2" style={{border: 1px solid #333; border-radius: 10px; padding: 5px; flex: 1;}}>
+                 <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+             </div>
+             <span className={`inline-flex items-center gap-2`} style={{border: 1px solid #333; border-radius: 10px; padding: 5px; flex: 1;}}>
+                 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
+                 {runtimeReady ? "готов" : "не загружен"}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin-top: 10px;}}>
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin-top: 10px;}}>
+         </section>
+     </div>
+
+     <div className="mt-5 grid md:grid-cols-3 gap-3" style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin-top: 10px;}}>
+         <div className="flex gap-2 rounded-xl border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin-bottom: 10px;}}>

```

```
plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
  alias: {
    "@": path.resolve(__dirname, "src"),
  },
},
server: {
  host: "0.0.0.0",
  port: 5173,
  strictPort: true,
  // предпросмотр может открываться через внешний про
  allowedHosts: true,
},
preview: {
  host: "0.0.0.0",
  port: 4173,
  strictPort: true,
  allowedHosts: true,
},
});
```
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
```

Универсальное пособие • полный исходный код

-
2. ****Структура JSON/TS:**** Каждая новая малая страница должна иметь объект `{ "content": ... }` для вставки в ``src/data/content.ts``.
 3. ****Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
 - Определение/правило в центре (``border-blue-500`` или ``border-blue-500``)
 - Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
 4. ****Не ломай верстку:**** Не придумывай свои CSS-классы. Используй только классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.

ФАЙЛ 4/87: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискретке
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/87: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "version": "1.0.0",
  "description": "A reader for the ExamPrep Remix Remix.",
  "author": "Remix Remix",
  "license": "MIT",
  "keywords": ["Remix", "Remix", "ExamPrep", "Reader"],
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "18.2.0",
    "react-dom": "18.2.0",
    "react-router-dom": "6.14.2",
    "vite": "4.3.9"
  },
  "devDependencies": {
    "@types/react": "18.2.0",
    "@types/react-dom": "18.2.0",
    "@types/react-router-dom": "6.14.2",
    "@types/vite": "4.3.9",
    "typescript": "5.0.4"
  }
}
```

```
"devDependencies": {
  "@tailwindcss/vite": "4.1.17",
  "@types/node": "22.19.17",
  "@types/pdfkit": "^0.17.6",
  "@types/react": "19.2.7",
  "@types/react-dom": "19.2.3",
  "@vitejs/plugin-react": "5.1.1",
  "esbuild": "^0.28.2",
  "tailwindcss": "4.1.17",
  "typescript": "5.9.3",
  "vite": "7.3.2",
  "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с поддержкой темной темы и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash
npm install
npm run dev # предпросмотр на http://0.0.0.0:5174
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- * ****Содержание**** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- * ****Переходы между темами**** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- * ****Всплывающие подсказки по терминам**** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симуля
- Словарь – `src/data/glossary.ts`, мини-анимации – `src
- * ****Реестр интерактивов**** – `src/components/vizRegistry` и для панели под главой, и для модальки из подсказки.

Структура

...

- src/
 - App.tsx – оболочка, привязка глав к визуалу
 - components/ – интерактивные симуляторы (Дейкстра)
 - data/ – контент пособия (главы/билеты)
- scripts/export/ – пайплайн код → txt → png → pdf
- public/export/ – сгенерированные TXT и PDF (обновление)
- .github/workflows/ – автоматизация

— — —

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен
азуемые теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `<h2>` и
- * Подзаголовки: Используют тег `<h3>`.

Текст и параграфы

* Обычный текст содержится в тегах ``<r>``.

Универсальное пособие • полный исходный код

```
-----
> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |
|--------------------|-----------|----------------------|
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет | Тема | Статус |
|---------------|--------------------------------|-----------------------------|
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. К |
| | ит вхолостую. | |
| **7** | Планарность и раскраска графов | Номерной гла |
| | огии**. | |
| **11** | Мосты в графах | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; к |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- * `stack-dfs` → `StackViz.tsx` – ****Стек и DFS****
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – ****Очередь**
- * `heap-beam-search` → `HeapViz.tsx` – ****Куча и лучевой**
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – ****Дер**
- * `dynamic-programming` → `DPVisualizer.tsx` – ****Динами**

| | | | |
|----|---|---|-------|
| 9 | 9. Графы. Топологическая сортировка | 1 | - |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | |
| 12 | 12. Графы. Точки сочленения | 1 | - - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | |
| 16 | 16. Графы. Кратчайший путь. Флойд | - | - |
| 17 | 17. Графы. Алгоритм Джонсона | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - - |
| 22 | 22. Z-функция | 4 | - - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - - |
| 24 | 24. Классы сложности, сведение задач | 4 | - - |
| - | Обзор: Кратчайшие пути и Графы | - | - - |
| - | Алгоритмы на карте | - | - - |
| - | Эйлер: Путь ≠ Цикл | - | - - |
| - | Мосты в графах: код + визуализация | 1 | - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш
3. ****Добавить 2D**** к 4 главам «Уровня 4» — минимум `inli`
4. ****Решить судьбу `alg-map`**** — раскрыть или удалить.

ФАЙЛ 9/87: `tsconfig.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
```



```
-----
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algov0/, локаль
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

```

    if (compilerHeight !== null) window.localStorage.set
  }, [compilerHeight]);
  /** Содержание свернуто (по умолчанию сворачиваем сам
const [sidebarCollapsed, setSidebarCollapsed] = useSt
  const cached = typeof window !== "undefined" ? wind
  if (cached !== null) return cached === "1";
  return typeof window !== "undefined" && window.inne
});
const toggleSidebar = useCallback(() => {
  setSidebarCollapsed((c) => {
    window.localStorage.setItem("sidebarCollapsed", c
    return !c;
  });
}, []);

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.inn
  };
  apply();
  window.addEventListener("resize", apply);
  return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
  0,
  chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[in

const goTo = useCallback((id: string) => {
  setActiveChapterId(id);

```

```

<div ref={contentRef} className={compilerOpen ? "
  {activeTab === "guide" ? (
    <>
      <MobileToc
        selectedChapterId={activeChapterId}
        setSelectedChapterId={goTo}
        currentTitle={activeChapter.title}
        index={index}
        total={chapters.length}
      />

      <div className="flex flex-col lg:flex-row m
        /* Сайдбар только на десктопе – на мобил
        <div
          className={`hidden lg:block shrink-0 bo
tion-all duration-200 ${sidebarCollapsed ?
        >
          {!sidebarCollapsed && <Sidebar selected
        </div>
        /* Вкладка-переключатель содержания у ле
        <button
          type="button"
          onClick={toggleSidebar}
          className="hidden lg:flex items-center
l-0 border-slate-700 bg-slate-900/90 px-1 p
          aria-label={sidebarCollapsed ? "Разверн
          title={sidebarCollapsed ? "Развернуть с
        >
          {sidebarCollapsed ? <PanelLeftOpen clas
          <span className="text-[10px] font-bold
        </button>

        <main id="main-content" className="flex-1
          /* Шапка главы с быстрыми переходами *
          <div className="flex items-center justifi
            <div className="min-w-0">
              <div className="text-[10px] upperca
                {activeChapter.category || "Униве

```

```
        </main>
    }}

    <footer className="p-8 text-center text-slate-600">
        © 2026 Universal Educational Guide. Интерактивный
    </footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
    {compilerOpen && (
        <PythonCompiler
            chapterId={activeChapter.id}
            chapterTitle={activeChapter.title}
            width={compilerWidth}
            height={compilerHeight}
            onWidthChange={setCompilerWidth}
            onHeightChange={setCompilerHeight}
            onOpenGuide={() => setActiveTab("guide")}
            onClose={() => setCompilerOpen(false)}
        />
    )}
}}
</div>
);
}
```

ФАЙЛ 12/87: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 KB

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
```

```

        return NodeFilter.FILTER_ACCEPT;
    },
    });

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
}

for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
        const surface = m[1].toLowerCase();
        const id = labelToId.get(surface);
        if (!id) continue;

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
        perTerm.set(id, usedTerm + 1);
        perSurface.set(surface, usedSurface + 1);

        if (m.index > last) frag.appendChild(document.createElement("span"));
        const span = document.createElement("span");
        span.className = "term-hint";
        span.dataset.termId = id;
        span.tabIndex = 0;
        span.setAttribute("role", "button");
    }
}

```

```
    if (!el.querySelector(".term-hint")) run();
  });
}
```

ФАЙЛ 13/87: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 К

```
@import "tailwindcss";
```

```
@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
    box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
  }
  .no-scrollbar::-webkit-scrollbar {
    display: none;
  }
  .no-scrollbar {
    scrollbar-width: none;
  }
}
```

```
@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
  }
}
```

```
.term-hint {
  cursor: help;
  border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
  color: #c7d2fe;
  border-radius: 3px;
  padding: 0 1px;
  transition:
    background-color 0.15s ease,
    color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
  background: rgba(99, 102, 241, 0.22);
  color: #fff;
  outline: none;
}
@media (hover: none) {
  .term-hint {
    border-bottom-style: solid;
    background: rgba(99, 102, 241, 0.12);
  }
}

/* Контент главы приходит готовым HTML — чиним его под
.chapter-body {
  max-width: 100%;
  overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
  min-width: 0;
}

.chapter-body :where(pre, code) {
  white-space: pre-wrap;
  word-break: break-word;
}
```

```
        font-size: 1.2rem !important;
    }
    .chapter-body :where(.text-xl) {
        font-size: 1.05rem !important;
    }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
    width: 8px;
    height: 8px;
}
::-webkit-scrollbar-track {
    background: #0f172a;
}
::-webkit-scrollbar-thumb {
    background: #334155;
    border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
    background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
    from {
        width: 0%;
    }
    to {
        width: 100%;
    }
}
```



```
}  
declare module '*.jpg?url' {  
  const src: string;  
  export default src;  
}  
declare module '*.jpeg' {  
  const src: string;  
  export default src;  
}  
declare module '*.png' {  
  const src: string;  
  export default src;  
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 17/87: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const additionalTickets: Chapter[] = [  
  {
```

```
    id: "sparse-table",
```

```
    title: "2. Двумерная разреженная матрица (Sparse Table)",
```

```
    type: "html",
```

```
    description: "Разреженная таблица для идемпотентных операций",
```

```
    category: "Продвинутые структуры",
```

```
    content: `
```

```
<section id="sparse-table" class="mb-12 scroll-mt-10">
```

```
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
    <span class="bg-indigo-600 text-white px-4 py-1">
```

```
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
  id: "treap",
```

```
  title: "3. Декартово дерево (Treap)",
```

```
  type: "html",
```

```
  description: "Дерево поиска по ключу и Куча по приоритету",
```

```
  category: "Продвинутые структуры",
```

```
  content: `
```

```
<section id="treap" class="mb-12 scroll-mt-10">
```

```
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">Тема</span>
```

```
    <h2 class="text-2xl sm:text-3xl font-bold text-white">3. Декартово дерево</h2>
```

```
  </div>
```

```
  <div class="space-y-8">
```

```
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
```

```
      <h3 class="text-xl font-bold text-blue-400">Описание</h3>
```

```
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
        <p class="text-lg text-blue-300 italic">Дерево Декарта — это бинарное дерево поиска, в котором узлы имеют два атрибута: ключ и приоритет.</p>
```

```
        <p class="text-xl font-mono text-white">Узлы хранят пары (ключ, приоритет).</p>
```

```
        метода: Split и Merge.</p>
```

```
      </div>
```

```
      <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
        <div class="bg-slate-800 p-6 rounded-lg">
```

```
          <div class="absolute -top-3 left-4 right-4 bottom-4">
```

```
            <div class="border border-emerald-500 shadow-md">
```

```
              Аналогия 1: Удар катаной (Split по ключу)</div>
```

```
            </div>
```

```
            <p class="text-slate-300 text-sm mb-4">Если узел X имеет приоритет меньше, чем приоритет его левого ребенка, то мы делаем операцию Split по значению X. Все, кто меньше X улетают в левое поддерево, а все, кто больше X, остаются в правом поддереве.</p>
```

```
          </div>
```

```
        </div>
```

```
    type: "html",
    description: "Дерево поиска, которое чинит само себя",
    category: "Продвинутые структуры",
    content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">

    <!-- 0. Определение -->
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-lg sm:text-xl font-mono">
        поиска <span class="text-slate-500">без</span>
        : серия поворотов, которая поднимает v в кор
        <p class="text-sm text-slate-400 text-c">
        т – поворот.</p>
      </div>

      <div class="grid grid-cols-1 lg:grid-cols-3">
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wid">
          <p class="text-white font-bold">мож
          <p class="text-slate-400 text-xs mt">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wid">
          <p class="text-emerald-300 font-bol">
          <p class="text-slate-400 text-xs mt">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wid">
          <p class="text-white font-bold">0(n
```

Аналогия 1: стопка бумаг на столе

```
</div>
<p class="text-slate-300 text-sm">У тебя
скиваешь его и, поработав, кладёшь <span class="text-emerald-400">
неделю самые нужные бумаги сами собой оказыва
ет ровно это: «взял → положил наверх».</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
```

Аналогия 2: тропинка через газон

```
</div>
<p class="text-slate-300 text-sm">Ты идёшь по дорожке, которая сама укорачивалась вдвое: чем чаще ты её
оплачивает все следующие. Пойдёшь один раз
</div>
```

```
</div>
```

<!-- 3. Поворот -->

```
<div class="bg-slate-800/60 p-6 rounded-xl border-rose-500 shadow-md">
  <h3 class="text-lg font-bold text-white mb-4">Поворот
  <p class="text-slate-300 text-sm mb-4">Поворот — это движение, три перевешенные ссылки</span> и
  <pre class="bg-slate-950 p-4 rounded-lg overflow-hidden">
```

<pre> / \ 20 C / \ A B </pre>	<pre> поворот 20 -----> <----- поворот 40 </pre>	<pre> / \ A 40 / \ B C </pre>
--	--	--

обход слева направо ДО: A 20 B 40 C

обход слева направо ПОСЛЕ: A 20 B 40 C ← тот же самый

```
<div class="grid grid-cols-1 md:grid-cols-2">
  <div class="bg-slate-900 rounded-lg p-4">
    <p class="text-emerald-400 font-bold">
    <p class="text-slate-300 text-xs">Поворот — это движение, три перевешенные ссылки
    сла поворотов — сломать его поворотами невозможно
  </div>
```

```

        </tr>
        <tr>
            <td class="py-3 pr-3 font-b<
            <td class="py-3 pr-3">х и р
            <td class="py-3 pr-3"><span
            <td class="py-3">р и г стал
        </tr>
    </tbody>
</table>
</div>

```

```

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделатъ руками.</p>
</div>

```

<!-- 5. Главная ловушка -->

```

<div class="bg-rose-950/30 p-6 rounded-xl borde
    <h3 class="text-lg font-bold text-rose-300
    <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
    <div class="grid grid-cols-1 lg:grid-cols-2
        <div class="bg-slate-950 rounded-lg p-4
            <p class="text-rose-400 font-bold t
            <pre class="overflow-x-auto text-[1

```

```

                20
            /
        40
    /
20

    →

        20
    /
20
    \
    40

    →

        \
        60
    /
    40

```

было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!</pre>

```

        <p class="text-slate-400 text-xs mt
мортизации не получится.</p>
</div>

```

а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути</span>. Все узлы, мимо
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
  <h3 class="text-lg font-bold text-white mb-1
  <p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag :
ах идея такая:</p>
  <div class="grid grid-cols-1 md:grid-cols-3
    <div class="bg-slate-900 rounded-lg p-4
      <p class="text-white font-bold mb-1
      <p class="text-slate-400 text-xs">c
    </div>
    <div class="bg-slate-900 rounded-lg p-4
      <p class="text-white font-bold mb-1
      <p class="text-slate-400 text-xs">н
    </div>
    <div class="bg-slate-900 rounded-lg p-4
      <p class="text-white font-bold mb-1
      <p class="text-slate-400 text-xs">д
    </div>
  </div>
  <p class="text-slate-400 text-xs mt-4">Отсю
сти запросов splay-дерево не хуже (с точнос
дерево. Оно как бы само подстраивается под
</div>
```

```
<!-- 8. Операции -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
  <h3 class="text-lg font-bold text-white mb-1
  <p class="text-slate-300 text-sm mb-4">Крас
лается в корне.</p>
  <div class="space-y-2 text-sm">
    <div class="bg-slate-900 rounded-lg p-3
  </span> <span class="text-slate-400">— сныс
```

```

x.parent = g; // x занял его место
if (g) { if (g.left === p) g.left = x; else g.right =
}

// поднимаем x в корень
function splay(x) {
  while (x.parent) {
    const p = x.parent, g = p.parent;

    if (!g) { // ZIG: деда нет
      rotate(x);
    } else if ((g.left === p) === (p.left === x)) {
      rotate(p); // ZIG-ZIG: сносим
      rotate(x);
    } else {
      rotate(x); // ZIG-ZAG: сносим
      rotate(x);
    }
  }
  return x; // теперь x — корень
}

// поиск: спустились и подняли найденное наверх
function find(root, key) {
  let cur = root, last = null;
  while (cur) {
    last = cur;
    if (key === cur.key) break;
    cur = key < cur.key ? cur.left : cur.right;
  }
  return last ? splay(last) : null; // splay делаем
}

```

<p class="text-xs text-slate-400">Вся работа происходит в функции splay(x);

 против своей

 т свою оценку.</p>

</div>

</details>

```

        еворачивают бамбук в другую сторону (см. бл
        </div>
        <div>
            <strong class="text-white block mb-
            <p class="text-slate-400">Чередовани
        вает один из них в корень, превращая дерево
        ступает только для <em>одной</em> конкретной
        </div>
        <div>
            <strong class="text-white block mb-
            <p class="text-slate-400">Треар дер
        play не хранит вообще ничего и держит балан
        p>
        </div>
    </div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сжатие суммы подматрицы за  $O(1)$ ",
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-emerald-4
            <div class="bg-slate-900 p-4 rounded-lg mb-
                <p class="text-lg text-emerald-300 ital
                <p class="text-xl font-mono text-white"

```



```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
    Аналогия 1: Карта мира
  </div>
  <p class="text-slate-300 text-sm mb-0" style="color: #a6c1ee; font-size: 0.8em; margin-bottom: 0;">
    . Границы не пересекаются в одной точке по-
    ы не сливались цветами, Всегда можно всего
  </div>
</div>
</div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border" style="background-color: #1e293b; color: #a6c1ee; padding: 10px; border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
      <h3 class="text-xl font-bold text-blue-400" style="color: #4299e1; font-size: 1.2em; margin-bottom: 10px;">
      <div class="bg-slate-900 p-4 rounded-lg mb-0" style="background-color: #0d1b2a; color: #a6c1ee; padding: 10px; border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
        <p class="text-lg text-blue-300 italic" style="color: #4299e1; font-size: 1.1em; margin-bottom: 10px;">
        <p class="text-xl font-mono text-white" style="font-family: monospace; color: #f0f0f0; margin-bottom: 10px;">
        перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2" style="display: grid; grid-template-columns: 1fr 1fr; gap: 10px;">
        <div class="bg-slate-800 p-6 rounded-lg" style="background-color: #1e293b; color: #a6c1ee; padding: 10px; border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">

```

```

</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Улицы с односторон
    </div>
    <p class="text-slate-300 text-sm mb
БЫХ направлениях по односторонним улицам. К
у на карте.</p>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Рёбра, удаление которых расхреначивает",
    category: "Графы. Продвинутое",
    content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-r
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border
      <h3 class="text-xl font-bold text-rose-400 r
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic r
        <p class="text-xl font-mono text-white">
и  $fup[v] > tin[u]$ .</p>
      </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg

```

```

        <p class="text-slate-300 text-sm mb-4">
            правильный роутер - и два сегмента офиса б
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
        id: "graph-euler",
        title: "13. Графы. Эйлеров цикл",
        type: "html",
        description: "Пройти по всем ребрам ровно 1 раз.",
        category: "Графы",
        content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-indigo-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-indigo-300 italic">
                <p class="text-xl font-mono text-white">
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 b
                        order border-emerald-500 shadow-md">
                        Аналогия 1: Рисование не отрыв
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        е сходится нечетное число линий, значит, из
                        а везде должно быть четное число (зашел-выш
                    </div>
                </div>
            </div>
        </div>
    </div>
`
    }
]
```

```

    },
    {
      id: "mst-kruskal",
      title: "18. Графы. Остовное дерево. Краскал",
      type: "html",
      description: "",
      category: "Графы. Остовные",
      content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-emerald-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">
              <p class="text-xl font-mono text-white">
                (проверяем через DSU) - берем в ответ.</p>
              </div>
            <div class="grid grid-cols-1 gap-6">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                  Аналогия: Прокладка интернет
                </div>
                <p class="text-slate-300 text-sm mb-4">
                  с самых дешевых дорог в стране". Он строит
                  единую сеть.</p>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "mst-prima",

```

```

        category: "Графы. Остовные",
        content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-4">
            изкому племени для объединения. К вечеру пле
            ства шлют гонцов к ближайшему чужому царству.
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
      },
      {
        id: "string-kmp",
        title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">

```



```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Судoku (P vs NP)
    </div>
    <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
  </div>
  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
      Аналогия 2: Машина-переводчик
    </div>
    <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведен
аче В, то В – это <b>NP-Полная</b> (сосредо
  </div>
</div>
</div>
</section>`,
  },
];

```

ФАЙЛ 18/87: src/data/content_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМ

```

export interface Chapter {
  id: string;
  title: string;

```

```

        финиш) .
    </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
    <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
        <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
            Путь: Нормальная прогулка
        </div>
        <p class="text-slate-300 text-sm mb-4">
            Ты вышел из дома <strong>(нечётная)</strong>.
            талый пришёл в бар <strong>(вторая нечётная)</strong>.
            Домой вернуться не смог, потому что
        </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md bg-slate-950">
        <div class="absolute -top-3 left-4 bg-slate-900 border border-rose-500 shadow-md">
            Цикл: Гамильтонов синдром (болезнь)
        </div>
        <p class="text-slate-300 text-sm mb-4">
            <strong>Гамильтон – это болезнь.</strong>
            ng> ровно раз и вернуться домой.
            Ты обходишь все бары (вершины) ровно
            Главное – зайти и выйти, не заходя
            Никто не знает, как решать быстро.
            <br><span class="text-xs text-rose-500">
                лноту).</span>
            </p>
        </div>
    </div>

</div>

<details class="bg-slate-900/40 rounded-lg border border-rose-500 shadow-md">
    <summary class="p-4 font-bold text-slate-400">
        order-slate-700/50">
            Душный режим: Формальное доказательство

```



```

        <span class="text-rose-400 font-
        <p class="text-xl font-bold text-
        <p class="text-xs text-slate-400
    </div>
  </div>
</div>

<p class="text-slate-300 text-sm">
  Если из сына u и всех его потомков <strong>
  Единственная ниточка. Порвётся – граф р
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-
  <div class="bg-slate-800 p-6 rounded-lg border-
    <div class="absolute -top-3 left-4 bg-slate-800
    border-emerald-500 shadow-md">
      Мост: Единственная веревка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь, что ты альпинист. Ты спустился в пещеру.
      Из пещеры и нет других выходов – только вверх.
      Если верёвка (ребро v-u) оборвётся – ты застрянешь.
      Если бы из пещеры и был чёрный ход, ты бы вышел.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
    Аналогия: Кровеносный сосуд
  </div>
  <p class="text-slate-300 text-sm mb-4">
    Граф – это кровеносная система. Ребра – артерии и вены.
    (Атеросклероз – это не мост, живём.
    Если же перерезать единственную артерию, то организм умрёт.
    Алгоритм DFS ищет такие «критические» ребра.
  </p>
</div>
</div>
</div>
```

```

        low[v] = min(low[v], low[to]);

        if (low[to] > tin[v]) {
            // ЭТО МОСТ!
            // bridge_list.push_back({v, to});
        }
    }
}
}

```

```

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}

```

```

</div>
<p class="text-xs text-slate-400">Занес
</div>
</details>
</div>
</section>`,
},
{
    id: "planarity-euler-formula",
    title: "Планарность: K5, K3,3 и формула Эйлера",
    type: "html",
    content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">

```

КЗ,3: 3 дома и 3 колодца

</div>

<p class="text-slate-300 text-sm mb-4">

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него <code class="text-white"> $V=6$

<code class="text-white"> $9 \leq 12$ </code> – проходит, но он в

Почему? Потому что у двудольного графа

Отсюда более строгое ограничение: <code class="text-white">

<code class="text-white"> $9 \leq 8$ </code>

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border">

<summary class="p-4 font-bold text-slate-400">

order-slate-700/50">

Доказательство непланарности K5 (Скрыть)

</summary>

<div class="p-5 text-sm text-slate-300 space-y-4">

<p class="text-blue-400">Доказательство

<p>

Пусть K5 планарен. $V = 5$, $E = 10$.

<code> $F = E - V + 2 = 10 - 5 + 2 = 7$

<code>Каждая грань ограничена минимумом

<code>Сумма длин граней $= 2E = 20$.

<code>Отсюда: $2E \geq 3F \Rightarrow 20 \geq 21$ – <code>

<code>Значит, K5 не может быть планарным.

</p>

</div>

</details>

</div>

</section>`,

},

{

id: "graph-articulation",

title: "12. Графы. Точки сочленения",

type: "html",

её перекроют (удалят вершину) – районы буд
>хрупкость системы.

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border-
<div class="absolute -top-3 left-4 bg-em
er border-emerald-500 shadow-md">

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной
абелем (мостом). Сами свитчи – это точки со

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border-
<summary class="p-4 font-bold text-slate-400
order-slate-700/50">

Душный мод: Код и корень DFS (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-

<p class="text-blue-400 font-bold">Особ

<p>

Для стартовой вершины обхода правил
то выше неё прыгать некуда). Поэтому для не
у него больше 1 независимого ребенка

</p>

<div class="bg-slate-950 p-4 rounded-lg

<pre class="text-xs font-mono text-

```
void dfs(int v, int p = -1) {  
    visited[v] = true;  
    tin[v] = low[v] = timer++;  
    int children = 0;
```

```
    for (int to : adj[v]) {  
        if (to == p) continue;
```

ФАЙЛ 19/87: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph-bridges
const toRemove = ["graph-planar-colors", "graph-bridges"];

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
  if (!toRemove.includes(c.id)) {
    // We prefer the newer versions (like tickets14_20)
    dedup.set(c.id, c);
  }
});

// Add the new ones
if (newChapters) {
  newChapters.forEach(c => dedup.set(c.id, c));
}
```

```

func: string;
/** Локальные переменные: имя → герг (усечённый), для
locals: Record<string, string>;
/**
 * JSON-снимок доступных имён (globals + locals). В о
 * можно напрямую отдать визуализации: i/j остаются ч
 */
values?: Record<string, DebugValue>;
}

export interface DebugResult {
  steps: DebugStep[];
  /** Локалы в момент возврата из модуля – «результат»
finalLocals?: Record<string, string>;
  /** Финальный машинно-читаемый снимок для визуализации
finalValues?: Record<string, DebugValue>;
  /** Текст последней необработанной ошибки (или пустая
error: string;
  /** true, если уперлись в лимит шагов. */
  truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
  const srcLiteral = JSON.stringify(userSrc); // JSON-с
  return `import sys as __sys, json as __json

__src = ${srcLiteral}
__steps = []
__final = {}
__final values = {}

```

```
def __capture(frame):
    local_values = __visible(frame.f_locals.items())
    # Внутри функции алгоритма важные структуры (memo,
    # глобальные. Локальные значения имеют приоритет на
    scope_values = __visible(frame.f_globals.items())
    scope_values.update(local_values)
    loc = {k: __safe(v) for k, v in local_values.items()}
    values = {k: __snapshot(v) for k, v in scope_values.items()}
    return loc, values

def __tracer(frame, event, arg):
    global __was_truncated
    if event == "line" and frame.f_code.co_filename == __main__.__file__
        and not (frame.f_code.co_name.startswith("<
        loc, values = __capture(frame)
        # PEP 709: с 3.12 компрекхеншны встроены во фрейм
        # на своей строке каждую итерацию. Склеиваем по
        if __steps and __steps[-1]["line"] == frame.f_lineno:
            __steps[-1]["locals"] = loc
            __steps[-1]["values"] = values
        else:
            __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
        if len(__steps) >= __CAP:
            # Нельзя просто отключить trace: бесконечный цикл
            # тогда продолжит работать вечно. Мягко прерываем
            __was_truncated = True
            raise __TraceLimit()
    # Финальные локалы программы (после последней строки)
    if event == "return" and frame.f_code.co_filename == __main__.__file__:
        loc, values = __capture(frame)
        __final.update(loc)
        __final_values.update(values)
    return __tracer

__err = ""
try:
    sys.settrace(__tracer)
```

```
/**
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
export function orderVars(
  cur: Record<string, string>,
  syncNames: string[]
): [string, string][] {
  const inSync = new Set(syncNames);
  const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
  const rest = Object.entries(cur)
    .filter(([k]) => !inSync.has(k))
    .sort(([a], [b]) => a.localeCompare(b));
  return [...syncEntries, ...rest];
}
```

ФАЙЛ 21/87: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 626 | РАЗМЕР: 10,5 КБ

```
import type { DemoKind } from "../components/hints/MiniHints";

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
  id: string;
  /** Варианты написания в тексте (регистр не важен). */
  labels: string[];
  title: string;
  /** Объяснение «на пальцах» — 1–2 фразы. */
}
```



```

labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
title: "Куча (priority queue)",
short:
    "Не отсортированный список, а «мешок, из которого",
formal:
    "Полное бинарное дерево в массиве: дети узла  $i$  лежат в узлах  $2i$  и  $2i+1$ ",
complexity: "вставка/извлечение  $O(\log n)$ , «посмотреть элемент  $O(1)$ ",
demo: "heap",
simulator: "heap-beam-search",
},
{
    id: "stack",
    labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
    title: "Стек (LIFO)",
    short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
    formal: "push / pop / top за  $O(1)$ . На стеке живёт рекурсия",
    demo: "stack",
    simulator: "stack-dfs",
},
{
    id: "queue",
    labels: ["очередь", "очереди", "очередью", "FIFO"],
    title: "Очередь (FIFO)",
    short: "Очередь в столовой: кто первым встал, тот первым и ест",
    formal: "push_back / pop_front за  $O(1)$ . Основа BFS",
    demo: "queue",
    simulator: "queue-bfs",
},
{
    id: "dsu",
    labels: ["DSU", "СНМ", "система непересекающихся множеств"],
    title: "DSU – система непересекающихся множеств",
    short:
        "«Кто твой староста?» Каждый элемент знает своего старосту",
    formal:
        "find со сжатием путей + union по рангу/размеру. Алгоритм Крускала",
    complexity: "почти  $O(1)$  – обратная функция Аккермана",
    demo: "dsu",
}

```

```

    id: "suffix-link",
    labels: ["суффиксная ссылка", "суффиксные ссылки",
    title: "Суффиксная ссылка",
    short:
        "«Куда откатиться, если следующая буква не подошла",
        строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топ",
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с",
        циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
        ,
    formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "пре",
    title: "Префиксные суммы",

```

```

    "Ребро, после удаления которого граф разваливается.
formal: "Ребро (u,v) — мост  $\text{low}[v] > \text{tin}[u]$ , где
    ребро в родителя — нет).",
complexity: "O(V + E) одним DFS",
demo: "bridge",
simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "т"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
    formal: "Для не-корня: v — точка сочленения  $\exists$  ребро",
    demo: "articulation",
    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная",
    formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
        "но).",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизир",
        "амортизированное", "амортизированного", "амортизир",
    ],
    title: "Амортизированная сложность",

```

```

    demo: "scc",
    simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функ",
        "префикс", "префикса", "суффиксом", "суффикс", "с",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция  $\pi$  (КМП)",
    short:
        "Для каждой позиции запоминаем: какой кусок начался",
        "от кусок.",
    formal:
        " $\pi[i]$  — длина наибольшего собственного префикса  $s$ ",
        "возвращая указатель по тексту.",
    complexity: " $O(n + m)$ ",
    demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "z-function",
    labels: ["Z-функция", "Z-функции", "Z-функцию", "z-ф",
    title: "Z-функция",
    short:
        "Для каждой позиции: сколько символов подряд, начинающихся",
        "с каждой буквы.",
    formal:
        " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i..n]$ ",
    complexity: " $O(n)$ ",
    demo: "prefix-function",
    simulator: "string-z-func",
},
{
    id: "dp",
    labels: [
        "динамическое программирование", "динамики", "дин",

```

```

        "самое дешевое ребро", "самое дешёвое ребро", "самое дешёвое ребро",
    ],
    title: "Жадный алгоритм",
    short:
        "На каждом шаге хватаем то, что лучше прямо сейчас",
    formal:
        "Корректность обычно доказывается «леммой о разрезе»",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова характеристика"],
    title: "Эйлеров путь и цикл",
    short:
        "Пройти по каждому ребру ровно один раз, не отрываясь",
    formal:
        "Связный граф: эйлеров цикл — все степени чётные; эйлеров путь — все степени чётные, кроме двух",
    simulator: "euler-path-vs-cycle",
},
{
    id: "planarity",
    labels: ["планарный", "планарность", "планарного", "планарность графа"],
    title: "Планарность",
    short:
        "Граф планарен, если его можно нарисовать на листе бумаги",
    formal:
        "Формула Эйлера:  $V - E + F = 2$  для связного планарного графа. Курсивом выделены вершины, рёбра и области",
    simulator: "planarity-euler-formula",
},
{
    id: "splay",
    labels: [
        "splay", "splay-дерево", "AVL", "AVL-дерево", "всё",
    ],
    title: "Splay и повороты",

```

```

        "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
        complexity: "O(V · E)",
        demo: "relax",
        simulator: "bellman-ford",
    },
    {
        id: "floyd",
        labels: [
            "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
            "промежуточная вершина", "промежуточную вершину",
        ],
        title: "Флойд-Уоршелл: разрешаем вершину k",
        short:
            "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
        formal:
            "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок вершин произвольный.",
        complexity: "O(V³) времени, O(V²) памяти",
        demo: "floyd",
        simulator: "floyd",
    },
    {
        id: "prim",
        labels: ["Прима", "Прим", "Prim"],
        title: "Алгоритм Прима",
        short:
            "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
        formal:
            "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
        complexity: "O(E log V)",
        demo: "mst",
        simulator: "mst-prim",
    },
    {
        id: "boruvka",

```

```
{
  id: "graph",
  labels: [
    "граф", "графа", "графе", "графы", "графов", "граф",
    "вершины", "вершина", "вершин", "ребра с весом",
  ],
  title: "Граф",
  short:
    "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
  formal:
    " $G = (V, E)$ . Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двусторонняя улица).",
  demo: "graph",
  simulator: "graph-dfs-bfs",
},
{
  id: "coord-compress",
  labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
  title: "Сжатие координат",
  short:
    "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Сжимаем координаты в один порядковый номер.",
  formal:
    "sort + unique + binary search: значение → индекс",
  complexity: " $O(n \log n)$ ",
  demo: "coord-compress",
},
{
  id: "treap",
  labels: ["Treap", "декартово дерево", "декартова д-во"],
  title: "Treap = Tree + Heap",
  short:
    "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
  formal:
    "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить). Ожидаемая высота  $O(\log n)$ .",
  complexity: "ожидаемо  $O(\log n)$ ",
}
```

```

    demo: "np",
  },
  {
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
    title: "Запрос на отрезке",
    short:
      "Спрашиваем не про один элемент, а про кусок массива  

      предподсчёты.",
    formal:
      "Суммы — префиксные суммы  $O(1)$ . Минимум без изменений.",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "строкам"],
    title: "Строка как массив символов",
    short:
      "Строковые алгоритмы держатся на трёх вопросах: где  

      (Z-функция) и где встречаются слова словаря (бордер-лист).",
    formal:
      "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция —  $O(N)$ .",
    demo: "prefix-function",
    simulator: "string-kmp",
  },
  {
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворотом"],
    title: "Поворот (rotation) в дереве",
    short:
      "Локальная перевеска двух узлов: ребёнок встанет на  

      дившемся месту. Порядок ключей при этом не ломается.",
    formal:
      "Правый поворот вокруг p:  $x = p.left; p.left = x$ .  

      g и Zig-Zag.",
    complexity: "O(1)",
    demo: "zig",
  },

```


Универсальное пособие • полный исходный код

```
-----  
/** Быстрый доступ по id. */  
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {  
  
/** Все метки, отсортированные от длинных к коротким (ч  
export const GLOSSARY_LABELS: { label: string; id: string }[] =  
  e.labels.map((label) => ({ label, id: e.id }))  
}).sort((a, b) => b.label.length - a.label.length);  
  
-----
```

ФАЙЛ 22/87: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 275 | РАЗМЕР: 1.2 КБ

```
-----  
import { Chapter } from "../types";  
  
export const graphChapters: Chapter[] = [  
  {  
    id: "intro",  
    title: "Обзор: Кратчайшие пути и Графы",  
    type: "html",  
    category: "Графы",  
    content: `  
      <section id="intro-content" class="mb-12">  
        <div class="flex items-center mb-6">  
          <span class="bg-indigo-600 text-white px-4 py-1 text-sm font-medium">Обзор</span>  
          <h2 class="text-3xl font-bold text-white">Обзор</h2>  
        </div>  
        <div class="space-y-8">  
          <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">  
            <h3 class="text-xl font-bold text-blue-400">Кратчайшие пути</h3>  
            <p class="text-slate-300 text-sm mb-4">Представьте себе карту города. Как найти самый короткий путь от дома до работы? Это классическая задача на поиск кратчайшего пути (Shortest Path Problem). Алгоритмы поиска кратчайшего пути применяются в навигаторах, социальных сетях (рекомендации друзей) и многих других областях.  
            <div class="mt-8 mb-4">  
              <div id="slot-mnemonic-cards"></div>  
            </div>  
          </div>  
        </div>  
      </section>`  
  }  
];
```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
        охождение улицы), Дейкстра сломается, так как не может
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border border-slate-700/50"
        <summary class="p-4 font-bold text-slate-300">
        -b border-slate-700/50">
            Строгое доказательство (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <p>Алгоритм использует приоритетную очередь (например,
            g V). Гарантирует корректность только для неориентированных
            </div>
        </details>

    <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>`
},
{
    id: "bellman-ford",
    title: "Билет 15. Форд-Беллман",
    type: "html",
    category: "Графы",
    content: `<section id="bellman-ford-content" class="text-slate-300">
    <div class="flex items-center mb-6">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Билет 15
        <h2 class="text-3xl font-bold text-white">Форд-Беллман
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
            <h3 class="text-xl font-bold text-rose-400">Описание
        </div>

        <div class="bg-slate-900 p-4 rounded-lg mb-6">

```

```
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-3xl font-bold text-white">Флойд
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border" style="border-color: #000000; border-width: 2px;">
    <h3 class="text-xl font-bold text-indigo-400">Аналогия 1: Автомагистрали

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
      Аналогия 1: Автомагистрали
    </div>
    <p class="text-slate-300 text-sm mb-4">
      евле долететь из Москвы в Париж, если мы сд
    <div id="slot-chapter-image-floyd" class="h-40">
    </div>
```

```
<!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
      Время работы
    </div>
    <p class="text-slate-300 text-sm mb-4">
      три вложенных цикла. Сложность  $O(V^3)$ . Если
    </div>
  </div>
```

```
<div id="slot-floyd-viz" class="mt-8"></div>
```

```

    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2" style="border-color: #000000; border-radius: 10px; border-style: solid; border-width: 2px; margin: 10px 0 10px 10px;">
        <div class="absolute -top-3 left-4 bg-rose-900 -500 shadow-md">
            Аналогия 2: Матёшка (Терминальные)
        </div>
        <p class="text-slate-300 text-sm mb-4">Представим, что мы нашли и "he", потому что оно спрятано внутри терминальные ссылки</b> (term_link) – прямые мосты
        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700" style="border-radius: 10px; margin: 10px 0 10px 10px;">
    <summary class="p-4 font-bold text-slate-400 outline outline-slate-700">
        Внутренняя структура автомата и код BFS (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-4">
        <p>Каждая вершина имеет таблицу переходов <code>transitions</code> и ссылку <code>term_link</code> (ближайшая терминальная вершина)
        <div class="bg-black/50 p-4 rounded-lg font-mono text-sm">
struct Node {
    map<char, int> go;          // Предвычисленные переходы
    int link = 0;              // Суффиксная ссылка: куда перейти по префиксу
    int term_link = 0;         // Терминальная ссылка: матёшка
    bool is_terminal = false;
    vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные ссылки
void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {

```

```
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border"
        <h3 class="text-xl font-bold text-purple-400">
          <p class="text-slate-300 text-sm mb-4">Когда
            ут быть большими дробными числами (долгота
            вы от 0 до N, и уже по ним строим графы.</p>
        </div>
      </div>
    </section>`
  }
};
```

ФАЙЛ 23/87: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМ

```
export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
  explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
  "euler-path-vs-cycle": [
    {
      question: "В связном графе ровно 2 вершины с нечётными степенями",
      options: [
        "Есть эйлеров цикл (вернусь домой!)",
        "Есть эйлеров путь (из одной нечётной в другую)",
        "Ни пути, ни цикла нет — это полный провал"
      ],
      correctIndex: 1,
      explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше 2 — эйлеров цикла нет, заканчиваем в другой."
```

```

        "Для того чтобы было больше переменных в коде",
        "Чтобы знать минимальный tin предка, до которого",
        "Чтобы хранить глубину рекурсии"
    ],
    correctIndex: 1,
    explanation: "low[v] хранит минимальное время входа в вершину v при
        поиске мостов и точек сочленения."
},
{
    question: "Какова временная сложность алгоритма поиска мостов?",
    options: [
        "O(V^2) – ищем мосты в квадрате",
        "O(V + E) – каждый элемент один раз",
        "O(2^V) – полный перебор"
    ],
    correctIndex: 1,
    explanation: "DFS обходит каждую вершину и каждое ребро."
},
"planarity-euler-formula": [
    {
        question: "Планарный граф имеет V=6, E=12. Возможно ли?",
        options: [
            "Да, если это двудольный граф",
            "Нет, нарушается неравенство E ≤ 3V - 6",
            "Да, если нет треугольных граней"
        ],
        correctIndex: 1,
        explanation: "E ≤ 3V - 6 ⇒ 12 ≤ 12 – на грани применимости.
            Так что такой граф непланарен."
    },
    {
        question: "Сколько граней у планарного графа с V=6?",
        options: [
            "3 грани",
            "5 граней",
            "10 граней"
        ],
    },

```

```
export interface VizStateEventDetail {
  chapterId: string;
  /** Строка и функция реально выполненного пользователем */
  line?: number;
  func?: string;
  /** Машинно-читаемые globals + locals: числа остаются */
  variables?: Record<string, DebugValue>;
  /** Имена, изменённые выделенной строкой. */
  changed?: string[];
}

/** Последний снимок хранится, чтобы поздно смонтировать
const latestByChapter = new Map<string, VizStateEventDetail>();

/** Транслировать реальные переменные визуализациям текста
export function emitVizState(
  chapterId: string,
  snapshot: Omit<VizStateEventDetail, "chapterId">
) {
  if (typeof window === "undefined") return;
  const detail: VizStateEventDetail = { chapterId, ...snapshot };
  latestByChapter.set(chapterId, detail);
  window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>("viz-state", { detail }));
}

/** Вернуть демонстрацию в ручной режим при закрытии/перезагрузке
export function clearVizState(chapterId: string) {
  if (typeof window === "undefined") return;
  latestByChapter.delete(chapterId);
  window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>("viz-clear", { chapterId }));
}

/** Текущий снимок компилятора для прямой отрисовки масива
export function useVizRuntime(): VizStateEventDetail | null {
  const chapterId = useContext(VizChapterContext);
  const [runtime, setRuntime] = useState<VizStateEventDetail>();
  return chapterId ? latestByChapter.get(chapterId) ?? null : null;
};
```

```

const requested =
  selectByVariables && detail.variables ? selectByVariables : null
// null означает: компонент рисует runtime-переменные
// не увидел знакомых захардкоженных имён.
if (requested === null || !Number.isFinite(requested)) {
  const target = Math.max(0, Math.min(Math.max(0, maxStep), requested));
  if (target !== currentStep) goToStep(target);
};

apply(latestByChapter.get(chapterId));
const handler = (e: Event) => apply((e as CustomEvent).detail);
window.addEventListener(CHANNEL, handler);
return () => window.removeEventListener(CHANNEL, handler);
}, [chapterId, currentStep, maxStep, goToStep, selectByVariables]);
}

/** Утилиты без небезопасных cast-ов для визуализаторов */
export const vizNumber = (value: DebugValue | undefined) : number | null {
  typeof value === "number" && Number.isFinite(value) ? value : null;
}

export const vizString = (value: DebugValue | undefined) : string | null {
  typeof value === "string" ? value : null;
}

export const vizArray = (value: DebugValue | undefined) : Array<any> | null {
  Array.isArray(value) ? value : null;
}

export const vizRecord = (value: DebugValue | undefined) : Record<string, DebugValue> | null {
  value !== null && typeof value === "object" && !Array.isArray(value)
    ? (value as Record<string, DebugValue>)
    : null;
}

/* — Выбор демонстрации внутри страницы (вкладки ID / ID) */

const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
  chapterId: string;
}

```


Универсальное пособие • полный исходный код

```
-----
* которые понимает её визуализация: i, j, k, v, dist, l
* Компилятор выполняет любой Python-код и передаёт реа
* имён напрямую; привязки к номерам или тексту строк э
*/
```

```
export interface SyncVariable {
  /** Имя переменной, как в коде визуализации: i, j, k,
  name: string;
  /** Роль в алгоритме (показывается во всплывающей под
  role: string;
  /** Характерный диапазон/значение на демонстрации. */
  range?: string;
}
```

```
export interface PageSync {
  /** Заголовок демонстрации на странице (если интеракт
  vizTitle?: string;
  /** Чему соответствует один шаг визуализации (для тул
  stepNote?: string;
  /** Переменные синхронизации. Пусто – на странице нет
  variables: SyncVariable[];
  /** Полный референсный Python-код, который кнопка-гла
  code?: string;
}
```

```
/** Справочные значения, указанные внутри самих симулято
```

```
export const PAGE_SYNC: Record<string, PageSync> = {
  "segment-trees": {
    vizTitle: "Дерево отрезков",
    stepNote: "Шаг = спуск по вершине v: либо читаем тр
    code: `n = 8                                # длина массива
```

```
tree = [0] * (2 * n)
i = 0                                # лист: tree[n + i]
```

```
v, l, r = 1, 0, n - 1                # вершина и её отрезок
m = (l + r) // 2                       # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
  variables: [
```

```

        { name: "p", role: "родитель узла x перед поворотом", range: 1,
          { name: "g", role: "дедушка узла x — нужен в случае поворота", range: 1,
        },
    },
    "sparse-table": {
        vizTitle: "Разреженная таблица (1D и 2D)",
        stepNote: "Шаг = одна ячейка  $st[i][j] = \min(st[i][j - 1], st[i + (1 \ll (j - 1))][j - 1])$ ",
        code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
    st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])
    variables: [
        { name: "n", role: "длина массива / сторона квадрата", range: 1,
        { name: "k", role: "уровень таблицы: ячейка хранит минимум из k предыдущих", range: 1,
        { name: "i", role: "начало блока в строке уровня", range: 1,
        { name: "j", role: "номер уровня при построении (от 0 до k)", range: 1,
        { name: "r, c", role: "строка и столбец ячейки в таблице", range: 1,
        { name: "kx, ky", role: "уровни по вертикали и горизонтали", range: 1,
    ],
},
    "prefix-sums-2d": {
        vizTitle: "Префиксные суммы (1D и 2D)",
        stepNote: "Шаг = заполнение очередного  $P[i] = P[i - 1] + a[i - 1]$ ",
        code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
    P.append(P[i - 1] + a[i - 1])
print("P =", P)`
    variables: [
        { name: "n", role: "число строк матрицы", range: 1,

```

```

-----
i = n - 1                                # всплываемый элемент

parent = (i - 1) // 2                    # родитель
kids = (2 * i + 1, 2 * i + 2)           # дети
print(f"i={i} parent={parent} kids={kids}")`,
    variables: [
        { name: "n", role: "текущее число элементов в куче" },
        { name: "i", role: "индекс элемента, который сейчас обрабатываем" },
        { name: "(i-1)//2", role: "индекс родителя вершины" },
        { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка" },
    ],
},
"stack-dfs": {
    vizTitle: "Стек и обход в глубину",
    stepNote: "Шаг = push новой вершины на стек или pop",
    code: `st = []                        # стек = рекурсия
st.append("A")                          # push — зашли в вершину
st.append("B")
top = st.pop()                          # pop — возврат на уровень выше
print("pop →", top, "стек:", st)`,
    variables: [
        { name: "top", role: "вершина стека — то, откуда мы вышли" },
        { name: "v", role: "текущая вершина графа, которую обрабатываем" },
    ],
},
"queue-bfs": {
    vizTitle: "Очередь и обход в ширину",
    stepNote: "Шаг = dequeue вершины из головы очереди",
    code: `from collections import deque

q = deque(["A"])
q.append("B")                            # enqueue соседа
v = q.popleft()                          # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
    variables: [
        { name: "front", role: "голова очереди — вершина, которую обрабатываем" },
        { name: "v", role: "вершина, извлечённая из очереди" },
        { name: "u", role: "сосед вершины v, которого кладем в очередь" },
    ],
},

```

```

// отдельного виджета нет, тема опирается на DFS/BFS
variables: [],
},
"top-sort": {
  vizTitle: "Топологическая сортировка",
  stepNote: "Шаг = выход рекурсии из вершины v: она д
  code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2],
used, order = set(), []

def dfs(v):
  used.add(v)
  for to in adj[v]:      # шаг: ребро v → to
    if to not in used:
      dfs(to)
  order.append(v)        # выход из v!

for v in adj:
  if v not in used:
    dfs(v)
print(order[::-1])      # разворот – ответ`,
variables: [
  { name: "v", role: "текущая вершина DFS", range:
  { name: "to", role: "вершина, куда ведёт ребро из
  { name: "i", role: "перебор стартовых вершин во в
  { name: "order", role: "список выхода из рекурсии
  ],
},
"scc-kosaraju": {
  vizTitle: "Компоненты сильной связности (Косарайю)"
  stepNote: "Шаг = вершина из order (в обратном поряд
  code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [], 4:
adjT = {}
for v in adj:
  adjT[v] = []
for v in adj:
  for to in adj[v]:
    adjT[to].append(v) # транспонирование

```

```

-----
timer = 3                                # как на демо

v, to = "B", "G"                         # шаг: возврат из G
is_bridge = low[to] > tin[v]              # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
    variables: [
        { name: "v", role: "текущая вершина DFS", range:
        { name: "to", role: "сосед; ребро (v, to) проверя
        { name: "tin[v]", role: "время входа в вершину –
        { name: "low[v]", role: "минимум tin, куда можно
        { name: "timer", role: "глобальный счётчик времени
    ],
},
"euler-path-vs-cycle": {
    vizTitle: "Эйлеров путь и цикл",
    stepNote: "Шаг = один клик по следующей вершине мар
    code: `edges = [(0,1), (0,2), (1,3), (2,4), (1,2),
deg = {}
for u, v in edges:                       # степени вершин
    deg[u] = deg.get(u, 0) + 1
    deg[v] = deg.get(v, 0) + 1

odd = []
for v in deg:
    if deg[v] % 2:
        odd.append(v)
print(deg, "нечётных:", len(odd))
# 0 → цикл • 2 → путь • иначе – нельзя`,
    variables: [
        { name: "path", role: "текущий маршрут – последов
        { name: "last", role: "последняя вершина пути – о
        { name: "deg(v)", role: "степень вершины: чётная
    ],
},
"planarity-euler-formula": {
    vizTitle: "Планарность: K5 и K3,3",
    stepNote: "Шаг = попытка перетащить вершину так, что
    code: `V, E = 5, 10                    # K5      (K3,3: V=

```

```

        { name: "v", role: "сосед вершины u, до которого",
        { name: "w", role: "вес ребра (u, v)", range: "по",
        { name: "dist[v]", role: "лучшее известное рассто
    ],
},
"bellman-ford": {
    vizTitle: "Форд–Беллман",
    stepNote: "Шаг = одна релаксация ребра (u, v, w) в",
    code: `n, m = 7, 10                                # вершин, рёбер
dist = {"S": 0}

for i in range(1, n):      # итерация i = 1 ... n-1
    u, v, w = "S", "A", 5  # шаг: очередное ребро
    if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
        dist[v] = dist[u] + w  # релаксация
    if i == 1:
        print(f"итерация {i}: {dist}")
        break`
    variables: [
        { name: "n", role: "число вершин графа – ровно n-1",
        { name: "m", role: "число рёбер, по которым проходят",
        { name: "i", role: "номер итерации (после i итераций)",
        { name: "u, v, w", role: "текущее ребро: откуда, куда, вес",
        { name: "dist[v]", role: "расстояние; если обновлено, то новое",
    ],
},
floyd: {
    vizTitle: "Флойд–Уоршелл",
    stepNote: "Шаг = инициализация матрицы, смена промежуточного",
    code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
      [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0],
n = 7
for k in range(n):
    print(f"посредник k={k}")
    for i in range(n):
        for j in range(n):

```

```

-----
for v in "ABC":
    parent[v] = v
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

mst = []
for w, u, v in edges:          # шаг: очередное ребро
    if find(u) != find(v):     # разные компоненты → берём
        parent[find(u)] = find(v)
        mst.append((w, u, v))
print(mst)\,
    variables: [
        { name: "n", role: "число вершин; в остов войдёт n",
        { name: "m", role: "число рёбер, сортируем по весу",
        { name: "i", role: "ребро сортируем по весу w; ра",
        { name: "parent[v]", role: "DSU: кто представитель",
        { name: "rank[v]", role: "высота дерева DSU – по k",
    ],
},
"mst-prim": {
    vizTitle: "Прим",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

start = "A"
heap = [(0, start, "start")]    # (вес, вершина, откуда)

w, v, parent = heapq.heappop(heap) # шаг: берём минимум
taken = {start}                 # дерево растёт от start
print(f"берём {v} за {w} (из {parent})")\,
    variables: [
        { name: "v", role: "вершина, которую только что д",
        { name: "u", role: "ещё не взятый сосед v – канди",
        { name: "w", role: "вес ребра (v, u) – ключ в куче",
        { name: "start", role: "стартовая вершина, из неё",
    ],

```

```

    { name: "i", role: "индекс текущего символа — счи
    { name: "j", role: "длина текущего наилучшего сов
    { name: "π[i]", role: "префикс-функция: длина наи
        ица" },
    ],
},
"string-z-func": {
    vizTitle: "Z-функция",
    stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]"
    code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0          # правый Z-блок [l, r]
z = [None] * n            # пустые клетки: значения p
z[0] = 0

for i in range(1, n):     # шаг: вычисляем z[i]
    if i <= r:
        z[i] = min(r - i + 1, z[i - l]) # из блока
    else:
        z[i] = 0           # свежая клетка: с нуля
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1       # досчёт в лоб
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1
print(z)`
    variables: [
        { name: "n", role: "длина строки s", range: "n = "
        { name: "i", role: "позиция, для которой считаем .
        { name: "l, r", role: "правый крайний Z-блок: отр
        { name: "z[i]", role: "длина наибольшего общего п
    ],
},
"aho-corasick": {
    vizTitle: "Ахо–Корасик",
    stepNote: "Шаг = построение бора по образцам, затем
    code: `trie = {"": {}}          # бор: вершина →
state, c = "", "a"                # шаг: автомат идёт по симв

```



```

half = 1 << (k - 1)
for r in range(size):
    for c in range(size):
        st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k - 1),
                                   + half, c + half, k - 1, k - 1]))`,
variables: [
    { name: "r, c", role: "левый верхний угол блока, ",
    { name: "k", role: "уровень: блоки 2^k x 2^k (шаг",
    { name: "st2[(r,c,k,k)]", role: "минимум квадратн
    ],
},
"sparse-table#2d-query": {
    vizTitle: "Разрезанная таблица 2D: запрос",
    stepNote: "Демонстрация кликабельна вручную: выбери
    code: `# st2[(r, c, kx, ky)] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
    variables: [
        { name: "r1, c1, r2, c2", role: "углы запрашиваем
        { name: "kx, ky", role: "уровень самого крупного
    ],
},
"prefix-sums-2d#2d": {
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручную: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
    S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):

```

```
/**
 * Инициализация скелета: только объявление демо-данных
 * (без for/while/def/if/print). Именно это показывается
 * умолчанию – дальше пользователь пишет свой вариант,
 * заменяет редактор полным референсным кодом.
 */
function extractInit(code: string): string {
  const keep: string[] = [];
  for (const line of code.split("\n")) {
    const t = line.trim();
    if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
      keep.push(line);
    }
  }
  // убрать пустые строки с конца
  while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
    keep.pop();
  }
  return keep.join("\n");
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, demoId: string) {
  const sync = getPageSync(chapterId, demoId);

  const head = sync.vizTitle
    ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
    : `# «${chapterTitle}»\n# демо на странице нет – сводка`;

  if (!sync.code) return head;
  const init = extractInit(sync.code);
  return `${head}\n${init}\n`;
}
```

```

        <p class="text-slate-300 text-sm mb-4">
            ы рассылать 10 000 писем, он пишет одно письмо
            счетах сотрудников только тогда, когда соотв
            женное обновление).</p>
        </div>

        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 border-rose-500 shadow-md">
                Аналогия 2: Матрешки-коробки
            </div>
            <p class="text-slate-300 text-sm mb-4">
                ше, и так далее. Если нам нужно покрасить п
                Внутри всё синее!" на большую коробку. Опус
            </div>
        </div>

        <details class="bg-slate-900/40 rounded-lg">
            <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
                Строгое доказательство / Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
function push(node) {
    if (lazy[node] !== 0) {
        lazy[2 * node] += lazy[node];
        tree[2 * node] += lazy[node];
        lazy[2 * node + 1] += lazy[node];
        tree[2 * node + 1] += lazy[node];
        lazy[node] = 0;
    }
}</pre>
            </div>
        </details>
    </div>
</div>
</section>

```



```

} </pre>
      </div>
    </details>
  </div>
</div>
</section>`
  },
  {
    id: "splay-tree",
    title: "4. Splay-дерево",
    type: "html",
    description: "Дерево поиска, которое самобалансируется",
    category: "Продвинутые структуры",
    content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-blue-300 italic">
              <p class="text-xl font-mono text-white">
                (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border
                order border-emerald-500 shadow-md">
                  Аналогия 1: VIP-лифт
                </div>
                <p class="text-slate-300 text-sm mb-4">
                  вится корнем дерева (VIP-статус). Если ты ч
                  ка почти не требуется времени!</p>
                </div>
              <div class="bg-slate-900 p-6 rounded-lg"

```

```

        ска. За счет сдвига этого листа в корень, д
        /p>
        </div>

        <div>
            <strong class="text-white block">
                <p>Если агент постоянно переключо
                находящимися на разных концах дерева, его п
                ащая дерево в вытянутый бамбук для другого.
                . Постоянное свинг-переключение превратит пр
            </div>

            <div>
                <strong class="text-white block">
                    <p>Хотя один поиск может стоять
                    емах обладают прекрасным свойством: они не п
                    по пути. "Палочное" дерево прессуется в сб
                    осов.</p>
                </div>
            </div>
        </div>
    </div>
</section>`
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сжатие суммы подматрицы за  $O(1)$ ",
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
            <h2 class="text-2xl sm:text-3xl font-bold text-
        </div>
    <div class="space-y-8">

```

```

        content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Позитивное планирование
          </div>
          <p class="text-slate-300 text-sm mb-4">
(нельзя поехать и заработать). Он всегда в пути
          </div>
          <!-- Аналогия 2 -->
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
              Ограничения
            </div>
            <p class="text-slate-300 text-sm mb-4">
дный и не умеет пересматривать уже "зафиксированный"
            </div>
          </div>
          <div id="slot-dijkstra-viz" class="mt-8"></div>
        </div>
      </div>
    </section>`
  },

```

```

    </div>
</section>`
  },
  {
    id: "floyd",
    title: "16. Графы. Поиск кратчайшего пути. Флойд",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>

      <div class="bg-slate-800 p-6 rounded-lg border"
        <div class="absolute -top-3 left-4 bg-slate-700
          border-emerald-500 shadow-md">
          Суть фокуса (По шагам)
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Главный герой – внешний цикл
          шаг class="bg-slate-900 text-pink-400"
          шу себе проходить через вершину k?»</i>
        </p>
        <ul class="text-sm text-slate-300 space-y-2">
          <li><b>Шаг k=0:</b> Ты знаешь только
          <li><b>Шаг k=1:</b> Разрешаем пересадки
          е» через путь class="text-indigo-300"
          <li><b>Шаг k=2:</b> Разрешаем пересадки
          внутри этих путей уже могут быть пересадки ч

```



```

        <li>Добавляем <b>фиктивную верш
        <li>Запускаем от неё <b>медленно
b>«потенциалы»</b> <code class="text-pink-400">
        <li>Меняем старые веса: новое р
        <li><b>Магия!</b> Все веса тепер
ренние потенциалы сокращаются).</li>
        <li>Теперь смело запускаем <b>быстро
    </ol>
    </div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
  id: "mst-kruskal",
  title: "18. Графы. Остовное дерево. Краскал",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">

```

```
        Сеть растёт только из одного связного куска
      </div>
    </div>
  </div>
</section>`
},
{
  id: "mst-boruvka",
  title: "20. Графы. Остовное дерево. Борувка",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-4">
            изкому племени для объединения. К вечеру пле
            ства шлют гонцов к ближайшему чужому царству.
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`
}
```

```
      rder border-emerald-500 shadow-md">
        ♂ Аналогия 1: Поиск без отка
      </div>
      <p class="text-slate-300 text-sm mb
        Мы идём по строке слева направо.
строки СЕЙЧАС закончился под моим пальцем?»
      Представь, ты ищешь в тексте слов
x</code>. Тупой алгоритм начнет искать занов
</code>, а это начало нашего слова! Не надо
    </p>
  </div>

  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg
    <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
    Аналогия 2: LLM стриминг
  </div>
  <p class="text-slate-300 text-sm mb
    Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Пример вычисления (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300
    <p>Тот же пример: <b>abcabcd</b></p>
    <ul class="list-disc pl-5 space-y-2
      <li><b>a</b></li>: Префиксов нет. <co
      <li><b>ab</b></li>: Из начала ("a")
    >
      <li><b>abc</b></li>: Опять мимо. <co
```

```

<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
      <p class="text-xl font-mono text-white">
        са 0) и её суффиксом, начинающимся с i.</p>
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Линейка-шаблон
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Тут мы берём всю строку и «прикидываю
          я начну читать строку отсюда, насколько длин
          Это самый быстрый способ найти
        >, считаешь Z-функцию, и там, где <code>Z[i]
        </p>
      </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
          Аналогия 2: LLM Самокопираст
        </div>
        <p class="text-slate-300 text-sm mb-4">
          В LLM Z-функция может применяться
          [i]</code> (где <i>i</i> указывает назад на
          сама себя.

```

```

        </div>
    </div>
</section>`
    },
    {
        id: "aho-corasick",
        title: "23. Алгоритм Ахо-Корасик",
        type: "html",
        category: "Строки",
        content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-l">
      <h3 class="text-xl font-bold text-blue-400 mb-4">

      <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
        <p class="text-sm text-blue-300 italic mb-2">0с
        <p class="text-lg font-mono text-white">Бор (Тр
      </div>
      <p class="text-slate-300 text-sm">Позволяет найти
        го один проход.</p>
    </div>

    <!-- Аналогии -->
    <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg border bo">
        <div class="absolute -top-3 left-4 bg-slate-700
          emerald-500 shadow-md">
          Аналогия 1: Паутина (Бор)
        </div>
        <p class="text-slate-300 text-sm mb-4">Представ
          ет — мы падаем по <b>суффиксной ссылке</b> ("
        </div>

```

```

{
  id: "complexity-classes",
  title: "24. Классы сложности, сведение задач",
  type: "html",
  category: "Теория сложности",
  content: `
<section id="complexity-classes" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-purple-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">
        <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
              Аналогия 1: Судoku (P vs NP)
            </div>
            <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
          </div>
          <!-- Аналогия 2 -->
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
              rder border-purple-500 shadow-md">
                Аналогия 2: Машина-переводчик
              </div>
              <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведен

```

```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия: Матрица = Таблица
    </div>
    <p class="text-slate-300 text-sm mb
гупо, но проверка "знает ли Вася Петю" мгр
    </div>
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
        Аналогия: Списки = Контакты
      </div>
      <p class="text-slate-300 text-sm mb
т. Оптимально для почти всех задач.</p>
    </div>
  </div>
</div>

```

```

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-indigo-400"

  <div class="grid grid-cols-1 xl:grid-cols-2"
    <!-- Аналогия DFS -->
    <div class="bg-slate-800 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
        ♂ DFS (Поиск в глубину) = Жадный
      </div>
      <p class="text-slate-300 text-sm mb
        <b>Что делает:</b> Жадный алгоритм находит ближайшую
ую вершину (например, минимальную по номеру) и пробует другой путь.
      </p>
      <ul class="text-xs text-indigo-300"

```



```

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
  <h3 class="text-xl font-bold text-emerald-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          ество компонент.</p>
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
              Аналогия 1: Острова
            </div>
            <p class="text-slate-300 text-sm mb-4">
              карту – остались ли серые (неисследованные)
              ь через океан – столько и компонент.</p>
            </div>
          </div>
        </div>
      </div>
    </section>`,
    },
    {
      id: "top-sort",
      title: "9. Графы. Топологическая сортировка",
      type: "html",
      description: "Разложение задач по порядку выполнения",
      category: "Графы",
      content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
          <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

        visited[v] = true;
        for (int to : adj[v]) {
            if (!visited[to]) {
                dfs(to);
            }
        }
        ans.push_back(v); // Добавляем в момент ВЫХОДА (post-order)
    }
}

```

```

void topological_sort(int n) {
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
    reverse(ans.begin(), ans.end()); // Разворачиваем с
}

```

```

        </div>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая-Жару.",
        category: "Графы. Продвинутое",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">10. Графы. Компоненты сильной связности (SCC)</span>
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)</h2>
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
            <h3 class="text-xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)</h3>

```

```

        <b>SCC (Билет 10)</b> склеивает
    </br><br>
        <b>Точки сочленения (Билет 12)</b>
    ость</strong> монолита. Вырвешь точку сочле
    </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
    -b border-slate-700/50">
        Душный мод: Код Алгоритм Косарайю
    </summary>
    <div class="p-5 text-sm text-slate-300
        <p class="text-emerald-400 font-bol
        <ol class="list-decimal list-inside
            <li>Запускаем DFS на исходном г
li>
            <li>Разворачиваем этот список (
            <li>Переворачиваем все ребра в
            <li>Идем по <code>order</code>:
!</li>
        </ol>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "graph-bridges",
        title: "11. Графы. Мосты",
        type: "html",
        description: "Рёбра, удаление которых расхреничивае
        category: "Графы. Продвинутые",
        content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
      <div class="bg-slate-900 p-4 rounded-lg mb-4"
        <p class="text-lg text-rose-300 italic"
          <div class="text-left font-mono text-sm"
            <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
          <div class="p-3 bg-slate-950 rounded"
            <span class="text-rose-400 font"
              <p class="text-xl font-bold tex
                <p class="text-xs text-slate-400
              </p>
            </div>
          </div>
        </div>
      </div>
    </div>

    <p class="text-slate-300 text-sm">
      <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong>
ег - это тупик со степенью 1).
    </p>
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2 gap"
    <div class="bg-slate-800 p-6 rounded-lg bor"
      <div class="absolute -top-3 left-4 bg-s
rder-rose-500 shadow-md">
        Аналогия: Главный перекресток
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Представь город, где два района сое
её перекроют (удалят вершину) – районы буд
g>хрупкость</strong> системы.
      </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg bor

```

```

        IS_CUTPOINT(v);
        ++children;
    }
}
if (p == -1 && children > 1)
    IS_CUTPOINT(v);
}</pre>

</div>
</div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">
    <h4 class="text-lg font-bold text-indigo-400">
    <p class="text-slate-300 text-sm mb-4">
        Это глубокий дуальный мост между ориентир
    </p>
    <ul class="list-disc list-inside text-sm text-slate-300">
        <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
        <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
        <li><strong>Как точка сочленения рушит :
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
    </ul>
</div>
</div>
</section>`,
    },
    {
        id: "graph-euler",
        title: "13. Графы. Эйлеров цикл",
        type: "html",
        description: "Пройти по всем ребрам ровно 1 раз.",
        category: "Графы",
        content: `

```

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re

interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
}

interface Edge {
  u: number;
  v: number;
}

const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
  { id: 1, x: 100, y: 150, label: "1" },
  { id: 2, x: 250, y: 250, label: "2" },
  { id: 3, x: 400, y: 150, label: "3" },
  { id: 4, x: 550, y: 150, label: "4" },
  { id: 5, x: 700, y: 50, label: "5" },
  { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
  { u: 3, v: 4 }, // 3 and 4 are articulation points (3
  { u: 4, v: 5 },
  { u: 5, v: 6 },
  { u: 6, v: 4 },
];
```

```

    newSteps.push({
      desc,
      visited: new Set(visited),
      ap: new Set(ap),
      currentNode: u,
      tin: [...currentTin],
      low: [...currentLow],
    });
  });
};

recordStep("Начало DFS (Тарьян) для поиска точек со

const dfs = (v: number, p: number = -1) => {
  visited.add(v);
  currentTin[v] = currentLow[v] = timer++;
  let children = 0;

  recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

  for (const to of adj[v]) {
    if (to === p) continue;

    if (visited.has(to)) {
      currentLow[v] = Math.min(currentLow[v], curren
      recordStep(
        `Обратное ребро ${v}-${to}. Обновляем low[${
          v,
        });
    } else {
      children++;
      dfs(to, v);
      currentLow[v] = Math.min(currentLow[v], curren

      recordStep(
        `Возврат в ${v} из ${to}. Обновляем low[${v
          v,
        });
    }
  }
};

```

```

    currentNode: null,
    tin: [],
    low: [],
  });

  useEffect(() => {
    let interval: NodeJS.Timeout;
    if (isPlaying && currentStepIndex < steps.length - 1) {
      interval = setInterval(() => {
        setCurrentStepIndex((prev) => prev + 1);
      }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearInterval(interval);
  }, [isPlaying, currentStepIndex, steps.length]);

  const togglePlay = () => setIsPlaying(!isPlaying);
  const reset = () => {
    setIsPlaying(false);
    setCurrentStepIndex(0);
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-
      <div className="bg-slate-800 p-4 border-b border-
        <h3 className="font-bold text-white flex items-
          <Info className="w-5 h-5 text-rose-400" />
          Моделирование поиска точек сочленения
        </h3>

        <div className="flex items-center gap-2 bg-slate-
          <button
            onClick={togglePlay}
            className="flex items-center gap-2 px-3 py-
              text-white"
          >
            {isPlaying ? {

```



```

        strokeDasharray={isBridge ? "8,4" : ""}
      />
    );
  }}}

  { /* Nodes */
  {nodes.map((node) => {
    const isCurrent = currentStep.currentNode === node.id;
    const isVisited = currentStep.visited.has(node.id);
    const isAp = currentStep.ap.has(node.id);

    const fill = isCurrent
      ? "#3b82f6"
      : isAp
      ? "#e11d48"
      : isVisited
      ? "#10b981"
      : "#1e293b";
    const stroke = isCurrent
      ? "#60a5fa"
      : isAp
      ? "#f43f5e"
      : isVisited
      ? "#34d399"
      : "#334155";
    const r = isAp ? 24 : 20;

    return (
      <g key={node.id}>
        <circle
          cx={node.x}
          cy={node.y}
          r={r}
          fill={fill}
          stroke={stroke}
          strokeWidth="3"
          className="transition-all duration-300"
        />
      </g>
    );
  })}

```

```

        </text>
      </>
    })
  </g>
};
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-4
    Статус алгоритма
  </h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-white font-medium min-h-
      {currentStep.desc}
    </p>
  </div>

  <div className="space-y-4 flex-1 overflow-y-a
    <div className="mb-4">
      <h5 className="text-xs text-slate-500 fon
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Точка сочленения
        </div>
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Посещена
        </div>
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Текущая
        </div>
      <div className="flex items-center gap-2 t
        <div className="w-8 h-1 bg-rose-500 bor
          Мост

```

ФАЙЛ 31/87: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
  previous: { [key: string]: string | null };
  log: string;
  hasNegativeCycle?: boolean;
  activeCodeLine: number;
}

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];
```

```

    { line: 9, text: '                return "ОБНАРУЖЕН ОТПР'
    { line: 10, text: '    return dist' },
  ];

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  let dist: { [key: string]: number } = { S: 0, A: 999 };
  let prev: { [key: string]: string | null } = { S: null };

  simulationSteps.push({
    iteration: 0, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: '    Фура заведена. Начинаем с Базы (S). Расст',
    activeCodeLine: 2,
  });

  const vCount = nodes.length;
  for (let iter = 1; iter < vCount; iter++) {
    let anyUpdate = false;
    for (const edge of edges) {
      const uDist = dist[edge.u];
      const vDist = dist[edge.v];

      let updated = false;
      if (uDist !== 999 && uDist + edge.w < vDist) {
        dist = { ...dist, [edge.v]: uDist + edge.w };
        prev = { ...prev, [edge.v]: edge.u };
        updated = true;
        anyUpdate = true;
      }

      simulationSteps.push({
        iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
        distances: { ...dist }, previous: { ...prev },
        log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}
          ${updated ? 'Релаксация успешна! dist[${edge.v}] = ' + dist[edge.v] : ''}`,
        activeCodeLine: updated ? 6 : 5,
      });
    }
  }

```

```

        setSteps(simulationSteps);
        setCurrentStepIndex(0);
        setIsPlaying(false);
    }, [hasCycle]));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null || liveV !== null || liveW !== null || liveDist !== null;
const currentStep: Step = compilerLinked
    ? {
        ...baseStep,
        iteration: liveI ?? 0,
        checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
        distances: Object.fromEntries(nodes.map((node) => [node.id, liveDist[node.id]])),
        log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU} -> ${liveV}, вес ${liveW}, расстояние ${liveDist[liveU]} -> ${liveDist[liveV]}`
    }
    : baseStep;

return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
        <div className="flex flex-wrap items-center justify-center gap-3">
            <div>Шаг: {currentStep.index}</div>
            <div>Итерация: {currentStep.iteration}</div>
            <div>Ребро: {currentStep.checkingEdge ? `${currentStep.checkingEdge.u} -> ${currentStep.checkingEdge.v}` : "None"}</div>
            <div>Вес: {currentStep.checkingEdge ? currentStep.checkingEdge.w : "None"}</div>
            <div>Расстояние: {currentStep.checkingEdge ? `${currentStep.checkingEdge.u} -> ${currentStep.checkingEdge.v}` : "None"}</div>
        </div>
        <div>Лог: {currentStep.log}</div>
    </div>
);

```

```

        ▲ Отрицательный цикл!
    </div>
  })

<svg className="w-full h-auto max-h-[500px]" >
  <defs>
    <marker id="arrow-bf-normal" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
    <marker id="arrow-bf-active" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
    <marker id="arrow-bf-cycle" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
  </defs>

  {edges.map((edge, idx) => {
    const uNode = nodes.find((n) => n.id === u);
    const vNode = nodes.find((n) => n.id === v);
    const isChecking = currentStep.checkingEdges;

    const isNeg = edge.w < 0;
    const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

    let strokeColor = '#475569';
    let marker = 'url(#arrow-bf-normal)';
    if (isChecking) { strokeColor = '#f59e0b'; }
    else if (currentStep.hasNegativeCycle && isCyclePart) {
      strokeColor = '#e11d48';
      marker = 'url(#arrow-bf-cycle)';
    }

    return (
      <g key={idx}>
        <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} stroke-width={isChecking && isCyclePart ? 4 : 2} marker={isChecking && isCyclePart ? '4,4' : 'none'} />
        <circle cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)} r={2} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} stroke={strokeColor} stroke-width={2} />
        <text x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Neg' : isCyclePart ? 'Cycle' : ''} font-size="11" font-weight="bold" />
      </g>
    )
  })}
</svg>

```

```

        return (
            <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
            {edge.v} ({edge.w})
            </span>
        )
    }}}}
</div>
    )}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-300">
      <div className="grid grid-cols-4 gap-2 text-rose-300">
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge === n.id;
          return (
            <div key={n.id} className={`p-2 rounded
: 'border-slate-700/60 bg-slate-800/40'}`}>
              <div className="text-xs text-slate-400">
                <div className={`font-mono font-bo
            </div>
          </div>
        )};
      }}}}
    </div>
  </div>
</div>

  {/* Код алгоритма */}
  <div className="w-full lg:w-1/2 bg-slate-900/60">
    <div>

```

```

    { from: 1, to: 3 }, { from: 1, to: 4 },
    { from: 2, to: 5 }, { from: 2, to: 6 }
  ];

const ADJ: { [key: number]: number[] } = {
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
  activeLine: number;
  queue: number[];
  visited: number[];
  currentNode: number | null;
  logs: string;
}

const generateBfsSteps = (): Step[] => {
  const steps: Step[] = [];
  const queue: number[] = [];
  const visited = new Set<number>();

  // Init
  steps.push({
    activeLine: 1,
    queue: [],
    visited: [],
    currentNode: null,
    logs: "Инициализация: начинаем с корня (0).",
  });

  queue.push(0);
  visited.add(0);
  steps.push({
    activeLine: 2,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: "Добавляем стартовую вершину 0 в очередь и по"
  });

```



```

        visited: Array.from(visited),
        currentNode: curr,
        logs: `Помечаем ${n} как посещенную и добавляем в очередь.`
    ));
    }
  }
} else {
  steps.push({
    activeLine: 6,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: curr,
    logs: `Соседей нет или все уже посещены.`
  });
}
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
  }, []);

```

```

    >
      {isPlaying ? <Pause className="w-5 h-5" />
    </button>
    <button
      onClick={handleNext}
      disabled={stepIdx >= STEPS.length - 1}
      className="p-2 text-emerald-400 hover:text-
ed:hover:bg-transparent"
      title="Шаг вперед"
    >
      <StepForward className="w-5 h-5" />
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
  {/* Визуал графа */}
  <div className="bg-slate-800 border border-slate
    50px]">
    <svg className="w-full h-full min-h-[350px]" >
      {/* Рёбра */}
      {EDGES.map((e, idx) => {
        const n1 = NODES.find(n => n.id === e.from)
        const n2 = NODES.find(n => n.id === e.to)
        const isVisited = step.visited.includes(n
        return (
          <line
            key={idx}
            x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
            className={`transition-all duration-500
          `}}
        />
      )
    )}
  {/* Вершины */}
  {NODES.map(n => {
    const isCurrent = step.currentNode === n.id
    return (
      <div
        key={n.id}
        className={`w-5 h-5 rounded-full ${isCurrent ? 'bg-emerald-400' : 'bg-slate-400'}
      >
    )
  )}
  </div>
</div>

```

```

        className="fill-white font-bold text-slate-400"
      >
        {n.label}
      </text>
    </g>
  );
}
</svg>
</div>

```

{/* Код и Очередь */}

```

<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono tracking-wider">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">Mar {step}</span>
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visited.add(start)' },
        { line: 3, code: '' },
        { line: 4, code: 'while not queue.isEmpty()' },
        { line: 5, code: '  node = queue.pop()' },
        { line: 6, code: '  for neighbor in graph[node]:' },
        { line: 7, code: '    if neighbor not in visited:' },
        { line: 8, code: '      visited.add(neighbor)' },
        { line: 9, code: '      queue.push(neighbor)' },
      ]}.map((cl) => {
        const isActive = step.activeLine === cl.line;
        return (
          <div key={cl.line} className={`px-2 py-2 border-l-2 border-blue-400 -ml-0.5` : `text-slate-400`} >
            <span className="opacity-40 w-6 inline-block"> </span>
            <span className="whitespace-pre">{cl.code}</span>
          </div>
        )
      })
    </div>
  </div>

```

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const imageMap: Record<string, { src: string; alt: string; caption: string; borderColor: string; captionColor: string; }> = {
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: 'Добрый – только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Фура по болоту',
    caption: 'Тяжёлая, но ей пофиг на ямы!',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Все ко Всем Флойд',
    caption: 'Все связаны со всеми!',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 ${info.borderColor}`}>
        <img src={info.src} alt={info.alt} className="w-full h-full" />
        <div>
          <p>{info.caption}</p>
          <p>{info.alt}</p>
        </div>
      </div>
    </div>
  );
}
```

```

    >
      <ChevronLeft className="w-4 h-4" />
      <span className="hidden sm:inline">Назад</span>
    </button>
    <span className="text-slate-500 font-mono tabular-nums" />
      {index + 1} / {total}
    </span>
    <button
      disabled={!next}
      onClick={() => next && onGo(next.id)}
      title={next ? next.title : "Это последняя тема"}
      className="flex items-center gap-1 px-2.5 py-2 rounded-xl
        :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
    >
      <span className="hidden sm:inline">Вперёд</span>
      <ChevronRight className="w-4 h-4" />
    </button>
  </div>
);
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" />
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-white
          transition-all"
      >
        <div className="flex items-center gap-1.5 text-slate-600" />
        <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая
      </div>
        <div className="text-sm font-bold text-slate-600" />
      </button>
    ) : (
      <div className="hidden sm:block" />
    )}
  </nav>
);

```

```

    index: number;
    total: number;
    onGo: (id: string) => void;
    onOpenSimulator: (vizId: string) => void;
    /** Перейти во вкладку компилятора – там уже подставлен код */
    onOpenCompiler?: () => void;
}

```

```

const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;

```

```

export const ChapterView: React.FC<Props> = ({ chapter,
const contentRef = useRef<HTMLDivElement>(null);
const [anchor, setAnchor] = useState<HintAnchor | null>(null);
const hideTimer = useRef<number | null>(null);
const [saving, setSaving] = useState<"idle" | "work">("idle");

```

```

    useTermHints(contentRef, [chapter.id]);

```

```

    const cancelHide = useCallback(() => {
        if (hideTimer.current) {
            window.clearTimeout(hideTimer.current);
            hideTimer.current = null;
        }
    }, []);

```

```

    const scheduleHide = useCallback(() => {
        cancelHide();
        hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);
    }, [cancelHide]);

```

```

    useEffect(() => {
        setAnchor(null);
        setSaving("idle");
    }, [chapter.id]);

```

```

    const saveHtml = useCallback(async () => {
        setSaving("work");
        try {

```

```

const handleFocus = (e: React.FocusEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>
  if (el) open(el);
};

const viz = getViz(chapter.id);

return (
  <>
    <article className="chapter-body">
      {/* панель загрузки: сохранить именно эту страницу */}
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-x-small border-1px border-slate-500 hover:text-white hover:border-emerald-500"
        >
          {saving === "work" ? (
            <Loader2 className="w-3.5 h-3.5 animate-spin" />
          ) : saving === "done" ? (
            <Check className="w-3.5 h-3.5 text-emerald-500" />
          ) : (
            <Download className="w-3.5 h-3.5" />
          )}
        </button>
        <button
          onClick={() => window.print()}
          title="Распечатать или сохранить в PDF средствами браузера"
          className="flex items-center gap-1.5 text-x-small border-1px border-indigo-500 hover:text-white hover:border-indigo-500"
        >
          <Printer className="w-3.5 h-3.5" /> Печать
        </button>
        <span className="text-[11px] text-slate-500">
          </div>

```

```

        title="Открыть Python: код выполняе
рации"
        className="flex items-center gap-1.5
t-emerald-400 hover:text-white hover:border
    >
        <Terminal className="w-3.5 h-3.5" />
    </button>
    })
  </div>
</div>
{viz.hint && <p className="text-xs text-sla
<VizChapterContext.Provider value={chapter.
  <viz.Component />
</VizChapterContext.Provider>
</section>
}}

  <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
  anchor={anchor}
  onClose={() => setAnchor(null)}
  onOpenSimulator={(id) => {
    setAnchor(null);
    onOpenSimulator(id);
  }}
  onCardEnter={cancelHide}
  onCardLeave={scheduleHide}
/>
</>
);
};

```



```

    stack: number[];
    bridges: string[];
    log: string;
    activeEdge: [number, number] | null;
    backEdge: [number, number] | null;
    activeCodeLine: number[];
}

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {

```

```

    log: "Идём C→D. tin[D]=5, low[D]=5.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 4, visitedIndices: [0,1,6,2,3,4],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:4,3:5,4:6},
    activeEdge: [3,4], backEdge: null,
    log: "D→E. tin[E]=6, low[E]=6.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: [4,5], backEdge: null,
    log: "E→F. tin[F]=7, low[F]=7.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: [5,3],
    log: "Из F видим D (посещён). Обратное ребро (F,D).",
    activeCodeLine: [7,8,9]
},
{
    currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) — не мо",
    activeCodeLine: [11,13,16,17]
},
{
    currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) — не мо",
    activeCodeLine: [11,13,16,17]
},

```

```

    return byLabel ?? (Number.isFinite(Number(value))
    );
    const v = toId(rawV);
    const to = toId(rawTo);
    if (v === null) return null;
    const candidates = DFS_STEPS
      .map(({step, index}) => ({ step, index }))
      .filter(({ step }) => step.currentNode === v && {
    return candidates.at(-1)?.index ?? null;
  });
  const [dfsAuto, setDfsAuto] = useState(false);
  const dfsTimer = useRef<NodeJS.Timeout | null>(null);

  useEffect(() => {
    if (dfsAuto) {
      dfsTimer.current = setInterval(() => {
        setDfsIdx(prev => {
          if (prev >= DFS_STEPS.length - 1) { setDfsAuto(
            return prev + 1;
          });
        }, 2500);
      }
      return () => { if (dfsTimer.current) clearInterval(
    }, [dfsAuto]);

    const step = DFS_STEPS[dfsIdx];

    return (
      <div className="space-y-4">
        <div className="flex items-center justify-between">
          <h4 className="text-base font-bold text-white">
            <div className="flex items-center gap-1 bg-slate-800 p-1">
              <button onClick={() => { setDfsAuto(false); s
                disabled={dfsIdx === 0}
                className="p-1.5 hover:bg-slate-700 rounded
              <Undo className="h-3.5 w-3.5" />
            </button>
            <button onClick={() => setDfsAuto(!dfsAuto)}

```

```

        else if (activeBack) { sc = "#f43f5e"; sw
        else if (step.visitedIndices.includes(e.u
        return <line key={e.key} x1={u.x} y1={u.y
            stroke={sc} strokeWidth={sw} strokeDash
            className="transition-all duration-500"
        }}}
        {step.currentNode !== null && step.currentN
            const n = GRAPH_NODES.find(x => x.id ===
            if (!n) return null;
            return <circle cx={n.x} cy={n.y} r={18} f
        >;
    }}())}
    {GRAPH_NODES.map(n => {
        const cur = step.currentNode === n.id;
        const vis = step.visitedIndices.includes(n
        const st = step.stack.includes(n.id);
        return <g key={n.id}>
            <circle cx={n.x} cy={n.y} r={12}
                fill={cur ? "#10b981" : st ? "#064e3b
                stroke={cur ? "#fff" : st ? "#34d399"
                strokeWidth={2.5}
                className="transition-all duration-300
            <text x={n.x} y={n.y+3} textAnchor="mid
label}</text>
            {vis && (
                <>
                    <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
                    <text x={n.x} y={n.y-20} textAnchor:
                } l:{step.low[n.id]||"?"}</text>
                </>
            )}
        </g>;
    }}}
</svg>
<div className="flex items-center justify-bet
    <span>|||ar {dfsIdx+1}/{DFS_STEPS.length}</sp
    <div className="flex-1 mx-2 bg-slate-950 h-
```

```

        <div key={id} className={`px-2 py-0
          i === step.stack.length-1
            ? "bg-emerald-600/20 text-emera
            : "bg-slate-900 text-slate-400
        }`}>Вершина {GRAPH_NODES.find(n=>n.
          <span className="text-slate-500 m
        </div>
      ))
    }
  </div>
</div>

{/* Log */}
<div className="bg-indigo-950/10 border borde
  <p className="text-[11px] text-slate-300 le
  <div className="mt-2 pt-2 border-t border-s
    <span className="text-slate-400">Мосты:</
    <span className="font-mono font-bold text
      {step.bridges.length > 0 ? step.bridges
    </span>
  </div>
</div>
</div>
</div>
</div>
  );
}

```

ФАЙЛ 37/87: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,

interface Node { id: string; x: number; y: number; name

```

```

edges.forEach(e => {
  adjList[e.u].push({ v: e.v, w: e.w });
});

const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push({v: start, d: 0})' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлекаем минимальный элемент' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbours(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push({v: v, d: dist[v]})' },
  { line: 11, text: '    return dist // Все кратчайшие пути найдены' },
];

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false, B: false };
  const prev: Record<string, string | null> = { A: null, B: null };

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: '    Дейкстра готов к доставке. Начинаем из Питера',
    activeCodeLine: 2,
  });
});

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;

```

```

        activeCodeLine: 9,
      });
    }
  }
}

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ' Дейкстра всё доставил! Все возможные крат
  activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(var
const compilerLinked = liveU !== null || liveV !== nu

```

```

    }` }
  >
  {isPlaying ? ' Пауза' : '▶ Пуск'}
</button>
<button
  onClick={() => { setIsPlaying(false); if (c
  disabled={currentStepIndex >= steps.length
  className="flex items-center gap-1 bg-indigo
  px-3 py-2 rounded-lg text-sm font-medium tr
  >
  ⏏
</button>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 m
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-indigo-950/2
    stify-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-indi
      ont-bold shadow-sm">
        ⏏ {currentStepIndex + 1} из {steps.length}
    </div>

    <svg className="w-full h-auto max-h-[500px]"
      <defs>
        <marker id="arrow-dh-normal" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#47556
        </marker>
        <marker id="arrow-dh-active" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#f59e0
        </marker>
        <marker id="arrow-dh-opt" markerWidth="6"
          <path d="M0,0 L6,3 L0,6 z" fill="#10b98
        </marker>
      </defs>
      {/* Пёпса */}
      {edges.map((edge, idx) => {

```



```

    {/* Вершины */}
    {nodes.map((node) => {
      const isCurrent = currentStep.currentNode === node.id
      const isVisited = currentStep.visited[node.id]
      const dist = currentStep.distances[node.id]

      return (
        <g key={node.id} className="transition-all duration-300">
          <circle
            cx={node.x} cy={node.y}
            r={isCurrent ? 22 : 18}
            fill={isCurrent ? '#4f46e5' : isVisited ? '#a5b4fc' : '#f8d7da'}
            stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f8d7da' : '#6c757d'}
            strokeWidth={isCurrent ? 4 : 2}
            className="transition-all duration-300"
          />
          <text x={node.x} y={node.y + 5} fill="black" font-weight="bold">
            {node.id}
          </text>
          <text x={node.x} y={node.y - 28} fill="white" font-weight="bold">
            {node.name}
          </text>
          <text x={node.x} y={node.y + 35} fill="white" font-weight="bold">
            {node.name.split(' ')[0]}
          </text>
        </g>
      )
    })}
  </svg>
  <div className="w-full bg-slate-950/80 p-4 rounded">
    <p className="font-semibold text-amber-400">Шаг {currentStep.index + 1} из {currentStep.steps.length}</p>
    <p className="min-h-[40px] flex items-center justify-between">
      <span>Путь: {currentStep.path}</span>
      <span>Длина: {currentStep.distance}</span>
    </p>
  </div>
  </div>

  {/* Список смежности */}

```

```

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-950">
        Таблица расстояний
      </h4>
      <div className="overflow-x-auto">
        <table className="w-full text-left border-collapse">
          <thead>
            <tr className="border-b border-rose-950">
              <th className="py-2 px-3">Вершина</th>
              <th className="py-2 px-3">Расстояние</th>
              <th className="py-2 px-3">Из (пред. шаг)</th>
            </tr>
          </thead>
          <tbody className="text-sm">
            {nodes.map((n) => {
              const dist = currentStep.distances[n.id]
              const prev = currentStep.previous[n.id]
              const vis = currentStep.visited[n.id]
              const isCurr = currentStep.currentNode === n.id

              return (
                <tr key={n.id} className={`border-b border-rose-950 ${
                  isCurr ? 'bg-indigo-950/60 font-medium' : ''
                }`} >
                  <td className="py-2.5 px-3 flex items-center">
                    <span className={`w-2 h-2 rounded-full ${
                      vis ? 'bg-rose-950' : 'bg-indigo-950'
                    }`} style={n.name}></span>
                    {n.name}
                  </td>
                  <td className="py-2.5 px-3 font-medium">
                    {dist === 999 ? '∞' : dist}
                  </td>
                  <td className="py-2.5 px-3 text-sm">
                    {prev || '-'}
                  </td>
                </tr>
              )
            })}
          </tbody>
        </table>
      </div>
    </div>
  </div>

```

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '..';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'react-icons/lib';

interface DPStep {
  id: number;
  n: number;
  action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
  desc: string;
  codeLine: number;
  memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "    if (n in memo) return memo[n];",
  /* 3 */ "    if (n <= 2) return 1;",
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
  /* 5 */ "    return memo[n];",
  /* 6 */ "}"
];

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState<boolean>(false);
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' | 'graph'>('table');

  useEffect(() => {
    const generatedSteps: DPStep[] = [];
    let stepId = 0;
    const memoMap: { [key: number]: number } = {};

    function simulateFib(n: number): number {
      if (n <= 2) {
        generatedSteps.push({
          id: stepId,
          n: n,
          action: 'base_case',
          desc: `Base case: fib(${n}) = 1`,
          codeLine: 3,
          memoState: memoMap
        });
        stepId++;
        return 1;
      }
      if (n in memoMap) {
        generatedSteps.push({
          id: stepId,
          n: n,
          action: 'memo_hit',
          desc: `Memo hit: fib(${n}) = ${memoMap[n]}`,
          codeLine: 2,
          memoState: memoMap
        });
        stepId++;
        return memoMap[n];
      }
      generatedSteps.push({
        id: stepId,
        n: n,
        action: 'call',
        desc: `Call: fib(${n})`,
        codeLine: 1,
        memoState: memoMap
      });
      stepId++;
      const n1 = n - 1;
      const n2 = n - 2;
      const v1 = simulateFib(n1);
      const v2 = simulateFib(n2);
      generatedSteps.push({
        id: stepId,
        n: n,
        action: 'calc',
        desc: `Calc: fib(${n}) = ${v1} + ${v2} = ${v1 + v2}`,
        codeLine: 4,
        memoState: memoMap
      });
      stepId++;
      memoMap[n] = v1 + v2;
      return v1 + v2;
    }

    simulateFib(targetN);
    setSteps(generatedSteps);
  }, [targetN]);
}
```

```

        codeLine: 4,
        memoState: { ...memoMap }
    });

    const res1 = simulateFib(n - 1);
    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'calc',
      desc: `Сохраняем в кэш: мемо[${n}] = ${res1} + ${res2}`,
      codeLine: 5,
      memoState: { ...memoMap }
    });

    return memoMap[n];
  }

  const finalRes = simulateFib(targetN);

  generatedSteps.push({
    id: stepId++,
    n: targetN,
    action: 'done',
    desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
    codeLine: 6,
    memoState: { ...memoMap }
  });

  setSteps(generatedSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [targetN]);

useEffect(() => {
  let timer: any = null;

```

```

    </div>
    <h3 className="text-2xl font-extrabold text-slate-900">
  </div>
  <p className="text-sm text-slate-400">
    Смотри, как таблица DP кэширует результаты
  </p>
</div>

<div className="flex flex-wrap items-center gap-2">
  <div className="flex items-center gap-2 bg-slate-900 p-2">
    <span>N:</span>
    <select
      value={targetN}
      onChange={(e) => setTargetN(Number(e.target.value))}
      disabled={isPlaying}
      className="bg-transparent text-white font-mono"
    >
      <option value={4}>N = 4</option>
      <option value={5}>N = 5</option>
      <option value={6}>N = 6</option>
      <option value={7}>N = 7</option>
    </select>
  </div>

  <div className="flex items-center bg-slate-950 p-2 gap-2">
    <button
      onClick={() => { setIsPlaying(false); setTargetN(4); }}
      className="p-2 text-slate-400 hover:text-white"
      title="Сбросить"
    >
      <RotateCcw className="w-4 h-4" />
    </button>
    <button
      onClick={() => { setIsPlaying(false); setTargetN(4); setStepIndex(currentStepIndex - 1); }}
      disabled={currentStepIndex === 0}
      className="p-2 text-slate-400 hover:text-white"
      title="Шаг назад"
    >

```

```

        isCurrent
        ? 'bg-emerald-950 border-emerald-500'
        : isCached
        ? 'bg-slate-900 border-emerald-500/50'
        : 'bg-slate-900/50 border-slate-800'
      }` }
    >
    <span className="text-xs text-slate-400">
    <span className={`text-xl md:text-2xl font-medium`}
      {isCached ? val : '0'}
    </span>
    {num <= 2 && <span className="text-[9px] font-mono">
    </div>
  }
  }}}
</div>

{currentStep && (
  <div className="mt-6 pt-4 border-t border-slate-800">
    <div className="text-sm font-medium text-slate-400">
      <span className="inline-block w-2 h-2 rounded-full">
        <span>{currentStep.desc}</span>
      </div>
      <div className="text-xs font-mono bg-slate-900 p-2">
        Текущий N: <strong className="text-emerald-400">
      </div>
    </div>
  )}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-800">
  <button
    onClick={() => setActiveTab('table')}
    className={`flex items-center gap-2 px-6 py-3 rounded-md`}
    activeTab === 'table'
    ? 'border-emerald-500 text-emerald-400 bg-emerald-950'
    : 'border-transparent text-slate-400 hover:text-slate-300'
  >

```

```

<div className="bg-slate-950 rounded-2xl border-1
  <div className="bg-slate-900 px-6 py-3 border-1
    <span className="text-xs font-bold font-mono text-emerald-400"
    <span className="text-xs text-emerald-400"
  </div>
  <pre className="p-6 font-mono text-sm space-x-1"
    {DP_CODE_LINES.map((line, idx) => {
      const lineNum = idx + 1;
      const isActive = currentStep?.codeLine === lineNum;
      return (
        <div
          key={lineNum}
          className={`flex items-center px-4 py-2 ${
            isActive
              ? 'bg-emerald-600/20 border-l-4 border-emerald-500'
              : 'text-slate-300 hover:bg-slate-800'
          }`}
        >
          <span className="w-8 text-xs text-slate-400"
            <span>{lineNum}</span>
          <span>{line}</span>
        </div>
      );
    })}
  </pre>
  <div className="p-6 bg-slate-900/50 border-1 border-slate-800"
    <span className="p-1.5 bg-emerald-500/20 border-1 border-emerald-400"
    <span>{currentStep?.desc}</span>
  </div>
</div>
)}
</div>

```

```

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800"
  <div className="flex items-center justify-between"
    <span>Прогресс выполнения</span>
    <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
  </div>

```

```

const [c1Msg, setC1Msg] = useState("Клики на вершину");
const [c1Done, setC1Done] = useState(false);

const resetC1 = () => {
  setC1Path([]); setC1VisitedEdges([]); setC1Msg("Клики на вершину");
};

const clickC1 = (vId: number) => {
  if (c1Done && c1Path.length > 0) return;
  if (c1Path.length === 0) {
    setC1Path([vId]);
    setC1Msg("Старт! Кликай соседние вершины, чтобы пройти все рёбра");
    return;
  }
  const last = c1Path[c1Path.length - 1];
  const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
  if (!edge) {
    setC1Msg("Нет ребра между этими вершинами! Выбери другую вершину");
    return;
  }
  const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
  if (c1VisitedEdges.includes(key)) {
    setC1Msg("Это ребро уже пройдено! Эйлер не прощает повторений");
    setC1Done(true); return;
  }
  const nextEdges = [...c1VisitedEdges, key];
  const nextPath = [...c1Path, vId];
  setC1VisitedEdges(nextEdges);
  setC1Path(nextPath);

  if (nextEdges.length === C1_EDGES.length) {
    const startDegree = C1_EDGES.filter(e => e.u === c1Path[0]).length;
    const endDegree = C1_EDGES.filter(e => e.v === c1Path[0]).length;
    const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
    if (isCycle && nextPath[0] === vId) {
      setC1Msg("    ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, факт!");
    } else {
      setC1Msg(`    ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! Сделай клик на вершину ${vId} и получишь цикл`);
    }
  }
};

```



```

        stroke={isLast ? "#fff" : visited ? "#666" : "#ccc"}
        strokeWidth={2.5}
        className="transition-all duration-200"
        <text x={n.x} y={n.y+4} textAnchor="middle">
        bel}</text>
      </g>
    }
  </svg>
  <div className="absolute top-2 left-2 bg-slate-900 text-white">
    Ребёп: {c1VisitedEdges.length}/{C1_EDGES.length}
  </div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
  c1Done && c1VisitedEdges.length === C1_EDGES.length
    ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-500"
    : c1Done
    ? "bg-rose-950/40 border-rose-500/30 text-rose-500"
    : "bg-slate-900/50 border-slate-700 text-slate-500"}
  `}>
  {c1Msg}
</div>
</div>
);
}

```

ФАЙЛ 40/87: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

```

```

interface Question {
  id: number;
  question: string;
}

```

```

    'Он использует меньше памяти благодаря коммунистиче
    'Он умеет работать с отрицательными весами без ци
    'Он позволяет выполнять шаги слияния колхозов (ко
    'Он был разработан в Токио и поэтому самый быстрый
  ],
  correctAnswer: 2,
  explanation: 'Абсолютно верно! В Боровке каждая ком
    делить вычисления на GPU или многопроцессорных кл
},
{
  id: 4,
  question: 'В чем главная суть Динамического програм
  options: [
    'Писать код очень быстро, динамично нажимая на кл
    'Запоминать (кешировать) результаты уже решенных
    'Использовать динамическую типизацию как в Python
    'Постоянно менять требования к проекту в процессе
  ],
  correctAnswer: 1,
  explanation: 'Точно! Главный принцип ДП – мемоизаци
},
{
  id: 5,
  question: 'Какая структура данных идеально помогает
  options: [
    'СНМ (Система непересекающихся множеств / Disjoin
    'Стек (LIFO).',
    'Красно-черное дерево.',
    'Хэш-таблица со списками коллизий.'
  ],
  correctAnswer: 0,
  explanation: 'Блестяще! СНМ (DSU) позволяет за прак
    '
  }
];

export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<num

```

```

    <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-600 shadow-xl shadow-indigo-500/30">
      <Award className="w-10 h-10 text-white" />
    </div>
    <h3 className="text-3xl font-bold text-white mb-6">Ты настоящий сеньор-раскрасчик?</h3>
    <p className="text-slate-400 text-sm mb-6">Ты набрал {score} баллов из {quizQuestions.length} возможных. Это означает:

    <div className="bg-slate-950 border border-slate-800 p-6">
      <p className="text-slate-400 text-sm uppercase font-medium">Оценки</p>
      <p className="text-5xl font-black text-indigo-400">{score}</p>
      <p className="text-slate-300 text-sm">
        {score === quizQuestions.length
          ? ' Идеально! Ты настоящий сеньор-раскрасчик!'
          : score >= 3
          ? ' Отличный результат! Главные концепции ты освоил!'
          : ' Стоит повторить главы пособия и еще раз пройти тест.'
        }
      </p>
    </div>

    <button
      onClick={handleRestart}
      className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
    >
      <RotateCcw className="w-5 h-5" />
      <span>Пройти тест заново</span>
    </button>
  </div>
);
}

return (
  <div className="bg-slate-900/90 border border-slate-800 p-6">
    <div className="flex items-center justify-between">
      <div className="flex items-center gap-3">
        <span className="bg-purple-600 text-white px-4 py-2 rounded-lg">Экспресс-тест</span>
      </div>
    </div>
  </div>
);

```

```

        ) : isAnswered && isSelected && !isCo
        <XCircle className="w-6 h-6 text-ro
    ) : (
        <div className={"w-6 h-6 rounded-fu
order-indigo-500 bg-indigo-500 text-white"
        {String.fromCharCode(65 + idx)}
        </div>
    )}
    </div>
    <div className="flex-1 text-sm md:text-l
        {option}
    </div>
</div>
    );
    }}}}
</div>
</div>

{isAnswered && (
    <div className="bg-slate-950 border border-slat
        <div className="flex items-center gap-2 mb-2
            <AlertCircle className="w-4 h-4" />
            <span>Объяснение:</span>
        </div>
        <p className="text-slate-300 text-sm leading-
            {currentQ.explanation}
        </p>
    </div>
)}

<div className="flex justify-end pt-4 border-t bo
    <button
        onClick={handleNext}
        disabled={!isAnswered}
        className={
            "px-8 py-3.5 font-bold rounded-xl transition
            (!isAnswered
                ? "bg-slate-800 text-slate-500 border bor

```

```

    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];

```

```

const initialMatrix = [
  [0, 4, 999, 2, 999, 999, 999],
  [999, 0, 3, 999, 999, 3, 999],
  [999, 999, 0, 999, 2, 999, 999],
  [999, 999, 999, 0, 4, 2, 999],
  [999, 999, 999, 999, 0, 999, 1],
  [999, 999, 999, 999, 999, 0, 5],
  [999, 1, 999, 999, 999, 999, 0]
];

```

```

const adjList: { [key: number]: { v: number; w: number }[] } = {
  0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
  1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
  2: [{ v: 4, w: 2 }],
  3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
  4: [{ v: 6, w: 1 }],
  5: [{ v: 6, w: 5 }],
  6: [{ v: 1, w: 1 }],
};

```

```

const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V)' },
  { line: 2, text: '  dist = copy(matrix)' },
  { line: 3, text: '  for k from 0 to V - 1: // Пос' },
  { line: 4, text: '    for i from 0 to V - 1:' },
  { line: 5, text: '      for j from 0 to V - 1' },
  { line: 6, text: '        if dist[i][k] + d' },
  { line: 7, text: '          dist[i][j] = ' },
  { line: 8, text: '  return dist' },
];

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];

```

```

        activeCodeLine: 8,
    });

    setSteps(simulationSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
}, []));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const rawMatrix = vizArray(vars?.d) ?? vizArray(vars?.d);
const liveMatrix = rawMatrix
    ? baseStep.matrix.map((row, i) => {
        const values = vizArray(rawMatrix[i]);
        return row.map((_ , j) => vizNumber(values?.[j]))
    })
    : undefined;

const compilerLinked = liveK !== null || liveI !== null || liveJ !== null;
const currentStep: Step = compilerLinked
    ? {
        ...baseStep,
        k: liveK ?? -1,
        i: liveI ?? -1,
        j: liveJ ?? -1,
    }
    : baseStep;

```

```

<svg className="w-full h-auto max-h-[500px]" >
  <defs>
    <marker id="arrow-fw-normal" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
    <marker id="arrow-fw-active" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
    <marker id="arrow-fw-via" markerWidth="6"
    th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
  </defs>
  {Object.keys(adjList).flatMap((uStr) => {
    const u = parseInt(uStr);
    return adjList[u].map((edge) => {
      const uNode = nodesSvg.find((n) => n.id
      const vNode = nodesSvg.find((n) => n.id
      const isTarget = currentStep.i === u &&
      const isVia = (currentStep.i === u && c
      let strokeColor = '#475569'; let marker
      if (isTarget) { strokeColor = '#10b981'
      else if (isVia) { strokeColor = '#818cf8
      return (
        <g key={`${u}-${edge.v}`}>
          <line x1={uNode.x} y1={uNode.y} x2=
          markerEnd={marker} strokeDasharray={isTarg
          <circle cx={({uNode.x + vNode.x) / 2
            isVia ? '#818cf8' : '#64748b'} strokeWidth
          <text x={({uNode.x + vNode.x) / 2} y=
            8fafc'} fontSize="10" fontWeight="bold" tex
          </g>
        </g>
      );
    });
  });
})}
  {nodesSvg.map((node) => {
    const isK = currentStep.k === node.id;
    const isI = currentStep.i === node.id;
    const isJ = currentStep.j === node.id;
    let fill = '#1e293b'; let stroke = '#64748b';
    if (isK) { fill = '#4f46e5'; stroke = '#a

```

```

rentStep.j === edge.v);
    return (
      <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
      {edge.v} ({edge.w})
    </span>
  )
  )}}
</div>
);
}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  /* Матрица расстояний */
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div className="flex justify-between items-center">
      <h4 className="text-lg font-bold text-rose-950">
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-center border">
        <thead><tr className="bg-rose-950/40 text-white">
          <th className="p-2 border-r border-rose-950">
            {nodeNames.map((n, idx) => <th key={idx}
-400 font-bold' : ''}`}>{n.split(' ')[0]} {
          </tr></thead>
          <tbody className="text-xs font-mono">
            {currentStep.matrix.map((row, i) => (
              <tr key={i} className="border-b border-rose-950">
                <td className={`p-2 bg-rose-950/40 text-white
tStep.k === i ? 'text-amber-400' : ''}`}>
                  {nodeNames[i].split(' ')[0]} {i}
                </td>
                {row.map((val, j) => {
                  const isUpd = currentStep.i === i
                  const isVia = currentStep.k !== -

```


ФАЙЛ 42/87: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};
```

```

    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start} (из ${ray.from(vis)})` });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Обработка узла ${v} (из ${q})`, visitedNodes: [...Array.from(vis)] });
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, desc: `Визит в узел ${u} (из ${v})`, backtrack: [...q], visitedNodes: [...Array.from(vis)] });
            }
        }
    }

    steps.push({ type: 'backtrack', node: '', desc: `Завершение обхода (из ${ray.from(vis)})` });

    return steps;
}

```

```

export function GraphTraversalViz() {
    const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
    const [autoPlay, setAutoPlay] = useState(false);
    const [stepIdx, setStepIdx] = useState(0);

    const steps = useMemo(() => mode === 'dfs' ? generateDfsSteps(startNode, adj) : generateBfsSteps(startNode, adj), [mode, startNode, adj]);
    useVizStepSync(stepIdx, setStepIdx, steps.length - 1);
}

```

```

return (
  <div className="space-y-4">
    <div className="flex bg-slate-900 border border-slate-800 p-4">
      <button onClick={() => setMode("dfs")}
        className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
        DFS (В глубину)
      </button>
      <button onClick={() => setMode("bfs")}
        className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
        BFS (В ширину / Волна)
      </button>
    </div>

    <div className="flex flex-col md:flex-row gap-4">
      {/* SVG Graph View */}
      <div className="bg-slate-900 border border-slate-800 p-4 min-h-[300px]">
        <svg className="w-full h-full max-w-[600px] max-h-[300px]">
          {/* Edges */}
          {edges.map((e, i) => {
            const u = nodes.find(n => n.id === e.u)
            const v = nodes.find(n => n.id === e.v)
            // Check if edge is part of the shortest path
            const edgeTraversed = isVisited(e.u, e.v)

            return (
              <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
                className={
                  edgeTraversed ? 'stroke-emerald-500' : 'stroke-slate-400'
                }
              />
            )
          })}

          {/* Nodes */}
          {nodes.map(n => {
            const visited = isVisited(n.id, n.id)

            return (
              <div key={n.id} className="flex flex-col items-center justify-center" style={{width: 30px, height: 30px}}>
                <div style={{width: 10px, height: 10px, background-color: visited ? 'emerald-500' : 'slate-400', border: 1px solid 'slate-800'}}></div>
                {n.id}
              </div>
            )
          })}
        </svg>
      </div>
    </div>
  </div>
)

```

```

        {/* Structure View (Stack / Queue) */}
        <div className="bg-slate-900 border border-emerald-500 p-2">
          <div className={`absolute -top-3 left-3 right-3 bottom-3
            ${mode === 'dfs' ? 'bg-slate-900 border border-emerald-500'}
            ${mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
          `}>
            <div className="flex flex-col gap-2">
              {step.queueOrStack && step.queueOrStack.map((item, idx) => {
                <div key={idx} className={`w-full p-2 text-emerald-200
                  ${idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
                    ? `bg-slate-900 border border-emerald-500`
                    : ''}
                `}>
                  Узел: {item}
                  {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
                    ? <span className="ml-2">Вызывает: {step.queueOrStack[idx + 1]}</span>
                    : ''}
                </div>
              })}
              {(!step.queueOrStack || step.queueOrStack.length === 0)
                ? <div className="text-slate-500">Пусто</div>
                : ''
              }
            </div>
          </div>
        </div>

        <div className="flex flex-col sm:flex-row gap-2">
          {/* Controls */}
          <div className="flex bg-slate-900 border border-emerald-500 p-2 text-slate-400 hover:text-white hover:shadow-lg">
            <button onClick={() => {setAutoPlay(!autoPlay)}}>
              {autoPlay ? 'Пауза' : 'Воспроизведение'}
            </button>
            <button onClick={() => {setStep(step + 1)}}>
              Следующий
            </button>
            <button onClick={() => {setStep(step - 1)}}>
              Предыдущий
            </button>
            <button onClick={() => {setStep(0)}}>
              Сброс
            </button>
            <button onClick={() => {setStep(step + 1)}}>
              Завершить
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

import { useVizRuntime, vizArray, vizNumber } from '../

```
interface Patient {  
  id: string;  
  name: string;  
  emoji: string;  
  symptom: string;  
  priority: number; // 1 to 99 (Severity)  
  animState: 'in' | 'idle' | 'winner' | 'out';  
}
```

// Max-heap helpers for Patients

```
function siftUp(arr: Patient[]): Patient[] {  
  const a = arr.map(x => ({ ...x }));  
  let idx = a.length - 1;  
  while (idx > 0) {  
    const parent = Math.floor((idx - 1) / 2);  
    if (a[parent].priority < a[idx].priority) {  
      [a[parent], a[idx]] = [a[idx], a[parent]];  
      idx = parent;  
    } else break;  
  }  
  return a;  
}
```

```
function siftDown(arr: Patient[]): Patient[] {  
  const a = arr.map(x => ({ ...x }));  
  const n = a.length;  
  let idx = 0;  
  while (true) {  
    let largest = idx;  
    const l = 2 * idx + 1;  
    const r = 2 * idx + 2;  
    if (l < n && a[l].priority > a[largest].priority) largest = l;  
    if (r < n && a[r].priority > a[largest].priority) largest = r;  
    if (largest !== idx) {  
      [a[largest], a[idx]] = [a[idx], a[largest]];  
      idx = largest;  
    }  
  }  
  return a;  
}
```

```

    });
    start.forEach((p, i) => {
      arr = heapInsert(arr, { id: `init${i}`, ...p, animS
    });
    return arr;
  })();

```

```
let uid = 300;
```

```

export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>();
  const runtime = useVizRuntime();
  const liveHeap = vizArray(runtime?.variables?.h) ?? v
  const liveI = vizNumber(runtime?.variables?.i);
  const visualHeap: Patient[] = liveHeap
    ? liveHeap.map((value, index) => {
      const tuple = vizArray(value);
      const priority = vizNumber(tuple?.[0]) ?? vizNum
      return {
        id: `python-${index}`,
        name: tuple ? String(tuple[1] ?? value) : Str
        emoji: " ",
        symptom: `индекс ${index}`,
        priority,
        animState: "idle",
      };
    })
    : heap;
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(
  const [log, setLog] = useState<string>('');
  const [shakeEmpty, setShake] = useState(false);
  const [busy, setBusy] = useState(false);
  const [healingPatient, setHealing] = useState<Patient

```

```
const addLog = (msg: string) => setLog(prev => [msg,
```

```
// --- INSERT: Новый пациент поступает в приемный пок
```

```

    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const randomShift = Math.floor(Math.random() * 10);
    const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
    insertPatient(preset.name, prio);
  };

```

```

const handleCustomInsert = (e: React.FormEvent) => {
  e.preventDefault();
  if (!customName.trim()) return;
  insertPatient(customName, customPriority);
  setCustomName('');
};

```

```

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
  if (busy || heap.length === 0) {
    setShake(true);
    addLog('⚠ Нет пациентов для экстренной госпитализации');
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);
  const topPatient = heap[0];
  setHealing(topPatient);

```

```

  addLog(`  СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

```

```

  // Step 1: Root lights up as winner
  setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, winner: true } : x));

```

```

  setTimeout(() => {
    // Step 2: Root exits the tree, tree reorganizes
    setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, winner: false } : x));
  }, 100);

```

```

  setTimeout(() => {
    // Run binary heap extraction
    setHeap(prev => {
      if (prev.length === 0) return [];
      const [top, ...rest] = prev;
      const newHeap = [...rest];
      // Insert topPatient into newHeap
      const index = 0;
      while (index < newHeap.length) {
        const leftIndex = 2 * index + 1;
        const rightIndex = 2 * index + 2;
        let swapIndex = index;
        if (leftIndex < newHeap.length && newHeap[leftIndex].priority < newHeap[swapIndex].priority) {
          swapIndex = leftIndex;
        }
        if (rightIndex < newHeap.length && newHeap[rightIndex].priority < newHeap[swapIndex].priority) {
          swapIndex = rightIndex;
        }
        if (swapIndex !== index) {
          [newHeap[index], newHeap[swapIndex]] = [newHeap[swapIndex], newHeap[index]];
        }
        index = swapIndex;
      }
      return newHeap;
    });
  }, 100);

```

```

        x2={`${lp.x}%`} y2={`${lp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  if (r < visualHeap.length) {
    const rp = nodePos(r);
    res.push(
      <line key={`er${idx}`}
        x1={`${p.x}%`} y1={`${p.y + 4}%`}
        x2={`${rp.x}%`} y2={`${rp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  return res;
});

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-emerald-950/40 border border-emerald-500">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-emerald-400 text-center">
          ty Queue)</h3>
        <p className="text-xs text-slate-300 mt-1 leading-relaxed">
          В реанимации не важно, кто пришел раньше, а кто в
          остояние</b> (максимальный приоритет).
          Куча (Heap) автоматически перестраивает пацков,
          оказывался на самом верху дерева (в корне).
        </p>
      </div>
    </div>

    {/* УПРАВЛЕНИЕ */}
    <div className="flex flex-col lg:flex-row items-stretch">

```



```

        </div>
    );
    }}}
</div>

<div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 round
    <div className="absolute top-4 left-4 text-[1
        Операционная койка
    </div>

<div className="flex-1 flex flex-col justify-
    {healingPatient ? (
        <div className="flex flex-col items-cente
            <div className="relative">
                <span className="text-6xl block anim-l
                <span className="absolute -top-4 -right
                <span className="absolute -bottom-2 -
            </div>
        <div>
            <h4 className="text-white font-black
                <span>Пациент: {healingPatient.name
            </h4>
            <p className="text-emerald-400 text-x
                Состояние: Стабилизация ({healingPa
            </p>
            <div className="flex justify-center g
                <span className="w-2 h-2 rounded-fu
                <Heart className="w-4 h-4 text-rose
            </div>
        </div>
    </div>
) : (
    <div className="text-center text-slate-60
        <span className="text-5xl mb-2 block">

```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```
/** Наведение на карточку ставит анимацию на паузу — мож
```

```
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {
```

```
  const [step, setStep] = useState(0);
```

```
  const paused = useContext(DemoPauseContext);
```

```
  const period = Math.round(ms * SPEED);
```

```
  useEffect(() => {
```

```
    if (steps <= 1 || paused) return;
```

```
    const t = setInterval(() => setStep((s) => (s + 1)),
```

```
    return () => clearInterval(t);
```

```
  }, [steps, period, paused]);
```

```
  return step;
```

```
}
```

```
export const C = {
```

```
  idle: "#334155",
```

```
  idleStroke: "#475569",
```

```
  text: "#e2e8f0",
```

```
  dim: "#94a3b8",
```

```
  active: "#6366f1",
```

```
  done: "#10b981",
```

```
  warn: "#f59e0b",
```

```
    </g>
  );

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    strokeWidth={width}
    strokeDasharray={dashed ? "4 3" : undefined}
    style={{ transition: MOVE }}
  />
);
```

ФАЙЛ 45/87: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7
  const pos: Record<string, [number, number]> = { i: [4, 4], j: [9, 9], k: [3, 3] };
  const viaOpen = step >= 1;
  const relaxed = step >= 2;
```

```

const pos: Record<string, [number, number]> = {
  A: [26, 40], B: [62, 92], C: [26, 92],
  D: [120, 40], E: [120, 92],
  F: [190, 34], G: [232, 78], H: [178, 96],
};
const colors = [C.active, C.warn, C.done];
const step = useLoop(comps.length + 1, 900);
const compOf = (v: string) => comps.findIndex((e) => e === v);

return (
  <Svg caption={step === 0 ? "Граф распался на острова" : ""}
    {comps.flatMap((edges, ci) =>
      edges.map(([u, v], i) => (
        <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
      ))
    )}
    {Object.entries(pos).map(([k, [x, y]]) => {
      const ci = compOf(k);
      return <Node key={k} x={x} y={y} r={12} label={ci}
    })}
  </Svg>
);
};

```

```

/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
  const step = useLoop(3, 1300);
  const pos: Record<string, [number, number]> = { A: [50, 40], B: [150, 40], C: [250, 40],
  const edges: [string, string, number][] = [["A", "B", 1], ["B", "C", 1], ["C", "A", 1],
  return (
    <Svg
      caption={
        step === 0
          ? "Вершины – объекты (города, слова, состояния)"
          : step === 1
            ? "Рёбра – связи между ними"
            : "Вес ребра – цена перехода"
      }
    >

```

```

<Svg
  caption={
    step === 0
      ? "Исходные координаты – огромные и с дырами"
      : step === 1
        ? "Сортируем уникальные значения"
        : "Каждое число заменяем его номером → плотн
  }
>
  {raw.map((v, i) => {
    <g key={i}>
      <rect x={startX + i * (boxW + 4)} y={20} width
      <text x={startX + i * (boxW + 4) + boxW / 2}
        {v}
      </text>
    </g>
  })}

  {step >= 1 &&
    sorted.map((v, i) => {
      <g key={v}>
        <rect x={30 + i * 52} y={62} width={46} hei
        <text x={53 + i * 52} y={77} textAnchor="mi
          {v}
        </text>
      </g>
    })}

  {step >= 2 &&
    raw.map((v, i) => {
      <g key={`c${i}`}>
        <rect x={startX + i * (boxW + 4)} y={98} wi
        <text x={startX + i * (boxW + 4) + boxW / 2
          {sorted.indexOf(v)}
        </text>
      </g>
    })}
  </Svg>

```

```

    }}}
  </Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
  <SplayFrames
    captions={["р – корень, х под ним", "Один поворот: "]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p":
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p":
    ]}
  />
};

/** Zig-Zig – х и р с одной стороны, крутим сначала верш
export const ZigZigDemo: React.FC = () => {
  <SplayFrames
    captions={["Линия g → p → x («бамбук»)", "Поворот В
    frames={[
      { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
      { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
      { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
    ]}
  />
};

/** Zig-Zag – х и р с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
  <SplayFrames
    captions={["Зигзаг: g влево, х вправо", "Поворот НИ
    frames={[
      { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
      { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
      { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
    ]}
  />

```

```

const path = ["A", "B", "D", "E", "F", "C"];
const step = useLoop(path.length + 1, 750);
const visited = path.slice(0, step);
const head = visited[visited.length - 1];
return (
  <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""}
    {GRAPH_EDGES.map(([u, v], i) => {
      const iu = visited.indexOf(u);
      const iv = visited.indexOf(v);
      const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
      return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "gray"} />
    })}
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
      <Node key={k} x={x} y={y} label={k} fill={k === "A" ? "red" : "gray"} />
    ))}
  </Svg>
);
};

export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["труссы", "штаны"],
    ["носки", "боты"],
    ["труссы", "носки"],
    ["штаны", "боты"]
  ];
  const order = ["труссы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, [0, 0]]));
  return (
    <Svg caption={`Порядок: ${placed.join(" → ") || "..."} />
      {edges.map(([u, v], i) => (
        <Edge key={i} a={pos[u]} b={pos[v]} color={placed.indexOf(u) < placed.indexOf(v) ? "red" : "gray"} />
      ))}
      {nodes.map((n) => (
        <g key={n.id}>
          <Node key={n.id} x={pos[n.id][0]} y={pos[n.id][1]} label={n.id} fill={placed.indexOf(n.id) < placed.length ? "red" : "gray"} />
        </g>
      ))}
    </Svg>
  );
};

```



```

    });
    const edges: [string, string][] = [
      ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
    ];
    const cut = step === 1;
    return (
      <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
        {edges.map(([u, v], i) => {
          const isBridge = u === "C" && v === "D";
          if (isBridge && cut) return null;
          return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
        })}
        {Object.entries(pos).map(([k, [x, y]]) => (
          <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
        ))}
      </Svg>
    );
  };
};

```

```

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
      {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />))}
      {Object.entries(pos).map(([k, [x, y]]) => (
        cut && k === "C" ? null : (
          <Node key={k} x={x} y={y} label={k} fill={k === "C" ? "red" : "black"} />
        )
      ))}
    </Svg>
  );
};

```

```

const inScc = (u: string, v: string) => step === 1 &&
return (
  <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
    {edges.map(([u, v], i) => {
      <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
    }}}
    {Object.entries(pos).map([k, [x, y]] => {
      <Node key={k} x={x} y={y} label={k} fill={step :
    }}}
  </Svg>
);
};

```

ФАЙЛ 47/87: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
  const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [
  const step = useLoop(states.length, 1000);
  const heap = states[step];
  const pos: [number, number][] = [[130, 24], [80, 68],
  const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
  return (
    <Svg caption="Новый элемент всплывает наверх, пока
      <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b=
      <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
      {heap.map((v, i) => {
        <Node key={i} x={pos[i][0]} y={pos[i][1]} label
      }}}
    </Svg>
  );
};

```

```

        >
      </g>
    )})
    <line x1={24} y1={71} x2={10} y2={71} stroke={C.d
  </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
  const root = (i: number): number => (p[i] === i ? i : root(p[i]));
  return (
    <Svg caption="find = «кто твой староста», union = 0" >
      {p.map((par, i) =>
        par !== i ? (
          <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}
            fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
        ) : null
      )}
      {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
    </Svg>
  );
};

```

```

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

```

```

export const TrieDemo: React.FC = () => {

```

```

    const p = TRIE_NODES[n.parent as number];
    return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
  ]}]
  {links.slice(0, step).map(([from, to], i) => {
    const a = TRIE_NODES[from];
    const b = TRIE_NODES[to];
    return (
      <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x + b.x) / 2, (a.y + b.y) / 2} ${b.x} ${b.y}`}
        fill="none" stroke={C.warn} strokeWidth={1.5} />
    );
  })}
  {TRIE_NODES.map((n) => (
    <Node key={n.id} x={n.x} y={n.y} label={n.ch} lvl={n.lvl} />
  ))}
</Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const nodes = [
    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
    { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
    { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых" />
      {nodes.map((n, i) => (
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={20}
            fill={n.lvl === step ? C.active : C.idle} stroke={C.warn} />
          <text x={n.x} y={n.y} textAnchor="middle" dom
        />
      ))}
    )
  );
};

```

```

const len = [1, 2, 4][step];
const count = 8 - len + 1;
return (
  <Svg caption={`Уровень j=${step}: готовые ответы дл
    {Array.from({ length: 8 }).map((_, i) => {
      <g key={i}>
        <rect x={12 + i * 30} y={18} width={26} height={18} />
        <text x={25 + i * 30} y={29} textAnchor="middle" />
      </g>
    })}
    {Array.from({ length: count }).map((_, i) => {
      <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 18} width={26} height={18}
        fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
    })}
  </Svg>
);
};

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  const avg = step ? (total / step).toFixed(2) : "-";
  return (
    <Svg caption={`Сделано ${step} операций, суммарно $
      {costs.map((c, i) => {
        <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18}
          fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} stroke={C.done} />
      })}
      <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
      <text x={130} y={20} textAnchor="middle" fontSize={14} />
    </Svg>
  );
};

export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (

```

```

    });
};

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово $`}
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>
              <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
                fill={idx === step - 1 ? C.active : fill}
                {filled && {
                  <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
                    fontSize={10} fontWeight="700" fill="black"/>
                }}
            </g>
          );
        })
      )
    );
  <text x={130} y={122} textAnchor="middle" fontSize={14}>
    каждая клетка считается один раз и переиспользуется
  </text>
</Svg>
);
};

```

ФАЙЛ 48/87: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
```

```
scc: SccDemo,  
"prefix-function": PrefixFunctionDemo,  
dp: DpDemo,  
floyd: FloydDemo,  
components: ComponentsDemo,  
graph: GraphBasicsDemo,  
"coord-compress": CoordCompressDemo,  
zig: ZigDemo,  
"zig-zig": ZigZigDemo,  
"zig-zag": ZigZagDemo,  
};  
  
export const MiniDemo: React.FC<{ kind: DemoKind }> = (  
  const [paused, setPaused] = useState(false);  
  const Cmp = REGISTRY[kind];  
  if (!Cmp) return null;  
  return (  
    <DemoPauseContext.Provider value={paused}>  
      <div  
        className="bg-slate-950/70 rounded-lg border bo  
        onMouseEnter={() => setPaused(true)}  
        onMouseLeave={() => setPaused(false)}  
      >  
        <Cmp />  
      </div>  
    </DemoPauseContext.Provider>  
  );  
);
```

ФАЙЛ 49/87: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useEffect } from "react";  
import { createPortal } from "react-dom";  
import { X } from "lucide-react";
```

```
        className="p-2 rounded-lg bg-slate-800 hover:
        aria-label="Заккрыть симулятор"
      >
        <X className="w-5 h-5" />
      </button>
    </div>
    <div className="overflow-y-auto overscroll-cont
      <Component />
    </div>
  </div>
</div>,
document.body
);
};
```

ФАЙЛ 50/87: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}
```

```
interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
```



```

        window.removeEventListener("scroll", onClose, true);
    };
    }, [anchor, onClose]);

```

```

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

```

```

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
      visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
  >
    <div className="bg-slate-900 border border-indigo-500"
      <div className="flex items-start gap-2 px-3 pt-3"
        <h4 className="text-sm font-bold text-indigo-400">
          <button
            onClick={onClose}
            className="text-slate-500 hover:text-white"
            aria-label="Заккрыть подсказку"
          >
            <X className="w-4 h-4" />
          </button>
        </div>
        <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto">
          <p className="text-[13px] leading-relaxed text-slate-300">

```

```

-----
import { RotateCcw, SkipForward, Undo } from 'lucide-react'
import { useVizStepSync, vizString } from '../data/vizSteps'

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
  useVizStepSync(step, setStep, MAX_STEP, (vars) => {
    const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
    return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
  });

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];

  const edges = [
    { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 4, labelDx: 0, labelDy: 0 },
    { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 5, labelDx: 0, labelDy: 0 },
    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
  ];

  const isNodeVisible = (id: string) => {
    if (id === 'S' && (step === 0 || step === 7)) return true;
    return false;
  };

  const getPotential = (id: string) => {
    if (step >= 2 && step < 7) {
      if (id === 'S') return 0;
      if (id === 'A') return 0;
      if (id === 'B') return -2;
      if (id === 'C') return -1;
    }
    return 0;
  };
}

```

```

    if (e.id === 'BC' && step === 5) return "url(#a
    if (e.id === 'BC' && step > 5) return "url(#arr
    if (e.id === 'CA' && step === 6) return "url(#a
    if (e.id === 'CA' && step > 6) return "url(#arr

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
    if (e.u === 'S') return e.weight;

    if (step < 4) return e.weight;

    if (e.id === 'AB') {
        if (step === 4) return `${e.weight} + (${0}
        return e.newWeight;
    }
    if (e.id === 'BC') {
        if (step < 5) return e.weight;
        if (step === 5) return `${e.weight} + (${-2}
        return e.newWeight;
    }
    if (e.id === 'CA') {
        if (step < 6) return e.weight;
        if (step === 6) return `${e.weight} + (${-1}
        return e.newWeight;
    }
    return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
    if (e.id === 'AB' && step === 4) return true;
    if (e.id === 'BC' && step === 5) return true;
    if (e.id === 'CA' && step === 6) return true;
    return false;
};

```

```

        );
        case 4: return (
            <div>
                <b>Шаг 3.1: Пересчет ребра A → B.</b>
                Старый вес: <span className="text-gray-400">1</span>
                Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-emerald-300">0</span>
            </div>
        );
        case 5: return (
            <div>
                <b>Шаг 3.2: Пересчет ребра B → C.</b>
                Старый вес: 1. <br/>
                Вычисляем:  $1 + (-2) - (-1) =$  <span className="text-emerald-300">0</span>
            </div>
        );
        case 6: return (
            <div>
                <b>Шаг 3.3: Пересчет ребра C → A.</b>
                Старый вес: 2. <br/>
                Вычисляем:  $2 + (-1) - 0 =$  <span className="text-emerald-300">1</span>
            </div>
        );
        case 7: return (
            <div className="text-emerald-300">
                <b>Готово!</b> Фиктивную вершину S
                Все новые веса ребер в графе стали 0
                При этом старые кратчайшие пути остались
                теми же!
            </div>
        );
        default: return "";
    }
};

return (
    <div className="bg-slate-800 p-4 rounded-xl border">
        <h4 className="text-sm md:text-base font-bold">
            <span className="w-2.5 h-2.5 bg-amber-400">

```

```

        const midX = (u.x + v.x) / 2
        const midY = (u.y + v.y) / 2

        const edgeColorClass = getEdgeColorClass(u, v)
        const textHigh = isEdgeTextHigh(u, v)
        const textEmer = isEdgeTextEmer(u, v)

        return (
          <g key={i}>
            <line
              x1={u.x} y1={u.y}
              className={`stroke-${edgeColorClass}`}
              markerEnd={getMarkerEnd(u, v)}
            />

            <text
              x={midX} y={midY}
              textAnchor="middle"
              dominantBaseline="bottom"
              className={`text-${edgeColorClass}`}
              style={{
                textHeight: textHigh,
                textEmer: textEmer,
                isS ? 'font-size: 1.2em' : ''
              }}
            >{getEdgeContent(u, v)}
            </text>
          </g>
        )
      })
    })
  }
}

```

```

  { /* Nodes */ }
  { nodes.map(n => {
    if (!isNodeVisible(n.id)) return null
    const pot = getPotential(n.id)

    return (
      <g key={n.id} className={`node-${pot}>
        <circle
          cx={n.x} cy={n.y}
          r={n.r}
          fill={n.fill}
          stroke={n.stroke}
          stroke-width={n.stroke-width}
        />
      >
    )
  }) }

```

```

        className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
        <RotateCcw strokeWidth={3}
      </button>
      <button onClick={() => setStep(1)
        className="p-3 bg-slate-800
-slate-400 transition-colors" title="Шаг на
        <Undo strokeWidth={3} size=
      </button>
      <button onClick={() => setStep(
        className={`flex-1 font-bol
          ${step === MAX_STEP
            ? 'bg-emerald-600/50'
            : 'bg-indigo-600 ho
          {step === MAX_STEP ? "Готов
          {step !== MAX_STEP && <Skip
      </button>
    </div>
  </div>

  {/* Progress Dots */}
  <div className="flex justify-center
    {Array.from({length: MAX_STEP +
      <div key={i} className={`w-1
125' : i < step ? 'bg-indigo-900' : 'bg-sla
    )}}
  </div>
</div>
</div>
</div>
</div>
);
}

```

```

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini";
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const v = vizNumber(vars.v);
  const orderLength = vizArray(vars.order)?.length;
  if (v === null && orderLength === undefined) return null;
  const candidates = steps
    .map(({step, index}) => ({step, index}))
    .filter(({step}) => (v === null || step.currentNode !== null));
  return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  // Generate step-by-step Kosaraju steps
  const generatedSteps: AlgoStep[] = [];
  const visited = new Set<number>();
  const order: number[] = [];
  const sccMap: Record<number, number> = {};

  const recordStep = {
    phase: AlgoStep["phase"],
    desc: string,
    currentNode: number | null,
    transposed: boolean,
  };

```

```

    });

    for (const to of adj[u]) {
        if (!visited.has(to)) {
            dfs1(to);
        }
    }

    order.push(u);
    recordStep(
        "dfs1",
        `DFS 1: вершина ${u} полностью обработана. Запи-
        у,
        false,
        -1,
    );
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        dfs1(i);
    }
}

const topoOrder = [...order].reverse();
recordStep(
    "transpose",
    `Первый проход завершен! Стек выхода в топологиче-
    ебра.`,
    null,
    true,
    -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

```



```

        "dfs2",
        `DFS 2: вершина ${u} из стека уже покрашена в
        u,
        true,
        sccId,
    );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент,
    null,
    true,
    sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
    phase: "init",
    desc: "Загрузка...",
    visited: new Set(),
    currentNode: null,
    order: [],
    transposed: false,
    sccMap: {},
    currentSccId: -1,
    activeEdges: initialEdges,
};

useEffect(() => {
    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1)
        timer = setInterval(() => {
            setCurrentStepIdx((prev) => prev + 1);
        }, 1000);
    return () => {
        if (timer) clearInterval(timer);
    };
}, [isPlaying, currentStepIdx]);

```

```

        {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]"
            {/* Markers for directed arrows */}
            <defs>
                <marker
                    id="arrow"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
                </marker>
                <marker
                    id="arrow-active"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"

```

```

        if {
            step.currentNode === edge.u ||
            step.currentNode === edge.v
        } {
            stroke = "#10b981";
            markerId = "arrow-active";
        }
    }

    // Adjust slightly for bidirectional link
    const isBidi =
        (edge.u === 3 && edge.v === 4) ||
        (edge.u === 4 && edge.v === 3);
    const dy = isBidi ? (edge.u === 3 ? -10 :

    return (
        <line
            key={`edge-${idx}`}
            x1={x1}
            y1={y1 + dy}
            x2={x2}
            y2={y2 + dy}
            stroke={stroke}
            strokeWidth="2.5"
            markerEnd={`url(#${markerId})`}
            className="transition-all duration-300"
        />
    );
}}}

{/* Nodes */}
{initialNodes.map((node) => {
    const isCurrent = step.currentNode === node.id;
    const isVisited =
        step.visited.has(node.id) ||
        (step.phase === "dfs1" && step.currentNode === node.id);
    const sccId = step.sccMap[node.id];

```

```

        fontWeight="bold"
      >
        SCC #{sccId}
      </text>
    })
  </g>
);
}}
</svg>

{step.transposed && (
  <div className="absolute top-4 left-4 bg-in
  1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>
)}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-4
  Порядок обхода и стек
  </h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-xs text-white leading-re
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-a
    {/* Top-Sort output / Order stack represent
    <div>
      <span className="text-[10px] text-slate-5
        СТЕК ВЫХОДА (DFS 1 ORDER):
      </span>
      <div className="flex flex-wrap gap-1 bg-s

```

```

    <div className="mt-4 pt-3 border-t border-slate-200">
      <span>
        Шаг {currentStepIdx + 1} из {steps.length}
      </span>
      <div className="flex gap-1.5">
        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev) => prev - 1)}
          className="bg-slate-900 border border-slate-200 px-2 py-1 rounded">
            Назад
          </button>
        <button
          disabled={currentStepIdx >= steps.length - 1}
          onClick={() => setCurrentStepIdx((prev) => prev + 1)}
          className="bg-slate-900 border border-slate-200 px-2 py-1 rounded">
            Вперед
          </button>
        </div>
      </div>
    </div>
  </div>
);
};

```

ФАЙЛ 53/87: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';

interface Vertex {
  x: number;
  y: number;
  id: string;
}

```

```
-----
{ id: '3-4', u: 3, v: 4, weight: 3, status: 'pending'
{ id: '1-2', u: 1, v: 2, weight: 4, status: 'pending'
{ id: '0-3', u: 0, v: 3, weight: 5, status: 'pending'
{ id: '1-4', u: 1, v: 4, weight: 6, status: 'pending'
{ id: '4-5', u: 4, v: 5, weight: 7, status: 'pending'
{ id: '3-6', u: 3, v: 6, weight: 8, status: 'pending'
{ id: '2-5', u: 2, v: 5, weight: 9, status: 'pending'
{ id: '6-7', u: 6, v: 7, weight: 10, status: 'pending'
{ id: '4-7', u: 4, v: 7, weight: 11, status: 'pending'
{ id: '7-8', u: 7, v: 8, weight: 12, status: 'pending'
{ id: '5-8', u: 5, v: 8, weight: 13, status: 'pending'
];
```

```
export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'
  const [vertices, setVertices] = useState<Vertex[]>(in
  const [edges, setEdges] = useState<Edge[]>(initialEdg
  const [stepIndex, setStepIndex] = useState<number>(0)
  const [heapQueue, setHeapQueue] = useState<string[]>(
  const [activeCheatTab, setActiveCheatTab] = useState<

  const [logs, setLogs] = useState<StepLog[]>([
    {
      title: 'Введение в раскраску (Краскал)',
      description: '«Краскал – это раскраска?». Предста
        овы начинать!',
      type: 'info',
    },
  ]));
```

```
// --- KRUSKAL STEPS ---
```

```
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Шаг 1: Берем самое дешевое ребро A-B (в
      description: 'Оно соединяет две бесцветные верши
```

```

        type: 'info'
    }, ...prev]);
},
// Step 6: Include B-C
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро B-C в осто́ве',
        description: 'Вершина C перекрашена в красный ц
        type: 'success'
    }, ...prev]);
},
// Step 7: Inspect A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Берем ребро A-D (вес 5)',
        description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
            ю – мы перекрашиваем всё в один цвет!"',
        type: 'warning'
    }, ...prev]);
},
// Step 8: Include A-D (Merge to purple)
() => {
    setVertices(prev => prev.map(v => v.color === 'blu
    setEdges(prev => prev.map(e => e.status === 'inclu
    setLogs(prev => [{
        title: 'Слияние кланов: Ребро A-D добавлено',
        description: 'Мы объединили красный и синий клан
        type: 'success'
    }, ...prev]);
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Берем ребро B-E (вес 6)',

```

```

        type: 'info'
    }, ...prev]);
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.',
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Берем ребро C-F (вес 9)',
        description: 'Оно соединяет вершины C и F, кото
        type: 'warning'
    }, ...prev]));
},
// Step 16: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро C-F.',
        type: 'error'
    }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Шаг 9: Берем ребро G-H (вес 10)',
        description: 'Соединяет фиолетовую G и бесцветн
        type: 'info'
    }, ...prev]));
},

```



```

    },
    // Step 22: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: '    Успех: MST построено Краскалом!',
            description: 'Все 9 вершин окрашены в единый фи
                + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
            type: 'success'
        }, ...prev]));
    },
];

// --- PRIM STEPS ---
const primSteps = [
    // Step 1: Start at E
    () => {
        setVertices(prev => prev.map(v => v.id === 4 ? {
        setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E
        setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5
        setLogs(prev => [{
            title: 'Шаг 1: Плесень зарождается в Токио (Вер
            description: 'Мы выбрали вершину E как стартовую
                ритетную очередь (Heap): D-E(3), B-E(6), E-F(
            type: 'info'
        }, ...prev]));
    },
    // Step 2: Pop D-E (3)
    () => {
        setEdges(prev => prev.map(e => e.id === '3-4' ? {
        setLogs(prev => [{
            title: 'Шаг 2: Куча выдает самое дешевое ребро
            description: 'Плесень тянется по ребру D-E к вер
            type: 'info'
        }, ...prev]));
    },
    // Step 3: Include D-E, add D's edges to heap

```

```

        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-E(6) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
        description: 'Плесень тянется к вершине C.',
        type: 'info'
    }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
        eap' } : e));
    setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (9)']);
    setLogs(prev => [{
        title: 'Успех: Вершина C захвачена',
        description: 'Ребро C-F(9) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {

```

```
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
    description: 'Плесень расширяется вниз к G.',
    type: 'info'
  }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
    eap' } : e));
  setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
  setLogs(prev => [{
    title: 'Успех: Вершина G захвачена',
    description: 'В кучу добавлено G-H(10).',
    type: 'success'
  }, ...prev]));
},
// Step 16: Pop C-F (9) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '2-5' ? {
  setLogs(prev => [{
    title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
    description: 'Вершины C и F уже заражены плесенью',
    type: 'warning'
  }, ...prev]));
},
// Step 17: Reject C-F
() => {
  setEdges(prev => prev.map(e => e.id === '2-5' ? {
  setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
  setLogs(prev => [{
    title: 'Отказ: Игнорируем ребро C-F',
    description: 'Выбрасываем из кучи.',
    type: 'error'
  }, ...prev]));
},
```

```

        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]);
},
// Step 22: Pop H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 12: Куча выдает H-I (вес 12)',
        description: 'Плесень тянется к последней чистой',
        type: 'info'
    }, ...prev]));
},
// Step 23: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setHeapQueue(['F-I (13 - цикл)']));
    setLogs(prev => [{
        title: '    Успех: Плесень захватила весь граф
        description: 'Все 9 вершин поглощены. Итоговый
            иной – мы разрастались из одной точки с помо
        type: 'success'
    }, ...prev]));
},
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
    // Step 1: Five-Year Plan 1 (Inspecting outgoing)
    () => {
        // Highlight the cheapest outgoing edges for EACH
        // Node 0(A) -> A-B(2)
        // Node 1(B) -> A-B(2)
        // Node 2(C) -> B-C(4)
        // Node 3(D) -> D-E(3)
        // Node 4(E) -> D-E(3)
        // Node 5(F) -> E-F(7)
    }
];

```

```

-----
// Step 3: Five-Year Plan 2 (Inspecting bridge between
() => {
  // Now both kolkhozes look at all outgoing roads
  // Outgoing roads between {A, B, C} and {D, E, F},
  // A-D (5), B-E (6), C-F (9).
  // Cheapest is A-D (5).
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
  setLogs(prev => [{
    title: 'Вторая пятилетка: Колхозы выбирают мост
    description: 'Теперь Красный и Синий колхозы од
      -D с весом 5. Остальные мосты B-E(6) и C-F(9)
    type: 'warning'
  }, ...prev]));
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
  setVertices(prev => prev.map(v => ({ ...v, color:
  setEdges(prev => prev.map(e =>
    e.status === 'included' || e.id === '0-3' ? { .
  ));
  setLogs(prev => [{
    title: 'Объединение завершено: Один гигантский
    description: 'Оба колхоза объединились по мосту
    type: 'success'
  }, ...prev]));
},
// Step 5: Check remaining edges & mark as rejected
() => {
  setEdges(prev => prev.map(e => e.status === 'pend
  setLogs(prev => [{
    title: ' Успех: MST построен Борувкой за 2 шаг
    description: 'Все лишние ребра (B-E, E-H, C-F,
      вес: 51. Благодаря параллельности, мы потрати
    type: 'success'
  }, ...prev]));
}
];

```

```

        title: 'Введение в Коллективизацию (Борувка)',
        description: 'Логика «Борувки»: Каждая из 9 д
            чтобы объединиться в колхоз.',
        type: 'info',
    },
  ]);
}
};

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex
    case 'blue': return 'bg-blue-500 border-blue-300
    case 'purple': return 'bg-purple-600 border-purple
    case 'mold': return 'bg-emerald-500 border-emerald
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIn
        return 'bg-slate-700 border-emerald-500 text-
      }
      return 'bg-slate-700 border-slate-500 text-slate
    }
  };

const getEdgeStroke = (status: string, color: string)
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
    if (color === 'mold') return '#10b981';
  }
  return '#475569';
};

return (
  <div className="space-y-12">

```

```

        (activeAlgo === 'prim'
        ? 'bg-emerald-600 text-white shadow-md rounded-xl border-2
        : 'text-slate-400 hover:text-slate-300
    }
  >
    <Layers className="w-3.5 h-3.5" />
    <span>Прима (Билет 19)</span>
  </button>
  <button
    onClick={() => handleReset('boruvka')}
    className={
      "flex items-center gap-1.5 px-3 py-2
+
      (activeAlgo === 'boruvka'
      ? 'bg-rose-600 text-white shadow-md rounded-xl border-2
      : 'text-slate-400 hover:text-slate-300
    }
  >
    <Users className="w-3.5 h-3.5" />
    <span>Борувка (Билет 20)</span>
  </button>
</div>

<button
  onClick={() => handleReset(activeAlgo)}
  className="flex items-center justify-center gap-2 font-
-200 font-semibold text-xs rounded-xl border-2
>
  <RotateCcw className="w-3.5 h-3.5" />
  <span>Сбросить</span>
</button>

<button
  onClick={handleNextStep}
  disabled={stepIndex >= currentSteps.length}
  className={
    "flex items-center justify-center gap-2 font-
initial cursor-pointer " +

```

```

    ) : activeAlgo === 'prim' ? (
      <
        <div className="flex items-center gap-2">
          <div className="w-3 h-1 bg-sky-600">
          </div>
          <div className="flex items-center gap-2">
            <div className="w-3 h-3 rounded-full">
            </div>
            </div>
          </div>
        </div>
      ) : (
        <
          <div className="flex items-center gap-2">
            <div className="w-3 h-3 rounded-full">
            </div>
            <div className="flex items-center gap-2">
              <div className="w-3 h-3 rounded-full">
              </div>
              <div className="flex items-center gap-2">
                <div className="w-3 h-3 rounded-full">
                </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      )
    </div>

    <div className="absolute top-4 right-4 bg-slate-300">
      {stepIndex} / {currentSteps.length}
    </div>

    {/* SVG Edges */}
    <svg className="absolute inset-0 w-full h-full" style={{
      aspectRatio:"none"
    }}>
      {edges.map(edge => {
        const u = vertices[edge.u];
        const v = vertices[edge.v];
        const strokeColor = getEdgeStroke(edge);
        const isInspecting = edge.status === 'inspecting';
        const isRejected = edge.status === 'rejected';

```



```

        key={vertex.id}
        style={{ top: vertex.y + "%", left: v
        className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
      >
        <span>{vertex.label}</span>
        {vertex.id === 4 && activeAlgo === 'p
          <span className="text-[8px] block -
        }}
      </div>
    )))
  </div>

  {stepIndex >= currentSteps.length && (
    <div className="absolute inset-x-6 bottom
shadow-xl z-20">
      <p className={"font-bold text-base " +
ld-300' : 'text-rose-300'}}>
        MST построено с помощью {activeAlgo
      </p>
      <p className="text-slate-300 text-xs mt
        Общий вес минимального остова: 51. Вс
      </p>
    </div>
  ))
</div>

  {/* Logs / Heap */}
  <div className="bg-slate-950 rounded-2xl bord
    <div className="p-4 bg-slate-900 border-b b
      <h4 className="font-bold text-slate-200 t
        Ход выполнения
      </h4>
    </div>
  </div>

  {activeAlgo === 'prim' && (
    <div className="p-3 bg-slate-900/60 borde

```

```
        <h5 className="font-bold text-sm text-slate-300">
      </div>
      <p className="text-xs text-slate-300">
    </div>
  )})
</div>
</div>
</div>
</div>
```

```
/* Cheat Sheet: Complexity & Code for all 3 algo
```

```
<div className="bg-slate-900/80 border border-slate-800 p-4">
  <div className="flex flex-col sm:flex-row items-center gap-2.5">
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-400" />
      <h4 className="text-xl font-bold text-white">
    </div>
```

```
<div className="flex bg-slate-950 p-1 rounded gap-2">
  <button
    onClick={() => setActiveCheatTab('kruskal')}
    className={
      "px-3 py-1.5 rounded text-xs font-medium transition-colors"
    }
    == 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"
  >
```

Краскал (Билет 18)

```
</button>
```

```
<button
```

```
  onClick={() => setActiveCheatTab('prim')}
  className={
    "px-3 py-1.5 rounded text-xs font-medium transition-colors"
  }
  == 'prim' ? "bg-emerald-600 text-white" : "bg-slate-800 text-slate-300"
>
```

Прима (Билет 19)

```
</button>
```

```
<button
```

```
  onClick={() => setActiveCheatTab('boruvka')}
  className={
    "px-3 py-1.5 rounded text-xs font-medium transition-colors"
  }
  == 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-800 text-slate-300"
>
```

```

    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

```

```

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

```

2. Жадный выбор ребер

edges.sort() # $O(E \log E)$

mst = []

for w, u, v in edges:

if union(u, v):

mst.append((u, v, w))`}

</pre>

</div>

</div>

</div>

}}

{activeCheatTab === 'prim' && {

<div className="grid grid-cols-1 lg:grid-cols-

<div className="lg:col-span-1 space-y-4">

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Clock className="w-3.5 h-3.5 text-em

</p>

<p className="text-2xl font-black text-v

<p className="text-slate-400 text-xs mt

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Award className="w-3.5 h-3.5 text-em

</p>

<p className="text-2xl font-black text-v


```

highlightColor: 'text-emerald-400',
rest: 'весами. Подсунь отрицательное ребро – сломает',
complexity: 'O((V+E) log V)',
border: 'border-blue-500/30 hover:border-blue-400/60',
titleColor: 'text-blue-400',
badge: 'text-blue-400/70 bg-blue-950/40',
},
{
  img: bellmanImg,
  alt: 'Фура по болоту Форд-Беллман',
  title: '  Фура по Болоту (Ф-Б)',
  desc: 'Медленный, тяжёлый – зато тащит',
  highlight: 'отрицательные',
  highlightColor: 'text-rose-400',
  rest: 'веса и детектит отрицательные циклы.',
  complexity: 'O(V · E)',
  border: 'border-rose-500/30 hover:border-rose-400/60',
  titleColor: 'text-rose-400',
  badge: 'text-rose-400/70 bg-rose-950/40',
},
{
  img: floydImg,
  alt: 'Все-ко-Все Флойд',
  title: '  Все ко Все (Флойд)',
  desc: 'Матрица + три цикла (',
  highlight: 'k, i, j',
  highlightColor: 'text-emerald-400',
  rest: ')). Ищет пути от всех ко всем.',
  complexity: 'O(V³)',
  border: 'border-emerald-500/30 hover:border-emerald-400/60',
  titleColor: 'text-emerald-400',
  badge: 'text-emerald-400/70 bg-emerald-950/40',
},
];

```

```

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap

```

```
/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
```

```
export const MobileToc: React.FC<Props> = ({ selectedCh
  const [open, setOpen] = useState(false);
```

```
  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : ""
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);
```

```
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escap
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
  }, []);
```

```
  return (
    <>
      <div className="lg:hidden sticky top-[57px] z-40
        <button
          onClick={() => setOpen(true)}
          className="w-full flex items-center gap-2.5 p-2
          aria-label="Открыть содержание"
        >
          <span className="p-1.5 rounded-lg bg-indigo-600
            <List className="w-4 h-4" />
          </span>
          <span className="min-w-0 flex-1">
            <span className="block text-[10px] uppercase
              Содержание • {index + 1} из {total}
            </span>
            <span className="block text-[13px] font-bol
          </span>
        </button>
```

```
-----
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
  activeTab: 'guide' | 'simulator';
  setActiveTab: (tab: 'guide' | 'simulator') => void;
  /** Открыта ли боковая панель компилятора. */
  compilerOpen: boolean;
  onToggleCompiler: () => void;
}

export const Navbar: React.FC<NavbarProps> = ({ activeTab
  const [busy, setBusy] = useState(false);

  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
              <h1 className="text-xl font-extrabold text-v
                Универсальное пособие
              </h1>
              <p className="text-xs text-indigo-400 font-
```

```

        uration-200 whitespace-nowrap ${
            compilerOpen
            ? 'bg-emerald-600 text-white shadow-lg
            : 'text-slate-400 hover:text-white'
        }` }
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={`${import.meta.env.BASE_URL}export/full_
    download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
    over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех страниц"
    >
    <FileDown className="w-4 h-4" /> Скачать PDF
</a>
<a
    id="download-txt-btn"
    href={`${import.meta.env.BASE_URL}export/full_
    download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
    :bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом"
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center
    er:bg-amber-600/20 border border-amber-500/

```



```

        [1,2],[1,3],[1,4],
        [2,3],[2,4],
        [3,4]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
    if (a===c||a===d||b===c||b===d) return
    const p1=findPt(a),p2=findPt(b),p3=find
    function ccw(ax:number,ay:number,bx:num
    return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
    }
    return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.
    && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
        for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1]
            if (!crosses.includes(i)) crosses.p
            if (!crosses.includes(j)) crosses.p
        }
    }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(
        const xing = crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x
        stroke={xing ? "#f43f5e" : "#4f46e5"}
    }
    const labels = ["A","B","C","D","E"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill=
            <text x={p.x} y={p.y+3} textAnchor="m
        </g>);
    }
    return <>{lines}{circles}</>;

```

```

        if (!crosses.includes(j)) crosses.push(j)
    }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
    const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
    const xing=crosses.includes(i);
    lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y} stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
}
const labels = ["\u04141", "\u04142", "\u04143"];
for(const p of pts) {
    circles.push(<g key={`c${p.id}`}>
        <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#f43f5e" stroke-width={2}/>
        <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
    </g>);
}
return <>{lines}{circles}</>;
})()
</svg>
<div className="absolute bottom-2 left-2 right-2 z-20">
     $V=6, E=9 \Rightarrow 2V-E=3$  (двудольный)
</div>
</div>
</div>
<div className="bg-amber-950/40 border border-amber-950/40 p-10">
    ▲ Запомни:  $K_5$  и  $K_{3,3}$  — два кита непланарности.
    гомеоморфный  $K_5$  или  $K_{3,3}$  — он непланарен. Теорема Кюри.
</div>
</div>
);
}

```

```

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index-guest.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.html`;
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed";
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as const;

interface PythonCompilerProps {
  /** Страница, с которой синхронизируется редактор. */
  chapterId: string;
  chapterTitle: string;
  /** Размер панели (px) управляется родителем и сохраняется */
  width?: number;
  height?: number | null;
  onWidthChange?: (w: number) => void;
  onHeightChange?: (h: number) => void;
  /** Перейти к странице пособия (посмотреть визуализацию) */
  onOpenGuide: () => void;
  /** Скрыть панель. */
  onClose: () => void;
}

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL);
  }
}

```

```

        label: "sorted(key=...)",
        tip: "Сортировка по ключу: рёбра по весу, пары по...",
        code: 'edges = [(3, "A", "B"), (1, "B", "C"), (2, "A", "C")]\n',
    },
],
},
{
    group: "Циклы",
    items: [
        {
            label: "for i in range(n)",
            tip: "Вложенные циклы по i и j – как обход матрицы",
            code: "n, m = 3, 4\nfor i in range(n):",
        },
        {
            label: "while ...",
            tip: "Классический while-счётчик – как n-1 итерация",
            code: "i = 1\nwhile i < 5:\n    print(f\"итерация {i}\")",
        },
        {
            label: "for i, x in enumerate(...)",
            tip: "Индекс и элемент одновременно: i – позиция, x – элемент",
            code: "for i, x in enumerate(['a', 'b', 'c']):",
        },
    ],
},
{
    group: "Структуры",
    items: [
        {
            label: "Стек (список)",
            tip: "LIFO, как рекурсия DFS: append = push, pop = pop",
            code: 'st = []\nst.append("A")    # push\nst.pop()    # pop',
        },
        {
            label: "Очередь (deque)",
            tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
            code: 'from collections import deque\nq = deque()\nq.append("A")    # push\nq.popleft()    # pop',
        },
    ],
},

```

```

/** Активная вкладка демонстрации страницы (у sparse-
const [demoId, setDemoId] = useState<string | null>(null)
const sync = useMemo(() => getPageSync(chapterId, demoId))
const syncVarBases = useMemo(
  () =>
    [...new Set(
      sync.variables.flatMap((v) =>
        v.name
          .split(/\\s*(?:,|\\/)\s*/)
          .map(baseVarName)
          .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/))
    )],
  [sync]
);
const syncVarSet = useMemo(() => new Set(syncVarBases))
/** Полный эталон: глаз сразу вставляет его в обычный
const template = useMemo(() => buildSyncTemplate(chapterId, demoId))
/** Дефолт редактора: только инициализация демо; она
const initTemplate = useMemo(() => buildInitTemplate(chapterId, demoId))

const [code, setCode] = useState(() => {
  if (!savedEditor) return initTemplate;
  if (savedEditor.code === savedEditor.template || savedEditor.code === savedEditor.template)
    return savedEditor.code;
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится");
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState("");
/** Принудительный повтор (глаз / Ctrl+Enter), даже если
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "highlight">("vars");
const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);

```

```

// Смена страницы (или вкладки демо): нетронутый редактор
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
  const prev = prevTplRef.current;
  if (initTemplate === prev.base && template === prev.full) {
    prevTplRef.current = { base: initTemplate, full: template };
    setPlaying(false);
    setEditHold(null);
    setDebug(null); // трасса ссылается на строки старого кода
    if (code === prev.base || code === prev.full || code === prev.partial) {
      setCode(initTemplate);
      setDesyncedFrom(null);
    } else {
      setDesyncedFrom(chapterId);
    }
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
  prevTplRef.current = { base: initTemplate, full: template };
  setCode(initTemplate);
  setDesyncedFrom(null);
  setStripTab("vars");
  setPlaying(false);
  setEditHold(null);
  setDebug(null);
  setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
    setRuntimeMessage("Не получилось скопировать – буфер переполнен");
  }
}, [code]);

```

```

        func: step?.func,
        locals,
        variables,
        changed: Object.keys(changed).filter((name) => ch
    });
}, []));

const publishDebug = useCallback(
  (result: DebugResult, idx: number, source: string) => {
    if (!hasViz || result.steps.length === 0) return;
    const snapshot = snapshotAt(result, idx, source);
    emitVizState(chapterId, {
      line: snapshot.line,
      func: snapshot.func,
      variables: snapshot.variables,
      changed: snapshot.changed,
    });
  },
  [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
  if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
  (nextIdx: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps
      return { ...d, idx };
    });
  },
  []
);

```

```

    setPlaying(false);
    setEditHold(null);
    setDebug(null);
    clearVizState(chapterId);
    setOutputOpen(false);
    setRuntimeMessage(runtimeReady ? "Обновляю трассу..."

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
    const runtime = await getPythonRuntime();
    setRuntimeReady(true);
    runtime.setStdout({ batched: (message) => stdout.push(message) });
    runtime.setStderr({ batched: (message) => stderr.push(message) });
    runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });

    const raw = await runtime.runPythonAsync(buildDebugRequest(chapterId, code, stdin));
    const result = JSON.parse(String(raw)) as DebugRequestResult;

    setDebug({ result, idx: 0, src: source, input, runtimeReady });
    const printed = [...stdout, ...stderr].join("\n");
    setOutput(printed || "Программа ничего не вывела");
    setRuntimeMessage(
        result.truncated
            ? `Выполнение остановлено после ${DEBUG_STEP_LIMIT} шагов`
            : "Трасса готова автоматически. ← → — шаги, Ctrl+Enter — продолжить"
    );
} catch (error) {
    const message = errorText(error);
    setDebug({ result: { steps: [], error: message, truncated: true }, idx: 0, src: source, input, runtimeReady });
    setOutput(`Ошибка:\n${message}`);
    setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
} finally {
    setRunning(false);
}

}, [chapterId, code, stdin, traceRequest, running, runtimeReady]);

```



```

    return () => window.removeEventListener("keydown",
    }, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
const curDebug = debug ? { step: debug.result.steps[d
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
    if (curDebugLine === undefined || !listRef.current)
    listRef.current
        .querySelector(`[data-line="${curDebugLine}"]`)
        ?.scrollIntoView({ block: "nearest" });
}, [curDebugLine]);

// производные данные шага: что изменилось, в каком п
// Строка, которую назначает текущий шаг, подсвечивае
// локали берём со следующего шага трассы (для послед
// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
    if (!debug || !curDebug?.step) return {};
    return snapshotAt(debug.result, debug.idx, debug.sr
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(cu
const debugOrdered = curDebug?.step ? orderVars(shown

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string) =>
    const ta = editorRef.current;
    setCode((current) => {
        if (!ta) return current + (current.endsWith("\n")
        const start = ta.selectionStart ?? current.length
        const end = ta.selectionEnd ?? start;
        const padded = (start > 0 && !current.slice(0, st
        const next = current.slice(0, start) + padded + c
        requestAnimationFrame(() => {
            ta.focus();
            ta.selectionStart = ta.selectionEnd = start + p
        });
        return next;
    });

```

```

        el.addEventListener("pointerup", onUp as EventListener)
        el.addEventListener("pointercancel", onUp as EventListener)
    })
>
    <div className="absolute left-1/2 top-1/2 h-10 w-10
        digo-400 transition-colors" />
</div>
})

{/* Левый край меняет ширину на десктопе. */}
{onWidthChange && (
    <div
        role="separator"
        aria-orientation="vertical"
        aria-label="Потяните, чтобы изменить ширину панели"
        title="Потяните, чтобы изменить ширину панели"
        className="hidden lg:block absolute top-0 bottom-0
        onPointerDown={(e) => {
            const el = e.currentTarget;
            el.setPointerCapture(e.pointerId);
            const startX = e.clientX;
            const startW = width ?? 560;
            const onMove = (ev: PointerEvent) => {
                const next = Math.round(startW + (startX - ev.clientX) * 2);
                onWidthChange(Math.max(380, Math.min(next, 1000)));
            };
            const onUp = () => {
                el.removeEventListener("pointermove", onMove);
                el.removeEventListener("pointerup", onUp);
            };
            el.addEventListener("pointermove", onMove);
            el.addEventListener("pointerup", onUp as EventListener);
        })
    >
        <div className="absolute left-1/2 top-1/2 -tr
            digo-500 transition-colors" />
    </div>
)}

```

```

<Tooltip
  side="bottom"
  content="Вставить полный эталонный код этого"
>
  <button
    type="button"
    onClick={() => {
      setCode(template);
      setDesyncedFrom(null);
      setPlaying(false);
      setEditHold(null);
      setDebug(null);
      setTraceRequest((n) => n + 1);
    }}
    className="p-1.5 rounded-lg text-indigo-300"
    aria-label="Вставить эталонный код"
  >
    <Eye className="w-4 h-4" />
  </button>
</Tooltip>
<label
  className="ml-1 inline-flex items-center gap-2"
  style={{border: '1px solid #444', padding: '2px 5px', transition: 'border-color 0.2s'}}
  title="Скорость автоматического проигрывания"
>
  {running ? <Loader2 className="w-3.5 h-3.5" style={{border: '1px solid #444'}} /> : null}
  <span>{running ? "авто..." : "авто"}</span>
  <select
    aria-label="Скорость автопрохода"
    value={playbackSpeed}
    onChange={(event) => setPlaybackSpeed(Number(event.target.value))}
    className="cursor-pointer rounded bg-slate-800 text-white"
  >
    {PLAYBACK_SPEEDS.map((speed) => (
      <option key={speed} value={speed}>{speed}
    ))}
  </select>

```

```

        hasViz ? "text-emerald-400" : "text-slate-400"
      }` }
    >
    <Link2 className="w-3 h-3" />
    {hasViz ? "синхронизировано" : "нет визита"}
  </span>
</Tooltip>
</div>
</div>

{stripTab === "vars" ? (
  <div className="px-3 pb-2.5 space-y-2 max-h-300">
    <div className="flex items-center gap-2 min-w-300">
      <Tooltip side="bottom" content="Открыть эскиз">
        <button
          type="button"
          onClick={onOpenGuide}
          className="inline-flex items-center gap-1.5 px-1.5 py-1.5 text-slate-300 hover:text-white hover:bg-slate-700"
        >
          <span className="truncate">{chapterTitle}</span>
        </button>
      </Tooltip>
    </div>
    {desynchedFrom !== null && (
      <Tooltip
        side="bottom"
        content="В редакторе ваш код, а страница была заменена."
      >
        <button
          type="button"
          onClick={resync}
          className="flex items-center gap-1.5 px-1.5 py-1.5 text-amber-400 hover:bg-amber-500/20 hover:text-white"
        >
          <Unlink className="w-3.5 h-3.5 shrink-0" />
          Страница сменилась – подставить её по умолчанию
        </button>
      </Tooltip>
    )}
  </div>
) : null}

```

```

        </Tooltip>
      })
    </div>
  })
</div>
) : (
  <div className="px-3 pb-2.5 max-h-36 overflow
    {SNIPPETS.map(({gItem}) => {
      <div key={gItem.group}>
        <div className="text-[9px] font-bold up
        <div className="flex flex-wrap gap-1.5"
          {gItem.items.map(({s}) => {
            <Tooltip key={s.label} side="bottom
              <button
                type="button"
                onClick={() => insertSnippet(s.
                className="inline-flex items-ce
                -[10.5px] text-indigo-300 hover:text-white
                title=""
              >
                <Plus className="w-3 h-3 text-s
                  {s.label}
                </button>
              </Tooltip>
            )})
          </div>
        </div>
      )})
    </div>
  )})
  <p className="text-[10px] text-slate-600">H
</div>
})
</div>

```

```

/* — Редактор + ввод
<div className="flex-1 min-h-0 flex flex-col">
  {debug ? (
    <div className="flex-1 min-h-[80px] flex flex
      /* панель управления шагами */

```

```

        <ChevronRight className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip content="В конец трассы">
      <button type="button" onClick={() => {
        tep || debug.idx >= debug.result.steps.length
        0 disabled:opacity-40 transition-colors">
        <SkipForward className="w-4 h-4" />
      </button>
    </Tooltip>
    <span className="ml-1.5 text-[10px] text-
      {curDebug?.step ? (
        <>
          шаг {debug.idx + 1}/{debug.result.steps.length}
          {curDebug.step.func !== "<module>" ? " " : ""}
        </>
      ) : (
        "шагов нет – исправьте ошибку и начните трассировку"
      )}
    </span>
    {debug.result.truncated && (
      <Tooltip content={`Выполнение остановлено по
        <span tabIndex={0} className="cursor-help rounded-full px-1.5 py-0.5">
          ЛИМИТ
        </span>
      </Tooltip>
    )}
    <Tooltip content={hasViz ? "Визуализация трассы.
    алонный текст не требуется." : "У страницы нет трассы"}>
      <span
        tabIndex={0}
        className={`cursor-help inline-flex items-center gap-1
        "border-emerald-500/40 bg-emerald-500/10 text-emerald-500">
        >
        {hasViz ? <Link2 className="w-3 h-3" href="#">
          {hasViz ? "имена → демо" : "свободный текст"}
        </span>

```

```

<div className="flex items-center gap-1.5"
  <Variable className="w-3 h-3 text-indigo-500" value={value} />
  Переменные на этом шаге
  <span className="normal-case tracking-normal" />
</div>
{debug.result.error && (
  <pre className="mt-1 whitespace-pre-wrap" />
)}
{debugOrdered.length === 0 && !debug.result.error && (
  <div className="mt-1 text-[11px] text-slate-500" />
)}
</div>
)}
<div className="mt-1 flex flex-wrap gap-1"
  {debugOrdered.map(([name, value]) => {
    const changed = debugChanged[name];
    const synced = syncVarSet.has(name);
    return (
      <span
        key={name}
        className={`inline-flex items-baseline gap-1.5
          ${changed ? "border-amber-500/50 bg-amber-500/10" : ""}
          ${synced ? "border-emerald-500/40 bg-emerald-500/10" : ""}
          ${!changed & !synced ? "border-slate-700 bg-slate-700/10" : ""}
        `}
      >
        <span className={changed ? "text-amber-500" : ""}>{value}</span>
        <span className="text-slate-500">{name}</span>
        <span className={changed ? "text-amber-500" : ""}>{value}</span>
      </span>
    );
  })}
</div>
</div>
</div>

```

```

    </div>

    { /* — Вывод
    <div className="shrink-0 border-t border-slate-800"
      <button
        type="button"
        onClick={() => setOutputOpen((o) => !o)}
        className="flex items-center justify-between"
        colors"
      >
        <span className="inline-flex items-center gap-2"
          <Terminal className="w-3.5 h-3.5 text-emerald-500"
            </span>
          <span className="inline-flex items-center gap-2"
            {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500"
              <span className="text-slate-600">{outputOpen ? 'Закрыть' : 'Открыть'}</span>
            </span>
          </span>
        </button>
        {outputOpen && (
          <pre className="flex-1 min-h-[70px] overflow-hidden leading-5 text-slate-300">
            {output}
          </pre>
        )}
      <div className="px-3 py-1.5 border-t border-slate-800"
    </div>
  </aside>
);
}

```

ФАЙЛ 59/87: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';

```



```

const runtime = useVizRuntime();
const liveQ = vizArray(runtime?.variables?.q);
const liveV = vizString(runtime?.variables?.v);
const displayQueue: Customer[] = liveQ
  ? liveQ.map((value, index) => ({
      id: `python-${index}`,
      name: String(value),
      emoji: " ",
      color: "from-emerald-500/20 to-emerald-600/20 b",
      bubbleText: `q[${index}] из Python`,
      animState: "idle",
    }))
  : queue;

const [servedHistory, setServedHistory] = useState<st
const [isServing, setIsServing] = useState(fa
const [shakeEmpty, setShake] = useState(fa
const [log, setLog] = useState<st

const addLog = (msg: string) => setLog(prev => [msg,

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
  if (isServing) return;
  if (queue.length >= 6) {
    addLog('⚠ Хвост очереди уже на улице, повару тяже
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

const preset = PRESETS[counter % PRESETS.length];
counter++;

const newGuest: Customer = {
  id: Date.now().toString(),
  ...preset,
  animState: 'in',

```

```

    }, 400);
  }, 900);
}, [queue, isServing]));

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: '🍴', color: 'red', text: 'Я  
т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: '🍴', color: 'blue', text: 'Я  
апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'green', text: 'Я  
без сметаны, плиз.', animState: 'in' },
  ]);
  setServedHistory([]);
  setIsServing(false);
  setLog([' Столовая сброшена. Снова голодная толпа!']);
  setTimeout(() => {
    setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
  });

  return (
    <div className="flex flex-col gap-6">
      {/* МНЕМОНИКА / ПРАВИЛО */}
      <div className="bg-indigo-950/40 border border-indigo-500 p-6">
        <div className="text-3xl"> </div>
        <div>
          <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
          <p className="text-xs text-slate-300 mt-1 leading-tight">
            В очереди за супом всё честно. Кто первый встал, тот и ест.  

            Новые гости встают строго <b>в конец</b> очереди.  

            Здесь нет обхода и блата — только строгий порядок.
          </p>
        </div>
      </div>
    </div>

    {/* УПРАВЛЕНИЕ */}
  );
};

```

```
    {liveQ && <span className="ml-2 text-emerald-500">
</div>
```

```
<div className="flex-1 flex flex-col justify-between">
  <div className="flex items-end justify-between">
```

```
    {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
    <div className="flex flex-col items-center">
      <div className="relative">
        <div className="absolute -top-10 left-1/2 transform translate-x-50%
          <div className="w-1.5 h-6 bg-slate-800 rounded" />
          <div className="w-2 h-4 bg-slate-400 rounded" />
        </div>
        <span className="text-5xl block animate-pulse">
      </div>
      <div className="text-center">
        <span className="text-3xl block">
        <span className="text-[10px] font-bold font-mono tracking-wider">
          ПОВАР
        </span>
      </div>
    </div>
```

```
    {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
    <div className="flex flex-col items-center">
      <span className="text-2xl animate-pulse">
      <span className="text-[8px] font-mono tracking-wider">
    </div>
```

```
    {/* СПИСОК ОЧЕРЕДИ */}
    <div className="flex-1 flex items-end gap-4">
      {displayQueue.length === 0 ? (
        <div className="flex-1 flex flex-col justify-between">
          <span className="text-4xl mb-1">
          <p className="text-xs font-bold font-mono">
          <p className="text-[10px]">Все сыты
        </div>
      ) : (
```

```

        absolute -bottom-2 text-[10px] font-mono
        ${isHead ? 'bg-amber-500' : 'bg-slate-900'}
      `}>
      {isHead && isTail ? 'h+t' : 't'}
    </span>
  })
</div>
</div>
);
})
})
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-amber-500 to-slate-900">
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
  <div className="absolute top-4 left-4 text-[10px] font-mono">
    Сытые гости (Архив FIFO)
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-end gap-2 font-mono">
    <div className="order-emerald-800/80 text-[9px] font-mono">
      <Sparkles className="w-3.5 h-3.5 animate-pulse">
        <span>СЫТЫ </span>
      </div>
    </div>

    <div className="flex-1 flex flex-col justify-end gap-2">
      {servedHistory.length === 0 ? (
        <div className="flex-1 flex flex-col items-center justify-center gap-2">
          <span className="text-5xl mb-2"> </span>
          <p className="text-xs font-bold font-mono">Здесь 6
          <p className="text-[10px] mt-1">Здесь 6

```

ФАЙЛ 60/87: src/components/SalmonAutomatonWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';

interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
  x?: number;
  y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR', 'FISH', 'SALMON']);
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: TrieNode }>({});
  const [simulationStep, setSimulationStep] = useState<number>(0);
  const [bfsQueue, setBfsQueue] = useState<number[]>([]);
  const [currentNode, setCurrentNode] = useState<number>(0);
  const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и попробуй!');
  const [isTalking, setIsTalking] = useState<boolean>(false);
  const [isBlinking, setIsBlinking] = useState<boolean>(false);

  // Регулярное моргание рыбы
```

```

        depth: newNodes[curr].depth + 1
      };
    }
    curr = newNodes[curr].children[c];
  }
  if (!newNodes[curr].words.includes(word)) {
    newNodes[curr].words.push(word);
    newNodes[curr].is_terminal = true;
  }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
  const childKeys = Object.values(newNodes[u].children);
  newNodes[u].y = 40 + depth * paddingY;

  if (childKeys.length === 0) {
    newNodes[u].x = 40 + currentX * paddingX;
    currentX++;
  } else {
    let leftX = -1;
    let rightX = -1;
    childKeys.forEach(v => {
      layoutDFS(v, depth + 1);
      if (leftX === -1) leftX = newNodes[v].x!;
      rightX = newNodes[v].x!;
    });
    newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX;
    if (leftX === -1) currentX++;
  }
};

layoutDFS(0, 0);

setNodes(newNodes);

```

```

const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];
  const updatedNodes = { ...nodes };
  rootChildren.forEach((childId) => {
    updatedNodes[childId].fail = 0;
  });

  setNodes(updatedNodes);
  setBfsQueue(queue);
  setSimulationStep(1);
  setCurrentNode(null);
  setSpeechText(
    'Начинаем обход в ширину (BFS)! Все вершины на графике  

    уже суффикса просто нет. Жми «Следующий шаг»!'
  );
};

// Один шаг BFS
const stepBFS = () => {
  if (bfsQueue.length === 0) {
    setSimulationStep(2);
    setCurrentNode(null);
    setSpeechText(
      'Ура! Очередь пуста! Мы успешно построили полный  

      граф и полюбоваться таблицей переходов!'
    );
    return;
  }

  const r = bfsQueue[0];
  const newQueue = bfsQueue.slice(1);
  const updatedNodes = { ...nodes };
  const rNode = updatedNodes[r];
  const childrenIds = Object.values(rNode.children);

  let logs = `Рассматриваем вершину #${r} (${rNode.children.length}

```

```

        </h4>
        <p className="text-sm text-slate-400">
            Построение дерева и расчет суффиксных ссы
        </p>
    </div>
</div>
<div className="flex items-center gap-2">
    <button
        onClick={() => buildInitialTrie(words)}
        className="flex items-center gap-1 bg-slate
            transition-colors"
        title="Сбросить автомат"
    >
        <RotateCcw className="w-4 h-4" /> Сбросить
    </button>
</div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
    {/* Аватар Лосося (Сеня) с анимацией моргания и
    <div className="flex flex-col items-center just
        <div className="w-40 h-40 relative flex items
            full border border-blue-500/30 shadow-inner
            {/* SVG Лосося */}
        <svg
            viewBox="0 0 200 200"
            className={`w-36 h-36 transition-transform
                isTalking ? 'scale-105 drop-shadow-[0_0
            ]`}
        >
            {/* Тело лосося */}
            <ellipse cx="100" cy="100" rx="70" ry="45
            <path d="M 40 85 Q 100 55 160 85 Q 100 115
                40 85 Z" />

            {/* Хвост */}
            <path
                d="M 35 100 L 5 70 L 15 100 L 5 130 Z"

```



```

        <div className="absolute top-8 right-1 text
    </div>
    <span className="mt-3 bg-rose-600/30 border b
        Эхо-Лосось Сеня
    </span>
</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
    tween min-h-[140px]">
    {/* Треугольник баббла */}
    <div className="absolute -left-3 top-1/2 -tra
        slate-800 border-b-[12px] border-b-transpar

    <div className="flex items-center space-x-2 t
        <Volume2 className={`w-4 h-4 ${isTalking ?
        <span>Прямое вещание из аквариума:</span>
    </div>

    <p className="text-slate-200 text-base leadin
        {speechText}
    </p>

    <div className="mt-4 pt-3 border-t border-sla
        <div className="text-xs text-slate-400 flex
            <Info className="w-3.5 h-3.5 text-blue-40
            Статус: {simulationStep === 0 ? 'Бор гото
            оен!'}
        </div>

        <div className="flex gap-2">
            {simulationStep === 0 && (
                <button
                    onClick={startBFS}
                    className="bg-emerald-600 hover:bg-em
                    -1 shadow-lg shadow-emerald-900/40 transiti
                >
                    <Play className="w-4 h-4" /> Запустить

```

```

    });
    const svgWidth = Math.max(maxX + 80, 400);
    const svgHeight = Math.max(maxY + 60, 200);

    return (
      <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
        <defs>
          <marker id="arrowhead" markerWidth="6"
            <polygon points="0 0, 6 3, 0 6" fill=
          </marker>
          <marker id="fail-arrow" markerWidth="6"
            <polygon points="0 0, 6 3, 0 6" fill=
          </marker>
        </defs>

        /* Draw Edges */
        {Object.values(nodes).map(node => {
          if (node.id === 0) return null;
          const parent = nodes[node.parent];
          if (!parent.x || !parent.y || !node.x || !node.y) return null;

          const isCurrent = currentNode === node.id;

          return (
            <g key={`edge-${node.id}`}>
              <line
                x1={parent.x}
                y1={parent.y + 16}
                x2={node.x}
                y2={node.y - 16}
                stroke={isCurrent ? '#60a5fa' : '#94a3b8'}
                strokeWidth="2"
                markerEnd="url(#arrowhead)"
              />
              <text
                x={{(parent.x + node.x) / 2 - 10}}
                y={{(parent.y + node.y) / 2}}
                fill="#94a3b8"

```

```

    if (!node.x || !node.y) return null;
    const isRoot = node.id === 0;
    const isCurrent = currentNode === node;
    const isInQueue = bfsQueue.includes(node);

    let fill = '#1e293b';
    let stroke = '#475569';

    if (isCurrent) {
        fill = '#2563eb';
        stroke = '#60a5fa';
    } else if (isInQueue) {
        fill = '#b45309';
        stroke = '#fbbf24';
    } else if (node.is_terminal) {
        fill = '#059669';
        stroke = '#34d399';
    }

    return (
      <g key={`node-${node.id}`}>
        <circle
          cx={node.x}
          cy={node.y}
          r="16"
          fill={fill}
          stroke={stroke}
          strokeWidth="2"
        />
        <text
          x={node.x}
          y={node.y + 4}
          fill="white"
          fontSize={isRoot ? "10" : "12"}
          fontWeight="bold"
          textAnchor="middle"
        >
          {isRoot ? 'R' : node.char}
        </text>
      </g>
    );
  }
}

```

```

        >
        x
      </button>
    </span>
  }}}
</div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
  <input
    type="text"
    value={newWord}
    onChange={(e) => setNewWord(e.target.value)}
    placeholder="НОВОЕ СЛОВО..."
    maxLength={8}
    className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
  />
  <button
    type="submit"
    className="bg-slate-700 hover:bg-slate-600 transition-colors"
  >
    <Plus className="w-3.5 h-3.5" /> Добавить
  </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
  <h5 className="text-white font-bold mb-3 flex" >
    <span className="flex items-center gap-1.5" >
      <span> </span> Таблица переходов и fail-
    </span>
    <span className="text-xs text-slate-400 font-mono" >
      Всего вершин: {Object.keys(nodes).length}
    </span>
  </h5>

```

```

                ? 'bg-amber-400'
                : 'bg-slate-600'
            }` }
        ></span>
        <span>#{node.id}</span>
        {node.id === 0 && <span className=
    </td>
    <td className="py-2.5 px-3">
        <span className="bg-slate-800 b
            {node.char}
        </span>
    </td>
    <td className="py-2.5 px-3 text-s
        {node.id === 0 ? '-' : `#${node
    </td>
    <td className="py-2.5 px-3">
        {node.id === 0 ? {
            <span className="text-slate-5
        } : {
            <span className="bg-indigo-95
                #{node.fail}
            </span>
        }}
    </td>
    <td className="py-2.5 px-3">
        {node.id === 0 || node.term_lin
            <span className="text-slate-5
        ) : {
            <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Cк
            #{node.term_link}
        </span>
        }}
    </td>
    <td className="py-2.5 px-3 text-s
        {Object.keys(node.children).len
        ? '0'
        : Object.entries(node.children

```

```

codeLine: number;
treeState: Record<number, number>;           // node index
highlightNodes: number[];                    // current
mergeChildren?: [number, number];           // pair be
arrayHighlight?: [number, number];           // [L, R]
result?: number;                             // partial
}

```

// ===== PURE LOGIC: build tree =====

```

function buildTreeFull(arr: number[]): Record<number, number> {
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  }
  go(1, 0, n - 1);
  return tree;
}

```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

function genBuildSteps(arr: number[]): Step[] {
  const n = arr.length;
  const steps: Step[] = [];
  let id = 0;
  const tree: Record<number, number> = {};

```

```

  const CODE = [
    "build(v, l, r) {",           // 1
    "  if (l === r) {",           // 2
    "    tree[v] = A[l]; return", // 3
    "  }",                         // 4
    "  mid = (l + r) >> 1",       // 5

```

```

let id = 0;

function go(v: number, l: number, r: number, ql: number, qr: number) {
  if (ql > r || qr < l) {
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
      codeLine: 2, treeState: tree, highlightNodes: [v], result: 0});
    return 0;
  }
  if (ql <= l && r <= qr) {
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
      codeLine: 4, treeState: tree, highlightNodes: [v], result: tree[v]});
    return tree[v];
  }
  const m = (l + r) >> 1;
  steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
    codeLine: 6, treeState: tree, highlightNodes: [v], result: 0});
  const left = go(2 * v, l, m, ql, qr);
  const right = go(2 * v + 1, m + 1, r, ql, qr);
  const res = left + right;
  steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`,
    codeLine: 8, treeState: tree, highlightNodes: [v], result: res,
    rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [2 * v, 2 * v + 1]});
  return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]: ${ans}`,
  codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans});
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
  const n = arr.length;
  // Build iterative tree: leaves at [n..2n-1]
  const tree: Record<number, number> = {};
  for (let i = 0; i < n; i++) tree[n + i] = arr[i];
  for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

  const steps: Step[] = [];
  steps.push({ id: 1, desc: `Build tree from array`, codeLine: 1, treeState: tree, highlightNodes: [], result: 0});
  for (let i = 0; i < n; i++) {
    steps.push({ id: i + 2, desc: `tree[${n + i}] = arr[${i}]`, codeLine: 2 + i, treeState: tree, highlightNodes: [n + i], result: arr[i]});
  }
  for (let i = n - 1; i > 0; --i) {
    steps.push({ id: i + 2 + n, desc: `tree[${i}] = (tree[${2 * i}] + tree[${2 * i + 1}])`, codeLine: 3 + i, treeState: tree, highlightNodes: [i], result: tree[i]});
  }
  return steps;
}

```

```
function buildVisualTree(treeState: Record<number, number>) {
  if (l === r) return { v, l, r, val: treeState[v] ?? 0 };
  const m = (l + r) >> 1;
  return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
    treeState, v + 1, m + 1, r) };
}
```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
  "build(v, l, r) {",
  "  if (l === r) { tree[v] = A[l]; return; }",
  "  // -----",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  build(2*v, l, mid);",
  "  build(2*v+1, mid+1, r);",
  "  tree[v] = tree[2*v] + tree[2*v+1];",
  "}"
];
```

```
const QUERY_TD_CODE = [
  "query(v, l, r, ql, qr) {",
  "  if (ql > r || qr < l) return 0;",
  "  // -----",
  "  if (ql <= l && r <= qr) return tree[v];",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  // спускаемся в обоих детей",
  "  return query(left) + query(right);",
  "}"
];
```

```
const QUERY_BU_CODE = [
  "queryBU(ql, qr) {",
  "  res = 0;",
  "  l = ql + n; r = qr + n + 1;",
  "  // -----",
  "  while (l < r) {",
  "    if (l & 1) res += tree[l++];",
  "    l = l >> 1; r = r >> 1;
  "  }
];
```



```

if (!p) return null;
const { step, idx, setIdx, playing, setPlaying, speed
const n = arr.length;

const vTree = useMemo(() => buildVisualTree(step.tree

const renderNode = useCallback((nd: VNode): React.Rea
  const isHigh = step.highlightNodes.includes(nd.v);
  const isChild = step.mergeChildren?.includes(nd.v);
  const has = nd.val !== null;

  let cls = 'bg-slate-800 border-slate-700 text-slate
  if (isHigh) cls = `${clr.ring} text-white ring-2 sc
  else if (isChild) cls = `${clr.childRing} text-ambe
  else if (has) cls = `${clr.filled} text-slate-200`;

  return (
    <div key={nd.v} className="flex flex-col items-ce
      <div className={`flex flex-col items-center jus
        ${cls}`}>
        <span className="text-[7px] opacity-60 font-m
        <span className="text-[9px] font-bold">[{nd.l
        <span className="text-xs font-extrabold font-i
      </div>
      {(nd.left || nd.right) && (
        <div className="flex flex-col items-center mt
          <div className="w-px h-2 bg-slate-700" />
          <div className="flex justify-around w-full
            {nd.left && renderNode(nd.left)}
            {nd.right && renderNode(nd.right)}
          </div>
        </div>
      </div>
    )}
  </div>
  );
}, [step, clr]);

return (

```

```

      : 'bg-amber-600/20 border-l-2 border-amber-600'
      <span className="inline-block w-4 text-slate-400">
    </div>
  );
  }}}
</pre>

```

```

{/* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 border-t-1 border-slate-800" >
  <span className={`mt-0.5 inline-block w-2 h-2 rounded-full`} style={{
    background-color: {step.result !== undefined && {
      : color === 'emerald' ? 'text-emerald-300' : 'text-slate-400'
    }}
  }}>
  <span className="leading-relaxed">{step.desc}</span>
  {step.result !== undefined && {
    <span className={`ml-auto shrink-0 font-mono text-sm`} style={{
      : color === 'emerald' ? 'text-emerald-300' : 'text-slate-400'
    }}>
    = {step.result}
  </span>
  }}
</div>

```

```

{/* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t-1 border-slate-800" >
  <div className="flex items-center bg-slate-900/60 border-t-1 border-slate-800" >
    <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
      <RotateCcw className="w-3 h-3 text-slate-400" />
    </button>
    <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
      <RotateCcw className="w-3 h-3 text-slate-400" />
    </button>
    <button onClick={() => setPlaying(!playing)}>
      <Play className="w-3 h-3 text-white" />
    </button>
    <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
      <RotateCcw className="w-3 h-3 text-slate-400" />
    </button>
    <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
      <RotateCcw className="w-3 h-3 text-slate-400" />
    </button>
  </div>
  <select value={speed} onChange={e => setSpeed(Number(e.target.value))}>
    <option value={1200}>Медл.</option>
  </select>

```

```

const randomize = () => {
  const size = arr.length;
  const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
  setArr(a);
  setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr), [arr]);
const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr), [arr]);
const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr), [arr]);

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
    {/* ---- TOP BAR: array editor + query range ---- */}
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4">
        {/* Array input */}
        <form onSubmit={handleApply} className="flex flex-col">
          <div className="flex items-center justify-between">
            <label className="text-[10px] font-bold uppercase">Array</label>
            <button type="button" onClick={randomize}>Refresh</button>
          </div>
          <div className="flex gap-2">
            <input
              type="text"
              value={customInput}
              onChange={e => setCustomInput(e.target.value)}
              className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
              placeholder="5, 8, 3, 12, 7, 2"
            />
            <button type="submit" className="bg-indigo-600 text-white px-4 py-2">Применить</button>
          </div>
        </form>
      </div>
    </div>
  </div>
);

```

```

        </div>
      </div>
      <div className="text-[10px] text-slate-500" style={style}>
        Запрос:  $\text{sum}(A[\{qL\}..\{qR\}]) = \{\text{arr.slice}(qL, qR)\}$ 
      </div>
    </div>
  </div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1" style={style}>
  <button onClick={() => setView('build')} className="border border-indigo-500 text-slate-200" style={style}>
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} className="border border-emerald-500 text-slate-200" style={style}>
    <Search className="w-3.5 h-3.5" /> Запрос: св
  </button>
  <button onClick={() => setView('theory')} className="border border-blue-500 text-slate-200" style={style}>
    <Info className="w-3.5 h-3.5" /> Почему / Спра
  </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon=""
    steps={buildSteps}
    code={BUILD_CODE}
    color="indigo"
    arr={arr}
  >

```

```

<div className="bg-slate-900 p-4 rounded-lg" style={{width: 100%; height: 100%;}}
  <div className="text-slate-400">Корень: L
  <div className="text-slate-300">mid = l(0
  <div className="text-indigo-300">→ Левый:
  <div className="text-emerald-300">→ Правый:
  h - l) >> l)} эл.)</div>
  <div className="text-slate-500 mt-2">Всего
  } внутренних.</div>
</div>
</div>

```

```

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols-2" style={{width: 100%; height: 100%;}}
  <div className="bg-slate-950 p-5 rounded-xl" style={{width: 100%; height: 100%;}}
    <div className="absolute -top-3 left-4 bg-white border border-emerald-500 shadow-md" style={{width: 100%; height: 100%;}}
      Запрос Сверху-вниз (Рекурсия)
    </div>
    <p className="text-xs text-slate-300 mb-3">
      Начинаем с корня. Если отрезок узла полн
      наче спускаемся в обоих детей.
    </p>
    <div className="space-y-1.5 text-xs">
      <div className="flex justify-between border-b border-slate-700" style={{width: 100%; height: 100%;}}
        <span className="text-rose-400 font-mono">0{
          <div className="flex justify-between border-b border-slate-700" style={{width: 100%; height: 100%;}}
            <span className="text-rose-400 font-mono">0{
              <div className="flex justify-between" style={{width: 100%; height: 100%;}}
                <span className="text-emerald-400 font-bold">✓ легко</span></div>
            </div>
          </div>
        </div>
      <div className="bg-slate-950 p-5 rounded-xl" style={{width: 100%; height: 100%;}}
        <div className="absolute -top-3 left-4 bg-white border border-amber-500 shadow-md" style={{width: 100%; height: 100%;}}
          Запрос Снизу-вверх (Итерация)
        </div>
        <p className="text-xs text-slate-300 mb-3">
          Стартуем с двух указателей (l и r) на л
        </p>
      </div>
    </div>
  </div>

```

```
interface SidebarProps {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Мобильный режим: список живёт в выдвижной панели */
  onNavigate?: () => void;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId }) => {
  const [query, setQuery] = useState("");

  const groups = useMemo(() => {
    const q = query.trim().toLowerCase();
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
    const map = new Map<string, typeof chapters>();
    for (const c of filtered) {
      const key = c.category?.trim() || "Прочие темы";
      if (!map.has(key)) map.set(key, []);
      (map.get(key) as typeof chapters).push(c);
    }
    return Array.from(map.entries());
  }, [query]);

  return (
    <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:p-4">
      <div className="p-4 pb-3 shrink-0">
        <h3 className="text-xs font-bold text-slate-400">
          <Compass className="w-4 h-4 text-indigo-400" />
        </h3>
        <div className="relative">
          <Search className="w-4 h-4 text-slate-500 absolute top-0 left-0">
            <input
              type="text"
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="Поиск по темам..."
              className="w-full bg-slate-800/70 border border-slate-700 focus:outline-none focus:border-indigo-500"
            />
          </div>
        </div>
      </div>
    </aside>
  );
}
```

```

        className="w-1.5 h-1.5 rounded-0"
        title="Есть интерактивная визуализация"
      />
    ))
    <span className="font-semibold text-indigo-400">
      <ChevronRight
        className={`w-4 h-4 shrink-0 mt-0.5
          isSelected ? "text-indigo-400" : "text-slate-400"
        }` />
    </button>
  );
  )})
</div>
</div>
)})
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-400 to-indigo-600
  ] text-slate-300 space-y-1.5 hidden lg:block">
  <div className="font-bold text-indigo-400 flex items-center">
    <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
  </div>
  <p className="leading-relaxed">
    Подчёркнутые термины в тексте раскрываются по клику в симуляторе.
  </p>
  <p className="text-slate-400 italic">Стрелки ← и → переключают вид
  </div>
</aside>
);
};

```

```

    });
    const [dequeueMessage, setDequeueMessage] = useState<

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный"',
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный"',
  { id: '3', text: 'Кандидат: "Привет, я испанский ин
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: '🥞' },
    { name: 'Кастрюля от супа', icon: '🍲' },
    { name: 'Чашка от кофе', icon: '☕' },
    { name: 'Миска с остатками салата', icon: '🥗' },
    { name: 'Тарелка от тако', icon: '🌮' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(`Положили поверх стопки: ${newPlate.name} ${newPlate.icon}`);
};

const popPlate = () => {
  if (stack.length === 0) {
    setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
    return;
  }
  const topPlate = stack[stack.length - 1];

```



```

    setDequeueMessage(`    Выдан суп первому в очереди (F
  });

const resetQueue = () => {
  setQueue([
    { id: '1', name: 'Студент Вася', icon: '    ' },
    { id: '2', name: 'Голодный кодер', icon: '    ' },
    { id: '3', name: 'Кот Борис', icon: '    ' },
  ]);
  setDequeueMessage("    Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {
    id: Date.now().toString(),
    text: `Кандидат: "${newCustomText}"`,
    probability: Number(newCustomProb)
  };
  setHeap(prev => [...prev, item]);
  setNewCustomText('');
  setHeapMessage(`    Добавлена гипотеза с вероятностью
};

const popHeap = () => {
  if (heap.length === 0) {
    setHeapMessage("⚠    Куча пуста! Нет кандидатов для
    return;
  }
  // Find highest probability
  let maxIdx = 0;
  for (let i = 1; i < heap.length; i++) {
    if (heap[i].probability > heap[maxIdx].probability)
      maxIdx = i;
  }
}

```

```

>
  <div className="flex items-center gap-3">
    <Layers className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-500' : 'bg-slate-800/60 border-slate-700/60 text-slate-500'} `}>
      <div className="text-left">
        <div className="font-bold text-sm text-white">
          <div className="text-xs opacity-80">LIFO
        </div>
      </div>
      <span className="text-xs bg-blue-500/20 text-white">
        DFS
      </span>
    </div>
  </button>

  <button
    onClick={() => setActiveTab('queue')}
    className={`flex items-center justify-between gap-3 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-500' : 'bg-slate-800/60 border-slate-700/60 text-slate-500'} `}
  >
    <div className="flex items-center gap-3">
      <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-500' : 'bg-slate-800/60 border-slate-700/60 text-slate-500'} `}>
        <div className="text-left">
          <div className="font-bold text-sm text-white">
            <div className="text-xs opacity-80">FIFO
          </div>
        </div>
      </div>
      <span className="text-xs bg-indigo-500/20 text-white">
        BFS
      </span>
    </div>
  </button>

  <button
    onClick={() => setActiveTab('heap')}
    className={`flex items-center justify-between gap-3 ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500 text-emerald-500' : 'bg-slate-800/60 border-slate-700/60 text-slate-500'} `}
  >
    <div className="flex items-center gap-3">
      <AlignLeft className={`w-5 h-5 ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500 text-emerald-500' : 'bg-slate-800/60 border-slate-700/60 text-slate-500'} `}>
        <div className="text-left">
          <div className="font-bold text-sm text-white">
            <div className="text-xs opacity-80">Heap
          </div>
        </div>
      </div>
      <span className="text-xs bg-emerald-500/20 text-white">
        BFS
      </span>
    </div>
  </button>

```

```

        <button
            onClick={popPlate}
            className="flex-1 md:flex-none flex item
border-blue-500/40 px-4 py-2.5 rounded-xl
        >
            Помыть верхнюю
        </button>
        <button
            onClick={resetStack}
            title="Сбросить"
            className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
        >
            <RotateCcw className="w-5 h-5" />
        </button>
    </div>
</div>

{popMessage && {
    <div className="bg-blue-950/60 border borde
imate-fade-in">
        <span className="inline-block w-2 h-2 roun
        {popMessage}
    </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
    >
        {stack.length === 0 ? {
            <div className="text-center py-12 text-sl
                <div className="text-5xl mb-3"> </div>
                <p className="text-base font-bold">Стор
                <p className="text-xs mt-1">Добавьте гр
            </div>
        } : {
            <div className="w-full max-w-sm flex flex
                <div className="absolute top-0 text-xs

```

```

    </div>
  })

  { /* QUEUE TAB */
  {activeTab === 'queue' && {
    <div className="space-y-6">
      <div className="bg-slate-800/80 p-5 rounded-x
        tify-between gap-4">
        <div>
          <h3 className="text-lg font-bold text-whi
            <span> Симуляция очереди за супом</span>
            <span className="text-xs font-mono bg-i
          /span>
          </h3>
          <p className="text-slate-400 text-xs mt-1
            Кто первым пришел, тот первым получил с
          </p>
        </div>
        <div className="flex flex-wrap items-center
          <button
            onClick={addPerson}
            className="flex-1 md:flex-none flex ite
          -2.5 rounded-xl text-sm font-bold shadow-lg
          >
            <Plus className="w-4 h-4" /> Встать в о
          </button>
          <button
            onClick={servePerson}
            className="flex-1 md:flex-none flex ite
          er border-indigo-500/40 px-4 py-2.5 rounded
          >
            Налить суп первому
          </button>
          <button
            onClick={resetQueue}
            title="Сбросить"
            className="p-2.5 bg-slate-800 hover:bg-
          tion-colors cursor-pointer"

```

```

        <div
          key={person.id}
          className={`flex flex-col items-center
            ${isFirst
              ? 'bg-gradient-to-b from-indigo-500 to-slate-900'
              : 'bg-slate-900/90 border-slate-500'}
          `}
        >
          {isFirst && (
            <span className="absolute -top-10 left-1/2 transform translate-x-50%
              text-slate-50 tracking-wider shadow">
                Первый (Head)
              </span>
            )}
          <span className="text-4xl mb-2">{
            <span className={`font-bold text-slate-50
              ${person.name.startsWith('П') ? 'text-slate-50' : 'text-slate-400'}
            `}>
              {person.name}
            </span>
            <span className={`text-[10px] font-mono
              ${isFirst ? 'bg-indigo-500/30 text-white' : 'text-slate-400'}
            `}>
              {isFirst ? 'Получает сун' : `В ${person.name}
            </span>
          </div>
        </div>
      );
    }
  )}

```

```

    <div className="flex flex-col items-center justify-center gap-6
      min-w-[120px] h-[120px]">
      <Plus className="w-6 h-6 mb-1" />
      <span className="text-xs font-mono uppercase">
        ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • ИЛИ
      </span>
    </div>
  </div>
)}
<div className="mt-6 text-xs text-slate-500">
  ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • ИЛИ
</div>

```

```

    { /* Add Form */ }
    <form onSubmit={addHypothesis} className="bg-
      s-12 gap-4 items-end">
      <div className="md:col-span-6">
        <label className="block text-xs font-bold">
          <input
            type="text"
            value={newCustomText}
            onChange={e => setNewCustomText(e.target.value)}
            placeholder="например: "Привет, я лучший!"
            className="w-full bg-slate-900 border border-
              rder-emerald-500 transition-colors"
          />
        </div>
        <div className="md:col-span-3">
          <label className="block text-xs font-bold">
            Вероятность: <span className="text-emerald-500">
          </label>
          <input
            type="range"
            min="0.01"
            max="0.99"
            step="0.01"
            value={newCustomProb}
            onChange={e => setNewCustomProb(Number(e.target.value))}
            className="w-full accent-emerald-500 cursor-pointer"
          />
        </div>
        <div className="md:col-span-3">
          <button
            type="submit"
            disabled={!newCustomText.trim()}
            className="w-full flex items-center justify-center gap-2
              erald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
          >
            <Plus className="w-4 h-4" /> Добавить в список
          </button>
        </div>
      </form>
    )}
  </div>
)

```

```

        }` }
    >
    <div className="flex items-center"
      <span className={`flex items-center
        isTop ? 'bg-emerald-500 text-
      }`}>
        #{index + 1}
      </span>
      <div>
        <div className={`font-bold text-
          {item.text}
        </div>
        {isTop && (
          <span className="text-[10px]
-0.5 rounded inline-block mt-1">
            Вершина Кучи (Root) • Буд
          </span>
        )}
      </div>
    </div>

    <div className="flex items-center"
      {/* Custom progress bar */}
      <div className="hidden sm:block"
        <div
          className={`h-full rounded-
            style={{ width: `${percent}%
          ></div>
        </div>
        <span className={`font-mono font-
          isTop ? 'text-emerald-400 text-
        }`}>
          {percent}%
        </span>
      </div>
    </div>
  </div>
);
}}}
```

```

const items = useMemo(() => {
  const titleById = new Map(chapters.map((c) => [c.id, c.title]));
  const all = Object.entries(VIZ_REGISTRY).map(([id, entry], index) => ({
    id,
    entry,
    chapterTitle: titleById.get(id),
  }));
  const q = query.trim().toLowerCase();
  if (!q) return all;
  return all.filter(
    (i) =>
      i.entry.title.toLowerCase().includes(q) ||
      (i.entry.hint ?? "").toLowerCase().includes(q) ||
      (i.chapterTitle ?? "").toLowerCase().includes(q)
  );
}, [query]);

return (
  <div>
    <div className="mb-6">
      <h2 className="text-xl sm:text-2xl font-extrabold">Визуализация</h2>
      <p className="text-sm text-slate-400 leading-relaxed">
        Все интерактивные демонстрации в одном месте.
        здесь их можно открыть напрямую.
      </p>
    </div>

    <div className="relative mb-6 max-w-md">
      <Search className="w-4 h-4 text-slate-500 absolute top-0 right-0">
        <input
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Поиск тренажёра..."
          className="w-full bg-slate-900 border border-indigo-500
            focus:outline-none focus:border-indigo-500"
        />
      </div>
      {query && (

```



```
        <span className="truncate max-w-[9rem]
        </button>
      )}
    </div>
  </div>
  )})
</div>
</div>
);
};
```

ФАЙЛ 65/87: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useCallback, useMemo, useState } from "
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, U
```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерев
```

```
 * поднять отмеченный узел в корень. Клик по узлу = оди
```

```
 * Порядок поворотов выбираешь сам; правильный ли он по
```

```
 * только после того, как узел окажется наверху (или по
```

```
 */
```

```
export interface TNode {
```

```
  key: number;
```

```
  left: TNode | null;
```

```
  right: TNode | null;
```

```
}
```

```
const leaf = (key: number): TNode => ({ key, left: null
```

```
const clone = (t: TNode | null): TNode | null =>
```

```

    const n6: TNode = { key: 6, left: n5, right: leaf(7) };
    const n4: TNode = { key: 4, left: leaf(3), right: leaf(5) };
    return { key: 8, left: n4, right: leaf(9) };
  },
},
];

```

/* ----- операции над деревом ----- */

```

export const findPath = (root: TNode | null, key: number): TNode[] => {
  const path: TNode[] = [];
  let cur = root;
  while (cur) {
    path.push(cur);
    if (key === cur.key) return path;
    cur = key < cur.key ? cur.left : cur.right;
  }
  return [];
};

```

```

/** Поднять узел key на один уровень (поворот с родителем) */
export function rotateUp(root: TNode, key: number): TNode {
  const path = findPath(root, key);
  if (path.length < 2) return root;

  const x = path[path.length - 1];
  const p = path[path.length - 2];
  const gg = path.length >= 3 ? path[path.length - 3] : null;

  if (p.left === x) {
    p.left = x.right;
    x.right = p;
  } else {
    p.right = x.left;
    x.left = p;
  }

  if (!gg) return x;

```

```

        nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
        if (n.left) edges.push([n.key, n.left.key]);
        if (n.right) edges.push([n.key, n.right.key]);
        walk(n.right, depth + 1);
    };
    walk(root, 0);

    const cols = Math.max(1, order);
    const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
    const colW = 46;
    const rowH = 62;
    return {
        nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
        edges,
        width: cols * colW,
        height: rows * rowH + 20,
    };
}

/* ----- КОМПОНЕНТ ----- */

export const SplayRotationSandbox: React.FC = () => {
    const [presetIdx, setPresetIdx] = useState(0);
    const preset = PRESETS[presetIdx];

    const [tree, setTree] = useState<TNode>(() => preset.build());
    const [history, setHistory] = useState<TNode[]>([]);
    const [moves, setMoves] = useState<{ node: number; count: number }>({ node: 0, count: 0 });
    const [showAnswer, setShowAnswer] = useState(false);

    const target = preset.target;
    const solved = tree.key === target;

    const reset = useCallback(
        (idx = presetIdx) => {
            setPresetIdx(idx);
            setTree(PRESETS[idx].build());
            setHistory([]);
        },
        [presetIdx]
    );

```

```

        <span className="font-mono font-bold text-indigo-600">
        поворотов выбираешь сам, разбор покажем в к
    </p>
</div>
<button
  onClick={() => reset((presetIdx + 1) % PRESETS.length)}
  className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-bold transition-colors shrink-0"
>
  <Dice className="w-3.5 h-3.5" /> Другое дерево
</button>
</div>

<div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar" >
  {PRESETS.map((p, i) => {
    <button
      key={p.name}
      onClick={() => reset(i)}
      className={`px-3 py-1.5 rounded-lg text-sm font-medium transition-colors
        ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-indigo-600"}
      `}
    >
      {p.name}
    </button>
  })}
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4" >
  <svg
    viewBox={`0 0 ${width} ${height}`}
    className="h-auto block mx-auto"
    style={{ minWidth: Math.min(width * 1.5, 420) }}
    role="img"
  >
    {edges.map(([a, b], i) => {
      const na = nodes.find((n) => n.key === a);
      const nb = nodes.find((n) => n.key === b);
      if (!na || !nb) return null;
    })}
  </svg>

```

```

        {n.key}
      </text>
    </g>
  );
  }}}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2"
  <button
    onClick={undo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1
      -white disabled:opacity-40 text-xs font-bol
  >
    <Undo2 className="w-3.5 h-3.5" /> Отменить
  </button>
  <button
    onClick={() => reset()}
    className="flex items-center gap-1.5 px-3 py-1
      ext-xs font-bold transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Заново
  </button>
  <button
    onClick={() => setShowAnswer((s) => !s)}
    className={`flex items-center gap-1.5 px-3 py-1
      showAnswer
        ? "bg-amber-600/20 border-amber-500 text-
        : "bg-slate-800 border-slate-700 text-sla
      }`}
  >
    {showAnswer ? <EyeOff className="w-3.5 h-3.5"
    {showAnswer ? "Скрыть разбор" : "Сдаться / по
  </button>

  <span className="text-[11px] text-slate-500 ml-
    поворотов: {moves.length} • минимум: {minimal

```

```
        <div className="opacity-80">Порядок верный,
      })
    </div>
  })
</div>
);
};
```

ФАЙЛ 66/87: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";
```

```
/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
 *
 * Что сделано ради понятности:
 * • у узлов настоящие числа (20 / 40 / 60), а не абстракции;
 * сразу видно, что дерево остаётся деревом поиска;
 * • под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
 * это и есть доказательство, что поворот ничего не меняет;
 * • кадр «прицел» ничего не двигает, только подсвечивает;
 * • на кадре «результат» остаются призраки – пунктирные линии в
 * местах и стрелки «откуда → куда»;
 * • по умолчанию анимация НЕ крутится сама: пользователь
 * идёт в своём темпе. Автопрокрутка включается кнопкой.
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
```

```

when: "родитель х — уже корень, деда нет",
rotations: "1 поворот",
keys: { x: 20, p: 40 },
inorder: ["A", "20", "B", "40", "C"],
ranges: "A < 20 · B между 20 и 40 · C > 40",
frames: [
    {
        ...ZIG_BEFORE,
        kind: "start",
        title: "Было: 20 висит под корнем 40",
        text: "Нам нужен ключ 20, а он на глубине 1. Деду",
        ms: RESULT_MS,
    },
    {
        ...ZIG_BEFORE,
        kind: "aim",
        title: "Прицел: крутим ребро 40 — 20",
        text: "Ничего пока не двигается. Смотри на подсве",
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [160, 50], A: [80, 130], p: [245, 130],
        edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
        kind: "result",
        title: "Стало: 20 — корень",
        text: "40 стало правым сыном двадцатки. Поддерев
            ева от 40 — это одно и то же место.",
        moved: ["x", "p", "B"],
        ms: RESULT_MS,
    },
],
};

```

```

/* ----- Zig-Zig -----
const ZZ_S0 = {
    pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
    edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]

```

```

        moved: ["p", "g", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 – 20",
        text: "А теперь обычный одиночный поворот – тот же",
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [100, 45], A: [45, 118], p: [188, 118],
        edges: [ ["x", "A"], ["x", "p"], ["p", "B"], ["p",
        kind: "result",
        title: "Стало: 20 в корне, бамбук стал кустом",
        text: "Сравни с первым кадром: А и В были на глуб
            дерево, а не только достаёт ключ.",
        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    },
    ],
};

```

```

/* ----- Zig-Zag -----
const ZG_S0 = {
    pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
    edges: [ ["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
    pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
    edges: [ ["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZAG: Case = {
    key: "zig-zag",
    label: "Zig-Zag",

```



```

    {
      pos: { x: [180, 50], p: [90, 130], g: [275, 130],
      edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
      kind: "result",
      title: "Стало: 20 и 60 – два сына сорока",
      text: "Дерево стало идеально симметричным: все че
        очку». ",
      moved: ["x", "p", "g", "C"],
      ms: RESULT_MS,
    },
  ],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: str
  x: { fill: "#6366f1", stroke: "#a5b4fc" },
  p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
  g: { fill: "#f59e0b", stroke: "#fcd34d" },
  sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x – ищем его",
  p: "p – родитель",
  g: "g – дед",
};

```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])

  return (
    <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
    <defs>

```

```

    return (
      <line
        key={` ${a} - ${b} - ${i} `}
        x1={pa[0]}
        y1={pa[1]}
        x2={pb[0]}
        y2={pb[1]}
        stroke={hot ? "#fbbf24" : "#475569"}
        strokeWidth={hot ? 5 : 2}
        opacity={focus && !hot ? 0.25 : 1}
        style={{ transition: "all 1200ms cubic-bezier(0.16, 0.24, 0.09, 0.23)" }}
      />
    );
  }
}

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
  const p = pos[id];
  if (!p) return null;
  const sub = isSubtree(id);
  const c = COLOR[sub ? "sub" : id];
  const r = sub ? 15 : 20;
  const hot = Boolean(focus && focus.includes(id));
  return (
    <g
      key={id}
      style={{
        transform: `translate(${p[0]}px, ${p[1]}px)`,
        transition: MOVE,
        opacity: dim(id) ? 0.28 : 1,
      }}
    >
      {hot && <circle r={r + 7} fill="none" stroke="black" />}
      <circle r={r} fill={c.fill} stroke={c.stroke} />
      <text
        textAnchor="middle"
        dominantBaseline="central"
        fontSize={sub ? 12 : 15}
        fontWeight="700"
      />
    </g>
  );
}

```

```
}, [playing, frame, duration, total]));
```

```
return (
```

```
  <div className="w-full bg-slate-950 rounded-2xl border
```

```
    <div className="flex gap-2 mb-4 overflow-x-auto no
```

```
      {CASES.map((c, i) => {
```

```
        <button
```

```
          key={c.key}
```

```
          onClick={() => setCaseIdx(i)}
```

```
          className={`px-3 sm:px-4 py-2 rounded-lg text
```

```
            i === caseIdx ? "bg-indigo-600 text-white
```

```
          }`}
```

```
        >
```

```
          {c.label}
```

```
        </button>
```

```
      )})
```

```
    </div>
```

```
    <div className="mb-3 text-xs sm:text-[13px] text-
```

```
      <span className="font-bold text-indigo-300">Кор,
```

```
      <span className="text-slate-500">({active.rotat
```

```
    </div>
```

```
    {/* шаг + пояснение стоят НАД картинкой: сначала
```

```
    <div className="mb-3 rounded-xl border border-sla
```

```
      <div className="flex items-center gap-2 mb-1 fl
```

```
        <span
```

```
          className={`text-[10px] font-bold uppercase
```

```
            current.kind === "aim"
```

```
              ? "bg-amber-500/20 text-amber-300"
```

```
              : current.kind === "start"
```

```
                ? "bg-slate-700 text-slate-300"
```

```
                : "bg-indigo-500/20 text-indigo-300"
```

```
          }`}
```

```
        >
```

```
          {current.kind === "aim" ? "прицел" : curren
```

```
        </span>
```

```
      <span className="text-sm font-bold text-white
```

```

    }
  />
</div>

<div className="flex flex-wrap items-center gap-2"
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f - 1 + total) % total);
    }}
    disabled={idx === 0}
    className="px-3 py-2 rounded-lg bg-slate-800 text-white
      d: hover:text-slate-300 text-xs font-bold transition-colors"
  >
    ← Назад
  </button>
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f + 1) % total);
    }}
    className="px-4 py-2 rounded-lg bg-indigo-600 text-white
      w-indigo-900/40"
  >
    {last ? "⏮ Сначала" : "Дальше →"}
  </button>
  <button
    onClick={() => setPlaying((p) => !p)}
    className="flex items-center gap-1.5 px-3 py-2 text-white
      text-xs font-bold transition-colors"
  >
    {playing ? <Pause className="w-3.5 h-3.5" /> : <Play />}
    {playing ? "Стоп" : "Авто"}
  </button>
  <button
    onClick={() => setSlow((s) => !s)}
    title="Автопрокрутка ещё медленнее"
    className={`flex items-center gap-1.5 px-3 py-2 text-white
      text-xs font-bold transition-colors`}
  >
    {slow ? <Slow /> : <Fast />}
    {slow ? "Медленно" : "Быстро"}
  </button>
</div>

```

```
import React, { useState } from "react";
import {
  RotateCcw,
  HelpCircle,
  BookOpen,
} from "lucide-react";

interface SplayNode {
  key: number;
  left: SplayNode | null;
  right: SplayNode | null;
  x?: number;
  y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
  if (!root) return { key, left: null, right: null };
  if (key < root.key) {
    root.left = insertBST(root.left, key);
  } else if (key > root.key) {
    root.right = insertBST(root.right, key);
  }
  return root;
};

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
  const y = x.left;
  if (!y) return x;
  x.left = y.right;
  y.right = x;
  return y;
};
```

```

    });
    const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

    // Splay operation that tracks steps!
    const splayStepByStep = (
      root: SplayNode | null,
      key: number,
    ): { newRoot: SplayNode | null; steps: string[] } => {
      const steps: string[] = [];
      if (!root) return { newRoot: null, steps: ["Дерево пусто"] };

      // We implement the classic top-down or bottom-up splay operation
      // For visualization logs, we do a bottom-up explanation
      let path: { node: SplayNode; dir: "left" | "right" | null }[] = [];
      let current: SplayNode | null = root;

      // Find the element or the element where search failed
      let lastNode: SplayNode = root;
      while (current) {
        lastNode = current;
        if (key === current.key) {
          path.push({ node: current, dir: null });
          break;
        } else if (key < current.key) {
          path.push({ node: current, dir: "left" });
          current = current.left;
        } else {
          path.push({ node: current, dir: "right" });
          current = current.right;
        }
      }

      const targetKey = current ? key : lastNode.key;
      const isFound = !!current;

      if (!isFound) {
        steps.push(`Поиск ${key}: элемент отсутствует в дереве`);
        steps.push(`Возвращаем последний просмотренный элемент: ${lastNode.key}`);
      }
    };
  }
}

```

```

        return rightRotate(node);
    } else {
        if (!node.right) return node;

        if (k < node.right.key) {
            // Zag-Zig (Right-Left)
            node.right.left = splayAction(node.right.left, target);
            if (node.right.left) {
                node.right = rightRotate(node.right);
                steps.push(`Компонент Zig-Zag: правый поворот`);
            }
        } else if (k > node.right.key) {
            // Zag-Zag (Right-Right)
            node.right.right = splayAction(node.right.right, target);
            node = leftRotate(node);
            steps.push(
                `Вращение Zag-Zag (Левый поворот родителя д...`
            );
        }

        if (!node.right) return node;
        steps.push(
            `Финальный поворот Zag (Левое вращение корня)`
        );
        return leftRotate(node);
    }
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
    setHighlightedNode(key);
    const { newRoot, steps } = splayStepByStep(tree, key);
    if (newRoot) {
        setTree(newRoot);
    }
};

```

```

    x: number,
    y: number,
    levelWidth: number,
  ): void => {
    if (!node) return;
    node.x = x;
    node.y = y;
    if (node.left) {
      computeCoordinates(node.left, x - levelWidth, y +
    }
    if (node.right) {
      computeCoordinates(node.right, x + levelWidth, y +
    }
  }
};

```

```

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

```

```

// Render lines and nodes

```

```

const renderLines = (node: SplayNode | null): React.R
  if (!node) return [];
  let lines: React.ReactNode[] = [];
  if (node.left && node.left.x && node.left.y) {
    lines.push(
      <line
        key={`l-${node.key}-${node.left.key}`}
        x1={node.x}
        y1={node.y}
        x2={node.left.x}
        y2={node.left.y}
        stroke="#475569"
        strokeWidth="2"
      />,
    );
    lines = lines.concat(renderLines(node.left));
  }
  if (node.right && node.right.x && node.right.y) {
    lines.push(

```



```

        y={node.y}
        dy=".3em"
        textAnchor="middle"
        fill="white"
        fontSize="12px"
        fontWeight="bold"
        className="select-none pointer-events-none"
      >
        {node.key}
      </text>
    </g>,
  );
  circles = circles.concat(renderNodes(node.left));
  circles = circles.concat(renderNodes(node.right));
  return circles;
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    {/* Header */}
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <BookOpen className="w-5 h-5 text-indigo-400"
        Интерактивное Splay-дерево
      </h3>
      <p className="text-xs text-slate-400">
        Кликните на вершины дерева, чтобы «вытащить»
        Splay.
      </p>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-12
      {/* Playfield */}
      <div className="lg:col-span-7 bg-[#0f172a] p-4
        <svg
          className="w-full h-full max-w-[600px] max-h-[320px]
          viewBox="0 0 600 320"
        >

```

```

        0"
        >
        Вставить
    </button>
</form>

<button
  onClick={resetTree}
  className="flex items-center gap-1.5 px-3"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Сброс
  </button>
</div>
</div>

{/* Logs */}
<div className="lg:col-span-5 bg-slate-950 border-1
  to overflow-y-auto custom-scrollbar">
  <h4 className="text-xs font-bold text-slate-400">
    Ход вращения (Splay-логи)
  </h4>
  <div className="flex-1 space-y-2 mb-4 overflow-y-auto">
    {log.map((step, idx) => (
      <div
        key={idx}
        className={`text-xs p-2.5 rounded transition
          idx === log.length - 1
            ? "border-l-2 border-indigo-500 bg-slate-900"
            : "text-slate-400 bg-slate-900/50"
        }`}
      >
        {step}
      </div>
    ))}
  </div>
</div>

<div className="border-t border-slate-800 pt-1">
  <p>

```

```

        </strong>
        , на котором он споткнулся! Это оставля
        сверху.
    </p>
</div>
<div className="mt-3 text-[10px] text-indigo
    Поиск настраивает локальный кэш под реали
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
    <div>
        <p className="font-bold text-slate-300 mb
            2. Эффект качелей (Swing)
        </p>
        <p className="text-slate-400 leading-rela
            Если агент запрашивает по очереди{" "}
            <code className="text-rose-400">[10]</c
            <code className="text-rose-400">[60]</c
            углы), дерево будет превращаться в палк
            сторону. Первая операция Splay(60) стои
            <strong className="text-white">0(n)</st
            глубину n. Вторая Splay(10) заставляет
            Постоянный свинг убивает производительн
            сложности <strong className="text-white
        </p>
    </div>
</div>
<div className="mt-3 text-[10px] text-rose-
    Кэшировать свингующие пары на концах — фа
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
    <div>
        <p className="font-bold text-slate-300 mb
            3. Амортизация  $O(\log n)$ 
        </p>
        <p className="text-slate-400 leading-rela

```

```

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20' },
  { name: 'Тарелка из-под пиццы', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20' },
  { name: 'Блюдец от торта', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20' },
  { name: 'Тарелка от тако', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20' },
  { name: 'Тарелка из-под суши', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20' },
  { name: 'Сковорода от глазуньи', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20' },
];

let counter = 0;

export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>(
    [
      { id: 'p1', name: 'Тарелка из-под спагетти', emoji: ' ', state: 'idle' },
      { id: 'p2', name: 'Тарелка из-под пиццы', emoji: ' ', state: 'idle' },
      { id: 'p3', name: 'Блюдец от торта', emoji: ' ', state: 'idle' },
    ]
  );

  const runtime = useVizRuntime();
  const liveStack = vizArray(runtime?.variables?.st) ?? [];
  const displayStack: Plate[] = liveStack
    ? liveStack.map((value, index) => ({
        id: `python-${index}`,
        name: String(value),
        emoji: " ",
        color: "from-indigo-500/20 to-indigo-600/20 border-indigo-500/20",
        isDirty: true,
        animState: "idle",
      }))
    : dirtyStack;

  const [cleanRack, setCleanRack] = useState<Plate[]>(
    []
  );
  const [isWashing, setIsWashing] = useState(false);
  const [showSoap, setShowSoap] = useState(false);

```

```

    addLog('△ POP: Нечего мыть! Все тарелки чистые.')
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.name}`);

  // Step 1: Start washing animation (sponge and soap)
  setDirtyStack(prev => prev.map(p => p.id !== topPlate.id));
  setShowSoap(true);

  setTimeout(() => {
    // Step 2: Wash complete. Plate becomes clean and goes to clean rack
    const cleanPlate: Plate = {
      ...topPlate,
      isDirty: false,
      animState: 'clean',
    };

    setCleanRack(prev => [cleanPlate, ...prev].slice(0, 3));
    setDirtyStack(prev => prev.slice(0, prev.length - 1));
    setShowSoap(false);
    setIsWashing(false);
    addLog(`  Готово! Чистая тарелка ${topPlate.emoji}`);
  }, 850);
}, [dirtyStack, isWashing]);

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', animState: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', animState: 'in' },
    { id: 'r3', name: 'Блюдце от торта', emoji: '🍰', animState: 'in' },
  ]);
  setCleanRack([]);
  setShake(true);
  setShowSoap(false);
  setIsWashing(false);
  addLog('🔄 Сброс!');
};

```

```

        onClick={popPlate}
        disabled={isWashing || dirtyStack.length === 0}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-900
        opacity-40 disabled:cursor-not-allowed text-white font-mono text-sm
        border border-slate-700 border-radius-10px padding-4 transition-all cursor-pointer"
      >
        <span> Помыть верхнюю (POP)</span>
      </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-mono text-sm
      cursor-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

  { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
  <div className="grid grid-cols-1 md:grid-cols-12 gap-4" >

    { /* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */ }
    <div className="md:col-span-7 bg-slate-900 rounded-2xl padding-4" >
      <div className="absolute top-4 left-4 text-white font-mono text-sm" >
        Грязная стопка (Стек вызовов / LIFO)
      </div>

      { /* Визуализация раковины */ }
      <div className="absolute top-4 right-4 flex items-center justify-center" >
        <div className="bg-slate-800/80 text-[10px] font-mono" >
          <span className="w-1.5 h-1.5 rounded-full border border-slate-700" >
            <span>РАКОВИНА </span>
          </span>
        </div>
      </div>

      { /* Мыльные пузыри при мытье */ }
      {showSoap && {
        <div className="absolute inset-0 pointer-events-none" >

```

```

        key={plate.id}
        className={`
          w-full max-w-[280px] h-10 rounded
          shadow-md select-none transition-all duration-200
          ${plate.color}
          ${plate.animState === 'in' ? 'animate-in' : ''}
          ${plate.animState === 'washing' ? 'animate-wash' : ''}
          ${isTop ? 'ring-4 ring-blue-500/50' : ''}
        `}
      >
        <span className="text-lg">{plate.name}</span>
        <span className="text-slate-800 text-sm">
          {isTop && (
            <span className="text-[9px] bg-blue-500/50 px-1">
              LIFO
            </span>
          )}
        </div>
      </div>
    );
  }
}
</div>
))

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r from-slate-200 to-slate-400">
  </div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 rounded-2xl p-4">
  <div className="absolute top-4 left-4 text-[10px] text-slate-300">
    Сушилка для чистой посуды
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-center gap-2">
    <div className="order-emerald-800/80 text-[9px] font-mono">
      <Sparkles className="w-3.5 h-3.5 animate-pulse">
        <span>ЧИСТО </span>
      </div>
    </div>
  </div>
</div>

```

```
        <span>{entry}</span>
      </div>
    )})
  </div>
</div>
);
};
```

ФАЙЛ 69/87: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef, useMemo } from "react";
import { useVizRuntime, vizArray, vizNumber, vizString } from "viz-runtime";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";
```

```
function generateKmpSteps(pattern: string, text: string) {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];
```

```
  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });
```

```
  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];
```

```
    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
    ;
```

```
    while (j > 0 && s[i] !== s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
      // яд назад: откат j = pi[pi[j] - 1] = pi[j - 1]
      j = pi[j - 1];
    }
  }
}
```



```

        }. Копипаста сработала.` }));
    }

    steps.push({ i, l, r, zTarget: z[i], zSource: i
    while (i + z[i] < n && s[z[i]] === s[i + z[i]])
        steps.push({ i, l, r, zTarget: z[i], zSource:
        падают! Окно растёт.` }));
        z[i]++;
    }

    if (i + z[i] < n) {
        steps.push({ i, l, r, zTarget: z[i], zSource:
        [i]}}' разные. Стоп.` }));
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Упё
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксирывае

    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
        steps.push({ i, l, r, z: [...z], desc: ` 0
    }

    if (z[i] === pattern.length) {
        steps.push({ i, l, r, z: [...z], found: true
    }
}
return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp"
const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
const [pattern, setPattern] = useState("aba");
const [text, setText] = useState("abacaba");

// Ensure inputs are valid dynamically

```

```

const reset = () => { setAutoPlay(false); setStepId

const baseStep = steps[stepIdx] || steps[0];
const vars = runtime?.variables;
const liveValues = vizArray(mode === "kmp" ? vars?.l
const hasLiveValues = !!vars && Object.prototype.ha
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveVal
const step = compilerLinked
  ? {
    ...baseStep,
    i: liveI ?? baseStep.i,
    j: liveJ ?? baseStep.j,
    l: liveL ?? baseStep.l,
    r: liveR ?? baseStep.r,
    [mode === "kmp" ? "pi" : "z"]: hasLiveValues
    desc: `Компилятор: i=${liveI ?? "-"}${mode ===
  }
  : baseStep;

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>
    </div>

```

```

        bgColor = "bg-emerald-900";
        borderColor = "border-emerald-900";
    }
    if (step.zTarget === index) {
        borderColor = "border-amber-500";
        bgColor = step.match === "L" ?
-900/40");
    }
}

return (
    <div key={index} className={
        /* L/R markers for Z */
        {mode === "z" && step.l
            <div className="absolute
        }
        {mode === "z" && step.r
            <div className="absolute
        }

        /* Index */
        <span className={`text-amber-500
            font-size-20px font-weight-700
            font-mono text-lg transition-colors
            {char}
        </div>

        /* Value array box */
        <div className="text-[#fcd34d]
        slate-700/50 font-mono font-bold">
            {(mode === "kmp" ?
        </div>

        /* KMP fingers */
        {mode === "kmp" && step
            <div className="absolute

```

```

    }}
    {step.found && mode ===
      <div className="absolut
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_
    }}
  </div>
)
}}
</div>
</div>

<div className="flex flex-col sm:flex-row gap-4"
  {/* Controls */}
  <div className="flex bg-slate-900 border-1px border-slate-800 rounded-lg disabled:opacity-30 disabled:hover:opacity-100"
    <button onClick={prevStep} disabled={step === 0}
      <Undo strokeWidth={3} size={16} />
    </button>
    <button onClick={playPause} className="flex-1"
      <div className="flex items-center justify-center min-w-[80px]"
        {autoplay ? <><Pause size={16} /> : <><Play size={16} />}
      </div>
    </button>
    <button onClick={nextStep} disabled={step === steps - 1}
      <SkipForward strokeWidth={3} size={16} />
    </button>
    <button onClick={reset} className="flex-1"
      <div className="flex items-center justify-center min-w-[80px]"
        <RefreshCw strokeWidth={3} size={16} />
      </div>
    </button>
  </div>

  {/* Description Log */}
  <div className="flex-1 bg-slate-900/50 border-1px border-slate-800 rounded-lg p-4 overflow-hidden">
    <div className="absolute top-0 right-0 bottom-0 left-0"
      <div className="font-mono text-sm"
        <div>
          {log}
        </div>
      </div>
    </div>
  </div>

```



```

    Pause,
    RotateCcw,
    Clock,
  } from "lucide-react";

interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}

interface TEdge {
  u: number;
  v: number;
}

const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
  { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
  { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
  { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
  { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
  { u: 0, v: 1 },
  { u: 0, v: 2 },
  { u: 1, v: 3 },
  { u: 2, v: 3 },
  { u: 3, v: 4 },
];

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
}
```

```

    recordStep(
      "Начало топологической сортировки. Используем кла
      null,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, () => []);
    dagEdges.forEach((e) => adj[e.u].push(e.v));

    const dfs = (u: number) => {
      visited.add(u);
      dfsStack.push(u);
      recordStep(
        `DFS: зашли в вершину ${u} (${dagNodes[u].name}
        u,
      );

      for (const to of adj[u]) {
        if (!visited.has(to)) {
          dfs(to);
        } else if (!fullyProcessed.has(to)) {
          // Explaining potential cycle detected (not in
          recordStep(
            `⚠ Внимание: Рёбра в серую вершину ${to} о
            u,
          );
        }
      }
    }

    fullyProcessed.add(u);
    dfsStack.pop();
    topoList.push(u); // prepend behavior: we write in
    recordStep(
      `DFS: закончили обработку '${dagNodes[u].name}'
      ,
      u,
    );
  };
};

```

```

    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)
    .join(" → ");
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучен
      </h3>

      <div className="flex items-center gap-2 bg-slate-
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            500 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

```



```

    { /* Edges */
    {dagEdges.map((edge, idx) => {
      const u = dagNodes.find((n) => n.id === edge.u)
      const v = dagNodes.find((n) => n.id === edge.v)

      const isActive =
        (step.currentNode === edge.u || step.currentNode === edge.v) ||
        step.visited.has(edge.v);

      return (
        <line
          key={`edge-${idx}`}
          x1={u.x}
          y1={u.y}
          x2={v.x}
          y2={v.y}
          stroke={isActive ? "#818cf8" : "#334155"}
          strokeWidth={isActive ? "3" : "2"}
          markerEnd={`url(#${isActive ? "dag-arrow-${idx}" : ""}`}
          className="transition-all duration-300"
        />
      );
    })}
  )}
}

```

```

    { /* Nodes */
    {dagNodes.map((node) => {
      const isCurrent = step.currentNode === node.id
      const isVisited = step.visited.has(node.id)
      const isProcessed = step.fullyProcessed.has(node.id)

      let fill = "#1e293b";
      let stroke = "#334155";
      let textColor = "text-slate-400";
      void textColor;

      if (isCurrent) {
        fill = "#312e81";
      }
    })}
  )}
}

```

```

        y={node.y + 12}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="9px"
        className="pointer-events-none"
      >
        {node.name.split(" ").slice(1).join(" ")}
      </text>
    </g>
  );
}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800 p-4">
    <p className="text-xs text-indigo-300 min-h-[100px]">
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto">
    {/* Topological List output */}
    <div>
      <span className="text-[10px] text-slate-500">
        РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
      </span>
      <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
        {step.topoList.length === 0 ? (
          <span className="text-slate-600 text-sm">
            ожидаем обработку узлов...

```

```
        Назад
      </button>
      <button
        disabled={currentStepIdx >= steps.length}
        onClick={() => setCurrentStepIdx((prev) => prev - 1)}
        className="bg-slate-900 border border-slate-800 text-white"
      >
        Вперед
      </button>
    </div>
  </div>
</div>
</div>
</div>
);
};
```

ФАЙЛ 72/87: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
```

```
export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, ч
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
  "heap-beam-search": {
    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы – куча сама держит лучшую
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .
```

```

"planarity-euler-formula": { title: "Планарность: K5 и K5,1",
dijkstra: {
  title: "Дейкстра",
  hint: "Жадно забираем ближайшую вершину и релаксируем",
  Component: () => (
    <>
      <ChapterImage vizType="dijkstra" />
      <DijkstraViz />
    </>
  ),
},
"bellman-ford": {
  title: "Форд-Беллман",
  Component: () => (
    <>
      <ChapterImage vizType="bellman-ford" />
      <BellmanFordViz />
    </>
  ),
},
floyd: {
  title: "Флойд-Уоршелл",
  Component: () => (
    <>
      <ChapterImage vizType="floyd" />
      <FloydViz />
    </>
  ),
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: () => (
"mst-kruskal": { title: "Краскал и DSU", Component: () => (
"mst-prima": { title: "Прим", Component: () => (
"mst-boruvka": { title: "Борувка", Component: () => (
"string-kmp": { title: "Префикс-функция (КМП)", Component: () => (
"string-z-func": { title: "Z-функция", Component: () => (
"aho-corasick": {
  title: "Ахо-Корасик",
  hint: "Бор, суффиксные ссылки и BFS-обход по уровням"
}

```

```

    words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
  0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, d: 100, r: 100,
    // Ветка H -> E -> R -> S
  1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, d: 100, r: 100,
  2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, d: 100, r: 100,
  3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, d: 100, r: 100,
  4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, d: 100, r: 100,
    // Ветка H -> I -> S
  5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, d: 100, r: 100,
  6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, d: 100, r: 100,
    // Ветка S -> H -> E
  7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, d: 100, r: 100,
  8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, d: 100, r: 100,
  9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, d: 100, r: 100,
    = 2 (HE)
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
  0: { H: 1, S: 7 },
  1: { E: 2, I: 5 },
  2: { R: 3 },
  3: { S: 4 },
  4: {},
  5: { S: 6 },
  6: {},
  7: { H: 8 },
  8: { E: 9 },
  9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training'>('training');

```

```

    }

    const char = textToScan[currentIndex];
    let curr = currentNode;
    let logMsg = `[Символ '${char}']`;

    let jumpType: 'normal' | 'fail' | null = null;
    void jumpType;
    let originalCurr = curr;

    if (TRIE_TRANSITIONS[curr][char] !== undefined) {
        const nextNode = TRIE_TRANSITIONS[curr][char];
        logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
        setJumpPath({ from: curr, to: nextNode, type: 'normal' });
        curr = nextNode;
        setActiveSearchCodeLine(3);
    } else {
        setActiveSearchCodeLine(2);
        let jumps = 0;
        while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
            curr = WATERFALL_NODES[curr].fail;
            jumps++;
        }
        const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
        if (originalCurr === 0) {
            logMsg += `В корне нет перехода по '${char}'. Лист`;
            setJumpPath(null);
        } else {
            logMsg += `Поток по '${char}' у вершины #${originalCurr}`;
            logMsg += `рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label})`;
            setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
            curr = nextNode;
        }
    }

    const currentMatches: { index: number; word: string } = [];

```

```

    },
    {
        targetNodeId: 1,
        title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
        desc: 'Мы обрабатываем первую вершину из очереди -  
        просто не существует. fail[H] автоматически нап
        codeLine: 5,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'H',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 7,
        title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
        desc: 'Обрабатываем дочернюю вершину #7 (S) на Dep
        ,
        codeLine: 5,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 2,
        title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
        desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
        а-разведчик плавает в ROOT и ищет ветку по "E".
        codeLine: 3,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'E',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 5,
        title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
        desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель
    
```



```

        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 9,
        title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С...
        desc: '    МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
            ба плывет в #1 (H) и ищет ветку "E". У вершины :
        codeLine: 4,
        scoutPos: 1,
        inspectedParentFail: 1,
        checkTargetChar: 'E',
        checkingBranchesFrom: 1,
    },
    {
        targetNodeId: 4,
        title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
        desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро...
            #7 (S). fail[HERS] = #7 (S). Автомат полностью
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },
};

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNode];

```

```

return (
    <div className="bg-slate-800/95 rounded-2xl p-6 border" >
        /* Шапка и переключатель режимов */
        <div className="flex flex-wrap items-center justify-center" >
            <div className="flex items-center space-x-3" >
                <span className="text-3xl" > </span>
                <div>

```

```

/* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
<div className="grid grid-cols-1 lg:grid-cols-3"
  {/* Управление шагами обучения */}
  <div className="bg-slate-900/90 p-5 rounded-x
    <div>
      <h5 className="text-indigo-300 font-bold"
        <span> </span> Полный обход в ширину (В
      </h5>
      <p className="text-xs text-slate-300 mb-3"
        Показаны все шаги очереди BFS (начиная с
        ар 8 (SHE → HE)</strong>, где наглядно видно
      </p>
      <div className="space-y-1.5 mb-6 overflow
        {TRAINING_STEPS_INFO.map((step, idx) =>
          <button
            key={idx}
            onClick={() => setTrainStep(idx)}
            className={`w-full text-left px-3 p
              ${
                idx === trainStep
                  ? 'bg-indigo-600 text-white font
                  : 'bg-slate-800 text-slate-400
              }`}
            >
              <span className="line-clamp-1">{step
                {idx === trainStep && <span classNa
              </button>
            )}}
          </div>
        </div>

      <div className="flex gap-2">
        <button
          onClick={() => setTrainStep((prev) => M
          disabled={trainStep === 0}
          className="flex-1 bg-slate-700 hover:bg
          ion-all"
        >

```

```

        </div>
        <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
        <span>3. while (v !== root && trie[v]
        {currentTrainInfo.codeLine === 3 && <
т, возврат в корень</span>}
        </div>
        <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
        <span>4. if (trie[v][char] !== undefin
/span>
        {currentTrainInfo.codeLine === 4 && <
ан IF!</span>}
        </div>
        <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
        <span>5. else fail[u] = root; // Bero
        {currentTrainInfo.codeLine === 5 && <
root</span>}
        </div>
    </div>
</div>

<div>
    <h5 className="text-slate-200 font-bold m
    <Lightbulb className="w-4 h-4 text-ambe
    </h5>
    <div className="bg-slate-800 p-4 rounded-:
w-inner min-h-[72px] flex items-center">
        {currentTrainInfo.desc}
    </div>
</div>
</div>
</div>
) : (
/* ПАНЕЛЬ РЕЖИМА ПОИСКА */
<div className="grid grid-cols-1 lg:grid-cols-3
    <div className="bg-slate-900/60 p-5 rounded-x

```

```

        </span>
      })
    </div>

    <button
      onClick={stepSearchSimulation}
      disabled={isSearchDone || textToScan.length === 0}
      className={`w-full py-2.5 rounded-lg font-medium transition-colors
        ${isSearchDone || textToScan.length === 0
          ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
          : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
      `}
    >
      <span>{currentIndex === 0 ? 'Начать план' : 'Продолжить'}</span>
      <ArrowRight className="w-4 h-4" />
    </button>
  </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl">
  <div>
    <h5 className="text-white font-bold mb-3">
      <span>✂</span> Текущее действие в коде
    </h5>
    <div className="bg-slate-950 p-4 rounded-2xl">
      <div className={`p-1.5 rounded flex items-center justify-between border-blue-500 text-blue-300 font-bold`} >
        <span>1. curr = state; char = text[i]
        {activeSearchCodeLine === 1 && <span>инициализация</span>}
      </div>
      <div className={`p-1.5 rounded flex items-center justify-between border-rose-500 text-rose-300 font-bold`} >
        <span>2. while (curr !== root && !trie[curr].next)
        {activeSearchCodeLine === 2 && <span>прыжок по fail!</span>}
      </div>

```

```
</div>
```

```
<div className="flex items-center justify-between">
  <h5 className="text-white font-bold text-base">
    <span> </span> Ветвящийся синий водопад (Борис)
  </h5>
```

```
<div className="flex flex-wrap gap-4 text-xs">
  <span className="flex items-center gap-1 text-white">
    <span className="w-3 h-0.5 bg-blue-500 inline-block">
    </span>
    <span className="flex items-center gap-1 text-white">
      <span className="w-3 h-0.5 border-t border-blue-500">
      </span>
    </span>
  </div>
</div>
```

```
{/* Само SVG дерево водопада */}
```

```
<div className="w-full overflow-x-auto custom-scrollbar">
  <svg viewBox="0 0 800 520" className="w-full h-full">
```

```
    {/* Горизонтальные линии и подписи уровней */}
    {[
      { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
      { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
      { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
      { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
      { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' }
    ]}.map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line
          x1="20"
          y1={level.y}
          x2="780"
          y2={level.y}
          stroke="#334155"
          strokeWidth="1"
          strokeDasharray="4,4"
          opacity="0.4"
        >
```



```

<g key={`node-${node.id}`} className="с
  {/* Внешнее свечение для активной вер
  {isCurrent && {
    <circle cx={node.x} cy={node.y} r="
  }}
  {isTargetChild && {
    <circle cx={node.x} cy={node.y} r="
  }}

  {/* Основной круг вершины */}
  <circle
    cx={node.x}
    cy={node.y}
    r={isCurrent || isTargetChild ? '24'
    fill={isCurrent ? '#2563eb' : isTar
    stroke={isCurrent ? '#ffffff' : isTa
    strokeWidth={isCurrent || isTargetCh
    className="transition-all duration-
  />

  {/* Текст внутри вершины */}
  <text
    x={node.x}
    y={node.y + 5}
    fill={isCurrent || isTargetChild ?
    fontSize={isCurrent || isTargetChil
    fontWeight="bold"
    textAnchor="middle"
  >
    {node.label}
  </text>

  {/* Метка искомой вершины в режиме об
  {isTargetChild && {
    <g transform={`translate(${node.x},
      <rect x="-35" y="-12" width="70"
      <text x="0" y="2" fontSize="10" f
  ИЩЕМ FAIL

```

```

        >
        {activeMode === 'training' ? ' ♂ ' :
        </text>
    </g>

    </svg>
</div>
</div>

{/* Список найденных совпадений (только в режиме
{activeMode === 'search' && (
    <div className="mt-6 bg-slate-900/50 p-5 rounded
        <h5 className="text-white font-bold mb-3 text
            <span> </span> Улов лосося (Найденные слова
        </h5>
        {foundMatches.length === 0 ? (
            <p className="text-sm text-slate-400 italic
                Пока ничего не поймали. Запустите шаги си
            </p>
        ) : (
            <div className="flex flex-wrap gap-2">
                {foundMatches.map((match, idx) => (
                    <div
                        key={idx}
                        className="bg-emerald-950 border bord
                    -sm font-bold shadow animate-[scaleUp_0.3s_
                    >
                        <CheckCircle2 className="w-4 h-4 text
                        <span>{match.word}</span>
                        <span className="text-[10px] bg-emera
                            Индекс {match.index}
                        </span>
                    </div>
                )))}
            </div>
        )}
    </div>
)}

```



```

    [9, 1, 2, 3],
    [4, 5, 6, 7],
  ];

const cellBase =
  "flex items-center justify-center rounded-md font-mono" +
  "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  const chapterId = useContext(VizChapterContext);

  // Вкладка 2D = своё демо (свой скелет): компилятор по
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, mode === "2d"
    }, [chapterId, mode]);

  return (
    <div className="w-full bg-slate-950 rounded-2xl border" >
      <div className="flex gap-2 mb-4 overflow-x-auto no-wrap" >
        {(
          [
            ["1d", "1. Одномерные суммы"],
            ["2d", "2. Двумерные суммы"],
          ] as const
        ).map(([key, label]) => (
          <button
            key={key}
            onClick={() => setMode(key)}
            className={`px-3 py-2 rounded-lg text-xs sm:text-sm
              mode === key ? "bg-indigo-600 text-white"
            }`}
          >
            {label}
          </button>
        ))}
      </div>
      {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
    </div>
  );
};

```

```

<div className="space-y-5">
  <div className="overflow-x-auto pb-1">
    <div className="inline-block min-w-full">
      {/* индексы */}
      <div className="flex gap-1.5 mb-1 pl-[52px]">
        {A1.map(({_, i}) => {
          <div key={i} className={` ${cellBase} !h-5`}>
            {i}
          </div>
        })}
      </div>

      {/* массив a */}
      <div className="flex gap-1.5 items-center mb-1">
        <div className="w-[46px] shrink-0 text-right">
          {A1.map(({v, i}) => {
            const inCover = covered && i >= covered.f
            const inRange = i >= range.l && i <= range.r
            const isDrivenAdd = shownDriven !== null && i === shownDriven
            return (
              <div
                key={i}
                onMouseEnter={() => setHover({ kind: 'add' })}
                onMouseLeave={() => setHover(null)}
                onClick={() => setRange((r) => (i < r.l || i > r.r) ? r : { l: i, r: i })}
                className={` ${cellBase} cursor-pointer`}>
                  {v}
                  {isDrivenAdd
                    ? "bg-amber-500 text-black ring-2"
                    : inCover
                    ? "bg-indigo-500 text-white ring-2"
                    : inRange
                    ? "bg-indigo-900/70 text-indigo-300"
                    : "bg-slate-800 text-slate-300"}
                >
                    {v}
                  </div>
                </div>
              </div>
            )
          })}
        </div>
      </div>
    </div>
  </div>

```

```

        {blank || masked ? "." : String(shown)}
    </div>
    );
  }}}
</div>
</div>
</div>

```

```

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border"
  {shownDriven !== null && !hover ? (
    <p className="text-slate-300">
      <b className="text-emerald-400">Компилятор
      <span className="font-mono text-white">i =
      <span className="font-mono text-emerald-300">
        {hasLiveP && <> = <span className="font-mono
      <span className="ml-1 text-slate-500">Пустой
    </p>
  ) : hover?.kind === "pref" ? (
    hover.i === 0 ? (
      <p className="text-slate-300">
        <b className="text-emerald-400">P[0] = 0<
        отрезка, начинающегося с нуля.
      </p>
    ) : (
      <p className="text-slate-300">
        <b className="text-emerald-400">
          P[{hover.i}] = {pref[hover.i]}
        </b>{" "}
        — это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
          a[0..{hover.i - 1}] = {A1.slice(0, hove
        </span>
      </p>
    )
  ) : hover?.kind === "a" ? (
    <p className="text-slate-300">
      <b className="text-indigo-400">

```

```
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveS = vizArray(vars?.S);
const hasLiveS = !!vars && Object.prototype.hasOwnProperty.call(vars, 'S');
```

```
const P = useMemo(() => {
  const p = Array.from({ length: n + 1 }, () => Array.from({ length: m + 1 }, () => 0));
  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= m; j++) {
      p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1];
    }
  }
  return p;
}, [n, m]);
```

```
const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });
```

```
useEffect(() => {
  const r1 = vizNumber(vars?.r1);
  const c1 = vizNumber(vars?.c1);
  const r2 = vizNumber(vars?.r2);
  const c2 = vizNumber(vars?.c2);
  if (r1 === null || c1 === null || r2 === null || c2 === null) return;
  setRect({
    r1: Math.max(0, Math.min(n - 1, Math.trunc(r1))),
    c1: Math.max(0, Math.min(m - 1, Math.trunc(c1))),
    r2: Math.max(0, Math.min(n - 1, Math.trunc(r2))),
    c2: Math.max(0, Math.min(m - 1, Math.trunc(c2))),
  });
}, [vars, n, m]);
```

```
const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;
```

```

    </div>
  </div>

  { /* Матрица префиксных сумм */ }
  <div className="overflow-x-auto">
    <div className="text-xs text-slate-400 mb-2">
      <div className="inline-block">
        {P.map((row, i) => {
          <div key={i} className="flex gap-1.5 mb-1">
            {row.map((v, j) => {
              const liveRow = vizArray(liveS?.[i]);
              const liveValue = liveRow?.[j];
              const blank = hasLiveS && (liveValue :
              const shownValue = hasLiveS ? liveVal
              const corner = cornerOf(i, j);
              const isHover = hover?.i === i && hov
              const isCompilerCell = liveI === i &&
              const cornerColor =
                corner === "A"
                  ? "bg-emerald-600 text-white"
                  : corner === "D"
                    ? "bg-sky-600 text-white"
                    : corner
                      ? "bg-rose-600 text-white"
                      : "bg-slate-900 border border
            return {
              <div
                key={j}
                onMouseEnter={() => setHover({ i,
                onMouseLeave={() => setHover(null
                className={` ${cellBase} cursor-he
                isCompilerCell
                  ? "bg-emerald-600 text-white
                  : isHover
                    ? "bg-amber-500 text-white
                    : blank
                      ? "bg-slate-950 text-slate
                      : cornerColor

```

```
        <p className="text-[11px] text-slate-500 mt-1">
          Вычли два «хвоста» и вернули дважды вычитенный
        </p>
      </div>
    </div>
  );
};
```

ФАЙЛ 75/87: src/components/visualizers/SparseTableViz.t
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 817 | РАЗМ:

```
import React, { useContext, useEffect, useState } from
import {
  emitVizDemo,
  useVizRuntime,
  vizArray,
  vizNumber,
  vizRecord,
  VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;
```

```

        for (let c = 0; c + (1 << ky) <= 8; c++) {
            if (kx > 0) {
                ST2D[r][c][kx][ky] = Math.min(
                    ST2D[r][c][kx-1][ky],
                    ST2D[r + (1 << (kx-1))][c][kx-1][ky]
                );
            } else {
                ST2D[r][c][kx][ky] = Math.min(
                    ST2D[r][c][kx][ky-1],
                    ST2D[r][c + (1 << (ky-1))][kx][ky-1]
                );
            }
        }
    }
}

```

```

export const SparseTableViz: React.FC = () => {
    // Эта страница посвящена именно двумерной таблице: 1D
    // вкладкой, но больше не подменяет 2D-визуализацию и
    const [tab, setTab] = useState<"1d" | "2d_build" | "2d_viz">("1d");
    const chapterId = useContext(VizChapterContext);

    // Каждая вкладка = своё демо с собственным скелетом:
    useEffect(() => {
        if (chapterId) emitVizDemo(chapterId, tab === "2d_build" ? "2d_build" : "2d_viz", [chapterId, tab]);
    }, [chapterId, tab]);

    return (
        <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
            <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
                <button
                    onClick={() => setTab("1d")}
                    className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white : "bg-slate-800 text-slate-400"
                >
                    1. Сворачивание 1D
                </button>
            </div>
        </div>
    );
}

```

```

const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'liveSource');
const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'liveTable');
const compilerLinked = liveI !== null || liveJ !== null;
const shownSource = hasLiveSource
  ? liveSource?.length
  ? liveSource
  : Array.from({ length: N }, () => null)
  : arr1D;
const liveTarget =
  liveI !== null && liveJ !== null && Number.isInteger(liveI) && Number.isInteger(liveJ)
  ? { i: liveI, j: liveJ }
  : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    computeSteps.push({ i, j });
  }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i, to: pointedCell.i + (1 << pointedCell.j) } : null;

return (
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-600">
          Построение Sparse Table (Min)
        </h3>
        <p className="text-sm text-slate-400">
          Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$ 
        </p>
      </div>
    </div>
  </div>
);

```



```

        w,44px)) h-[clamp(26px,8vw,44px)] text-[clamp(16px,8vw,26px)]
        inCover ? "bg-indigo-500 text-white" : "bg-white text-indigo-500"
      `}
    >
      {display}
    </div>
    <span className="text-[9px] text-slate-100" >{label}</span>
  </div>
);
}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
  {hover && covered ? (
    <p className="text-slate-300">
      <b className="text-sky-400">
        ST[{hover.i}][{hover.j}] = {stID[hover.i][{hover.j}]}
      </b>{" "}
      – это минимум на отрезке{" "}
      <span className="font-mono text-sky-300">
        [{covered.from}, {covered.to}]
      </span>{" "}
      длины  $2^{\{hover.j\}}$  = {1 << hover.j}:{ " "}
      <span className="font-mono text-slate-400">
        min({arrID.slice(covered.from, covered.to)}[0], {arrID.slice(covered.to, covered.to+1)}[0])
      </span>
    </p>
  ) : (
    <p className="text-slate-500">
      Наведите курсор (или тапните) на любое число
    </p>
  )}
</div>
</div>

<div className="overflow-x-auto pb-2">
  <div className="inline-block min-w-full bg-slate-100">
    <div className="grid grid-cols-[auto_repeat(4,1fr)]">
      {/* Headers */}
    </div>
  </div>
</div>

```

```
const isVisible = compilerLinked ? run
```

```
let isActive = !!liveTarget && liveTa
```

```
let isSrc1 = false;
```

```
let isSrc2 = false;
```

```
if (compilerLinked && liveTarget && l
```

```
  isSrc1 = i === liveTarget.i && j ===
```

```
  isSrc2 = i === liveTarget.i + (1 <<
```

```
} else if (!compilerLinked && step > 1
```

```
  const currentStep = computeSteps[ste
```

```
  isActive = currentStep.i === i && c
```

```
  if (currentStep.j - 1 === j) {
```

```
    if (currentStep.i === i) isSrc1 =
```

```
    if (currentStep.i + (1 << (current
```

```
  }
```

```
}
```

```
// Готовые вычисленные уровни демо не
```

```
// уровень j=0 всегда берём из уже сущ
```

```
const shownValue = compilerLinked ? r
```

```
const displayValue =
```

```
  shownValue === undefined || shownVa
```

```
  ? "."
```

```
  : typeof shownValue === "object"
```

```
    ? JSON.stringify(shownValue)
```

```
    : String(shownValue);
```

```
return {
```

```
  <div key={j} className="relative fl
```

```
    {!isValid ? {
```

```
      <div className="w-[clamp(30px,9
```

```
    ) : {
```

```
      <motion.div
```

```
        onMouseEnter={() => (compiler
```

```
        onMouseLeave={() => setHover(r
```

```
        onClick={() => (compilerLinke
```

```
    ]])
```

```

        top: 24,
        overflow: "visible",
      }}
    >
      <path
        d={`M 0 0 C 30 0, 30 ${isSrc1 ? "100%" : "0%"}
        fill="none"
        stroke={isSrc1 ? "#10b981" : "#f87171"}
        strokeWidth="3"
      />
    </motion.svg>
  }}
</div>
);
}}
</React.Fragment>
)}}
</div>
</div>
</div>

```

```

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-800 p-4">
  {compilerLinked && liveTarget ? (
    <p className="text-base text-slate-300">
      <span className="text-emerald-400 font-bold">Formula</span>
      <span className="font-mono text-white">i =
      – активна ячейка <span className="font-mono text-white">
        {(hasLiveTable || hasLiveSource) && <> = <src>
        ?.[liveTarget.j] ?? {liveTarget.j === 0 ? liveTarget : 1}
      </span>
    </p>
  ) : step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">Step {step}</span>
        </p>
      );
    })()
  ) : null}

```

```

const liveK = vizNumber(vars?.k);
const liveR = vizNumber(vars?.r);
const liveC = vizNumber(vars?.c);
const liveSource = vizArray(vars?.A);
const liveTable = vizRecord(vars?.st2);
const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'A');
const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'st2');
const codeCell = liveR !== null && liveC !== null ? {
  k: liveK,
  r: liveR,
  c: liveC,
  source: liveSource,
  table: liveTable,
  compilerLinked: codeCell !== null || liveK !== null
} : null;
const shownK = liveK !== null
  ? Math.max(0, Math.min(3, Math.trunc(liveK)))
  : hasLiveSource
    ? 0
    : k;

```

```

// Уровень и клетка берутся прямо из k/r/c; номер строки берется из k/r/c
useEffect(() => {
  setHover(null);
  setLocked(null);
}, [k]);

```

```

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

```

```

return (
  <div className="flex flex-col xl:flex-row gap-6">
    <div className="flex-1 min-w-0 bg-slate-900 p-3 sm:p-6">
      <h3 className="text-xl font-bold text-indigo-400">Уровень и клетка
      <p className="text-sm text-slate-400 mb-6 text-indigo-400">
        Для наглядности показываем только <b className="text-slate-400">клетку
        <br/>
        <span className="text-rose-300 animate-pulse">клетка!</span>
      </p>
      <div className="flex bg-slate-950 p-1 rounded-l-2">
        {[0, 1, 2, 3].map(step => (
          <button

```

```

// A — неизменяемый базовый слои
// только вычисляемые уровни, не
const shownValue = compilerLink
const isBlank = shownValue === 0
let highlightClass = isBlank
    ? 'bg-slate-950 text-slate-600'
    : isCurr
    ? 'bg-slate-800/80 text-slate-600'
    : 'bg-slate-800/50 text-slate-600'
let zIndex = 'z-0';

const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << str

if (isActive) {
    if (isCurr) {
        highlightClass = `bg-indigo-500 text-white`
        {locked && locked.r === r && locked.c === c}
        zIndex = 'z-10';
    } else {
        const prevL = 1 << (shownK
        const isTop = r < activeCell.r
        const isLeft = c < activeCell.c

        if (isTop && isLeft) {
            highlightClass = 'bg-rose-500 text-white'
md:scale-[1.08]';
        } else if (isTop && !isLeft) {
            highlightClass = 'bg-blue-500 text-white'
] md:scale-[1.08]';
        } else if (!isTop && isLeft) {
            highlightClass = 'bg-emerald-500 text-white'
9,0.3)] md:scale-[1.08]';
        } else {
            highlightClass = 'bg-amber-500 text-white'
3)] md:scale-[1.08]';
        }
        zIndex = 'z-10';
    }
}

```



```

        )
      }}({})
    </div>

    <div className="pt-3 mt-3 border-t border-l border-r border-b"
      > <span className="text-white" style={activeCellStyle}>
0">{ST2D[activeCell.r][activeCell.c][shownK]}</span>
    </div>
    <p className="mt-4 text-[12px] font-mono">
список матриц (слева) вниз, чтобы увидеть,
    </div>
  ) : (
    <div className="bg-slate-950/40 border border-slate-700
      mt-2 flex items-center justify-center animation"
      > Наведите курсор на матрицу {L}x{L}
    </div>
  )}
</div>
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);
  const runtime = useVizRuntime();

```

```

// Координаты запроса – те же захардкоженные имена,
useEffect(() => {
  const vars = runtime?.variables;
  const nr1 = vizNumber(vars?.r1);
  const nc1 = vizNumber(vars?.c1);
  const nr2 = vizNumber(vars?.r2);

```

```

    )
  }}}

  <div
    className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
    style={{
      top: `${(r1 / 8) * 100}%`,
      left: `${(c1 / 8) * 100}%`,
      marginTop: '8px',
      marginLeft: '8px',
      height: 'calc(' + (H/8*100) +
      width: 'calc(' + (W/8*100) +
    }}
  />

  {/* Показываем 4 угла в самой матрице */}
  <div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
    <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px)`,
    <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
    <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
    </div>

    <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
    <label className="flex items-cen
      <span className="text-slate-
      <div className="flex gap-2">
        <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
```



```

        <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
        <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

        <span className="text-slate-500">// 2.
        <span className="text-indigo-400">int<
        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
В запроса)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-rose-400">
r1</span>][<span className="text-white bg-r

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
во, чтобы правый край уперся в c2)</span><b
        &nbsp;&nbsp;&nbsp;<span className="text-blue-400">
r1</span>][<span className="text-white bg-b

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
рх, чтобы нижний край уперся в r2)</span><b
        &nbsp;&nbsp;&nbsp;<span className="text-emerald-400">
nded">r2 - Lx + 1</span>][<span className="

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
и влево от c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-amber-400">
">r2 - Lx + 1</span>][<span className="text
{'}}')});
</div>

<div className="w-full flex justify-cent
<div className="flex flex-col items-ce
<h4 className="text-slate-400 font-bo
Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{k
</h4>
<div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>
        {Array.from({length: matRows * ma

```

```

        <span className="text-rose-400">
        <span className="text-blue-400 m
n>
        <span className="text-emerald-400
span>
        <span className="text-amber-400 m
        &nbsp;<span className="text-slate-400 m
        <span className="ml-3 text-emerald-400 m
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
        </div>
    </div>
</div>
)
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 76/87: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРок: 137 | РАЗМЕР

```

        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').
            ? 'px-3 py-1 rounded text-sm font-bold l
            : 'px-3 py-1 rounded text-sm font-bold
```

```

        if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
        if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
        if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
    });

    document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
    document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
    document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
    const mobileMenuBtn = document.getElementById('mobile-menu-btn');
    const mobileDrawer = document.getElementById('mobile-drawer');
    const closeDrawerBtn = document.getElementById('close-drawer-btn');
    const drawerOverlay = document.getElementById('drawer-overlay');

    function toggleDrawer() {
        const isClosed = mobileDrawer.classList.contains('hidden');
        if (isClosed) {
            mobileDrawer.classList.remove('hidden');
            if (drawerOverlay) {
                drawerOverlay.classList.remove('hidden');
                // Timeout allows display:block to take effect
                setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
            }
            document.body.style.overflow = "hidden";
        } else {
            mobileDrawer.classList.add('hidden');
            drawerOverlay.classList.add('hidden');
            document.body.style.overflow = "auto";
        }
    }

    mobileMenuBtn.addEventListener('click', toggleDrawer);
    closeDrawerBtn.addEventListener('click', toggleDrawer);
    drawerOverlay.addEventListener('click', toggleDrawer);
});

```

```
        document.querySelectorAll('aside')
            .forEach(function(link) {
                if (link.getAttribute('href') !== null) {
                    link.classList.add('bg-white');
                } else {
                    link.classList.remove('bg-white');
                }
            });
    }, observerOptions);

    document.querySelectorAll('section[id^="questions"]')
        .forEach(function(section) {
            observer.observe(section);
        });
    });
</script>
</body>
</html>
```

ФАЙЛ 77/87: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 10.5 КБ

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Пособие по алгебре: От пиццы к предикатам</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js"></script>
  <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.1/es5/tex-mml-chtml.js"></script>
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\(', '\\)']]
      },
      svg: {
        fontFamily: 'serif'
      }
    };
  </script>
</head>
```

```
        max-width: 100% !important;
    }
    mjx-container[display="true"] {
        overflow-x: auto;
        overflow-y: hidden;
        max-width: 100%;
        scrollbar-width: none;
        -ms-overflow-style: none;
        -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar
        display: none;
    }

    @media (max-width: 640px) {
        html { font-size: 13px; }

        .uno-card { width: 40px; height: 60px; font
        .uno-card::before, .uno-card::after { font-
        .uno-card::before { top: 1px; left: 2px; }
        .uno-card::after { bottom: 1px; right: 2px;

        .uno-card.mini { width: 28px !important; he
        us: 4px; }
        .uno-card.mini::before, .uno-card.mini::aft

        .uno-equation .text-2xl { font-size: 1.2rem
        .uno-equation.gap-4 { gap: 0.5rem !important

        /* Allow horizontal scroll for equations in
        .formula-scroll { flex-wrap: nowrap !importan
        px; }
    }
    .uno-inner {
        position: absolute; width: 110%; height: 75%
        border-radius: 50%; transform: rotate(-30deg
        box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
    }
```

```

        background-attachment: local !important;
        font-family: "Comic Sans MS", "Caveat", "Se
    }
    body.theme-notebook .bg-slate-900, body.theme-n
    late-700 {
        background-color: rgba(255, 255, 255, 0.7)
        border-color: #cbd5e1 !important;
        box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
    }
    body.theme-notebook .text-slate-200, body.theme
        ant; }
    body.theme-notebook .text-slate-400 { color: #5
    body.theme-notebook .border-slate-600, body.the
        : #94a3b8 !important; }
    body.theme-notebook .font-mono { font-family: "
    body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1);
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
    transform-origin: center;
    animation: creepy-blink 3.8s infinite;

```

```

<marker id="arrow-pink" markerWidth="6" mar
    <path d="M0,0 L6,3 L0,6 Z" fill="#f472b
</marker>
<marker id="arrow-cyan" markerWidth="6" mar
    <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4b
</marker>
<marker id="arrow-indigo" markerWidth="6" m
    <path d="M0,0 L6,3 L0,6 Z" fill="#818cf
</marker>
<marker id="arrow-lime" markerWidth="6" mar
    <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63
</marker>
<marker id="arrow-green" markerWidth="6" ma
    <path d="M0,0 L6,3 L0,6 Z" fill="#4ade8
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
    <rect x="0" y="0" width="40" height="10"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
    <path d="M 10 40 L 15 20 Q 25 15 35 20

```

```
w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
<div class="h-full flex flex-col p-6 overflow-hidden lg:rounded-2xl lg:border lg:border-slate-700">
  <div class="flex justify-between items-center">
    <h3 class="font-black text-transparent bg-clip-text bg-gradient-to-r from-indigo-500 to-purple-500 text-sm">
      Содержание
    </h3>
    <button id="close-drawer-btn" class="text-slate-500 hover:text-slate-700">
      <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6">
        <path stroke-linecap="round" stroke-width="2" d="M6 18 18 6" />
      </svg>
    </button>
  </div>
  <nav class="space-y-3 text-sm font-medium">
    <details class="group" open>
      <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
        1. Алгебраические структуры
      </summary>
      <div class="pl-6 space-y-2 text-sm">
        <a href="#section-1-1" class="text-indigo-500">1.1. Группы</a>
        <a href="#section-1-2" class="text-indigo-500">1.2. Кольца</a>
        <a href="#section-1-3" class="text-indigo-500">1.3. Поля</a>
        <a href="#section-1-4" class="text-indigo-500">1.4. Модули</a>
      </div>
    </details>
    <details class="group">
      <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
        2. Комплексные числа
      </summary>
```



```
        <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-300">
            5. Группа корней из единицы
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-fuchsia-500">
        <a href="#q5-def" class="block hover-fuchsia-500">
        <a href="#q5-group" class="block hover-fuchsia-500">
        <a href="#q5-prim" class="block hover-fuchsia-500">
        <a href="#q5-crit" class="block hover-fuchsia-500">
        <a href="#q5-cyclic" class="block hover-fuchsia-500">
    </div>
</details>
```

```
<details class="group">
    <summary class="list-none cursor-pointer-lime-500 pl-3 font-bold text-lime-300">
        6. Делимость, НОД и Евклид
    </a>
</summary>
<div class="pl-6 space-y-2 text-lime-500">
    <a href="#q6-1" class="block hover-lime-500">
    <a href="#q6-2" class="block hover-lime-500">
    <a href="#q6-3" class="block hover-lime-500">
    <a href="#q6-4" class="block hover-lime-500">
    <a href="#q6-5" class="block hover-lime-500">
    <a href="#q6-6" class="block hover-lime-500">
</div>
</details>
```

```
<details class="group">
    <summary class="list-none cursor-pointer-pink-500 pl-3 font-bold text-pink-300">
        7. Взаимно простые числа
    </a>
</summary>
```

```

<a href="#q9-2" class="block">
<a href="#q9-3" class="block">
<a href="#q9-4" class="block">
</div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
    10. Делимость и Схема Г
  </a>
</summary>
<div class="pl-6 space-y-2 text-rose-400">
  <a href="#q10-1" class="block">
  <a href="#q10-2" class="block">
  <a href="#q10-3" class="block">
a>
    <a href="#q10-4" class="block">
    <a href="#q10-5" class="block">
  </div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
    11. НОД и НОК многочленов
  </a>
</summary>
<div class="pl-6 space-y-2 text-cyan-400">
  <a href="#q11-1" class="block">
  <a href="#q11-2" class="block">
    Единственности НОД</a>
    <a href="#q11-3" class="block">
    ПИРАЛЬ)</a>
    <a href="#q11-4" class="block">
    вращается!)</a>

```

```

        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q14-1" class="block">
            <a href="#q14-2" class="block">
            <a href="#q14-3" class="block">
deg)</a>
            <a href="#q14-4" class="block">
ition-colors border border-rose-500/30"> K
        </div>
    </details>

    <details class="group">
        <summary class="list-none cursor-pointer">
            <a href="#question-15" class="block border-cyan-500 pl-3 font-bold text-cyan-400">
                15. Производная. Критерии
            </a>
        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q15-1" class="block">
            <a href="#q15-2" class="block">
            <a href="#q15-3" class="block">
            <a href="#q15-4" class="block">
        </div>
    </details>

    <details class="group">
        <summary class="list-none cursor-pointer">
            <a href="#question-16" class="block border-red-500 pl-3 font-bold text-red-400">
                16. Неприводимые многочлены
            </a>
        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q16-1" class="block">
            <a href="#q16-2" class="block">
            <a href="#q16-3" class="block">
        </div>
    </details>

```

```

        document.querySelectorA
            if (other !== detail
                other.removeAtt
            }
        });
    }
});
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('clie
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersecting)
                visibleId = entry.t

                // Highlight active
                document.querySelec
                    a.style.fontWei
                    if (a.href.ends
                        a.style.fon
                    }
                });

```

```
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('h2');
header.innerText = 'Экспорт в ' + fileName;
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за особенностей веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
```

```

        document.body.appendChild(modal)
    }

function downloadMD() {
    let clone = document.querySelector('#clone')

    // Smart markdown parsing based on DOM
    // Smart markdown parsing based on DOM
    clone.querySelectorAll('h2').forEach(el => {
    clone.querySelectorAll('h3').forEach(el => {
    clone.querySelectorAll('.analog').forEach(el => {
    clone.querySelectorAll('li').forEach(el => {
    clone.querySelectorAll('img').forEach(el => {
        let caption = '';
        if (el.nextElementSibling && el.nextElementSibling.tagName === 'CAPTION') {
            caption = el.nextElementSibling.textContent;
        }
        el.textContent = '\n\n[ Изображение: ' + caption + ' ]';
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        let text = clone.innerHTML;
        text = text.replace(/\n{3,}/g, '\n\n');

        if (window !== window.top) {
            fallbackModal(text, 'Markdown');
            return;
        }
        const blob = new Blob([text], { type: 'text/markdown' });
        const url = URL.createObjectURL(blob);
        const link = document.createElement('a');
        link.href = url;
        link.download = 'vyisshaya_algebra.md';
        document.body.appendChild(link);
        link.click();
        document.body.removeChild(link);
    });
}

```

```

nt; }

body.bg-white [class*="]
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

```

```

        <a href="algebra.pdf" target="_blank">
            -sm font-bold flex items-center justify-center gap-2
                <span> </span> PDF
            </a>
            <button onclick="downloadMD()">
                -bold flex items-center justify-center gap-2
                    <span> </span> Markdown
                </button>
            <button onclick="downloadHTML()">
                t-sm font-bold flex items-center justify-center gap-2
                    <span> </span> HTML
                </button>
        </div>

        <!-- Смена темы -->
        <div>
            <div class="text-xs text-slate-500">
                <div class="flex items-center gap-2">
                    <button id="theme-dark" onclick="toggleTheme('dark')">
                        justify-center text-amber-300 hover:text-white
                        line-amber-500" title="Темная тема"> </button>
                    <button id="theme-light" onclick="toggleTheme('light')">
                        r justify-center text-amber-50 hover:text-white
                        ">* </button>
                    <button id="theme-terminal" onclick="toggleTheme('terminal')">
                        -center justify-center text-green-400 hover:text-green-500
                        терминала"> </button>
                </div>
            </div>
        </div>
    </div>
</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12">
    -base min-w-0">
        <header class="text-center mb-12 border-b border-slate-700">

```



```

<h2 class="text-3xl font-black text-cen

<div class="overflow-x-auto w-full pb-4
  <div class="grid grid-cols-3 gap-4
    <!-- Column headers -->
    <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
    <div class="bg-orange-900/40 bo
ольцо полиномов</div>
    <div class="bg-cyan-900/40 bord
сные числа</div>

    <!-- Row 1: База -->
    <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
  </a>
    <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
    <div class="text-xs text-orange
    <div class="font-bold text-slate
  </a>
    <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
    <div class="text-xs text-cyan-5
    <div class="font-bold text-slate
  </a>

    <!-- Row 2: Базовые операции / Евклид
    <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
  </a>
    <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange

```

```

        <!-- Row 5: Корни / Уравнения -->
        <div class="hidden md:block"></div>
        <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 6: Кратность -->
        <div class="hidden md:block"></div>
        <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <div class="hidden md:block"></div>

        <!-- Row 7: Производная -->
        <div class="hidden md:block"></div>
        <a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <div class="hidden md:block"></div>

        <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
        <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

        <a href="#question-17" onclick="setM
-1-4 border-teal-500 hover:bg-slate-700 tra

```

```
        </a>
    </div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
    <div class="flex items-center mb-6">
        <div class="bg-red-500/20 p-3 r
            <svg class="w-8 h-8 text-re
rg/2000/svg">
                <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
            </svg>
        </div>
        <h2 class="text-3xl pr-4 border
    </div>

    <div class="prose prose-invert max-w
        <p class="text-lg">
            Хватит тупить. Доказательств
есь для того, чтобы жать на кнопки. Вот жес
        </p>

        <h2 class="text-2xl font-bold b
(Меметика)</h2>

        <!-- Прямое -->
        <div class="bg-slate-800/80 p-6
            <h3 class="text-xl text-blue
            <p class="text-sm text-slate
            <ul class="list-disc pl-5 sp
                <li><span class="text-w
k$).</li>
                    <li><span class="text-w
                    <li><span class="text-w
ово.</li>
            </ul>
        </div>
```

усок k , замени его на жареного карася из

-

```
<!-- Двойная делимость -->
<div class="bg-slate-800/80 p-6">
  <h3 class="text-xl text-orange-400">
    <p class="text-sm text-slate-400">
      <ul class="list-disc pl-5">
        <li><span class="text-white">
          вверх по уравнениям).</li>
        <li><span class="text-white">
          рху вниз, показывая, что  $\Delta \vdots r_i$ 
        <li><span class="text-white">
          х. Твой  $d$  больше.</li>
      </ul>
    </p>
  </h3>
</div>
```

```
<h2 class="text-2xl font-bold bg-white">
  ритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md:grid-cols-2">
  <div class="bg-slate-800 p-6">
    <h4 class="text-red-400">
      <p class="text-sm text-slate-400">
        <p class="text-xs font-sans">
          со следующим. Если в конце 0 – пациент мертв
        </p>
      </p>
    </h4>
  </div>
```

```
<div class="bg-slate-800 p-6">
  <h4 class="text-red-400">
    <p class="text-sm text-slate-400">
      <p class="text-xs font-sans">
         $c$ . Пока получается 0 – корень жив. Как то.
        корня минус один.</p>
      </p>
    </h4>
  </div>
```

х двух, потом последних.


```
<code class="text-x
```

```
</div>
```

```
</li>
```

```
<li class="bg-slate-800/50
```

```
<span class="text-2xl">
```

```
<div>
```

```
<h4 class="text-cyan
```

```
<p class="text-sm t
```

есть сложение — есть ноль. Если есть умноже

```
<code class="text-x
```

```
</div>
```

```
</li>
```

```
<li class="bg-slate-800/50
```

```
<span class="text-2xl">
```

```
<div>
```

```
<h4 class="text-cyan
```

```
<p class="text-sm t
```

авь их драться. Умножение всегда врывается

```
<code class="text-x
```

```
</div>
```

```
</li>
```

```
</ul>
```

```
</div>
```

```
</section>
```

```
</div>
```

```
<div id="questions-container">
```

```
<!-- ===== ВОПРОС 1 =====
```

```

        const rules = (sheet as CSSStyleSheet).cssRules;
        parts.push(Array.from(rules).map((r) => r.cssText));
    } catch {
        const href = (sheet as CSSStyleSheet).href;
        if (!href) continue;
        try {
            const res = await fetch(href);
            if (res.ok) parts.push(await res.text());
        } catch {
            /* чужой домен – пропускаем */
        }
    }
}
return parts.join("\n");
}

```

```

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
    border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
@media print {
    body { background: #fff; color: #0f172a; }
    .export-note, details { break-inside: avoid; }
    details { display: block; }

```

```
    return { body: host.innerHTML, termIds: ids };
  }

function glossarySection(termIds: string[]): string {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escapeHtml(id)}">
        <h3>${escapeHtml(e.title)}</h3>
        <p>${escapeHtml(e.short)}</p>
        ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
        ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
      </div>`;
    })
    .join("\n");

  return `<section class="export-gloss">
    <h2>Словарик терминов из этой темы</h2>
    <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
      ны сюда.</p>
    ${items}
  </section>`;
}
```

```
function wrap(opts: { title: string; subtitle: string; stamp: string; css: string; extraCss: string }): string {
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
```



```
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/data/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.lastIndexOf("}"));
const vizIds = new Set([...registryBody.matchAll(/^\s{2}[a-zA-Z0-9_]+/g)].map(s => s.slice(2)));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
    interactive: vizIds.has(c.id),
  };
});
```

ФАЙЛ 81/87: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossa",
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m",
  external: ["react", "react-dom", "motion", "lucide-re",
  });

const { chapters } = await import(path.join(ROOT, "tmp/"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
  `(?<![\\p{L}\\p{N}_-])(?=${GLOSSARY_LABELS.map((l) => e
  "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
```

```
-----  
/**  
 * ШАГ 3: PNG -> PDF  
 *  
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в единый документ public/export/full_code_all_pages.pdf  
 */  
import fs from "node:fs";  
import path from "node:path";  
import PDFDocument from "pdfkit";  
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, }  
  
// A4 в пунктах PDF  
const A4 = { width: 595.28, height: 841.89 };  
  
async function main() {  
  if (!fs.existsSync(PAGES_DIR)) {  
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);  
  }  
  
  const pages = fs  
    .readdirSync(PAGES_DIR)  
    .filter((f) => f.endsWith(".png"))  
    .sort();  
  
  if (pages.length === 0) throw new Error("Не найдено ни одной страницы");  
  
  ensureDir(OUT_DIR);  
  
  const doc = new PDFDocument({  
    size: [A4.width, A4.height],  
    margin: 0,  
    autoFirstPage: false,  
    info: {  
      Title: "Универсальное пособие – полный исходный код",  
    },  
  });  
  doc.pipe(fs.createWriteStream(PDF_PATH));  
  for (const page of pages) {  
    doc.addPage(fs.readFileSync(path.resolve(PAGES_DIR, page)));  
  }  
  doc.end();  
}
```

Универсальное пособие • полный исходный код

```
-----
*
* Собирает ВСЕ исходный код репозитория в один текст
* public/export/full_code_all_pages.txt
*
* Список файлов берётся из git (чтобы ничего не забыть
* с фоллбэком на обход файловой системы, если git недо
*/
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в дан
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+\$/ , "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re
```

```

return files
  .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
  .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
  .filter((rel) => CODE_EXT.has(path.extname(rel)))
  .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
  .sort((a, b) => {
    const ia = ORDER.indexOf(categoryOf(a));
    const ib = ORDER.indexOf(categoryOf(b));
    return ia !== ib ? ia - ib : a.localeCompare(b);
  });
}

/** Достаём заголовки глав из данных пособия (без запуска)
function extractChapters(files) {
  const chapters = [];
  const seen = new Set();
  for (const rel of files.filter((f) => f.startsWith("src/"))) {
    const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
    const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
    let m;
    while ((m = re.exec(src)) !== null) {
      const [, id, title] = m;
      if (seen.has(id)) continue;
      seen.add(id);
      chapters.push({ id, title, file: rel });
    }
  }
  return chapters.sort((a, b) => {
    const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
    const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
    return na - nb;
  });
}

```

```

function main() {
  const files = listFiles();
  const chapters = extractChapters(files);
  ensureDir(OUT_DIR);
}

```

```

    });
    push();
    push();

    current = null;
    files.forEach((rel, i) => {
        const cat = categoryOf(rel);
        if (cat !== current) {
            current = cat;
            push(HR);
            push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
            push(HR);
            push();
        }
        const content = fs.readFileSync(path.join(ROOT, rel));
        const lines = content.split("\n");
        push(SUBHR);
        push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
        push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | `);
        push(SUBHR);
        push();
        push(content.replace(/\n+$/, ""));
        push();
        push();
    });

    push(HR);
    push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
    push(HR);

    const txt = out.join("\n") + "\n";
    fs.writeFileSync(TXT_PATH, txt, "utf8");

    console.log("[1/3] код -> txt");
    console.log(`файлов: ${files.length}`);
    console.log(`глав: ${chapters.length}`);
    console.log(`строк: ${txt.split("\n").length}`);
    console.log(`выход: ${path.relative(ROOT, TXT_PATH)}`);

```

```
    size: "A4",
    font: "DejaVu-Sans-Mono",
    fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
    pointsize: 8,
    /** Максимум символов в строке (далее – жёсткий пере
    cols: 138,
    /** Строк содержимого на странице (+2 служебные: шапк
    rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toF
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  for (const cmd of ["magick", "convert"]) {
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore"
      return cmd;
    } catch {
      /* пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-ge
  );
}
```

```

    }
    return parts;
}

function paginate(txt) {
    const source = txt.replace(/\r\n/g, "\n").split("\n");
    const pages = [];
    for (let i = 0; i < source.length; i += PAGE.rows) {
        pages.push(source.slice(i, i + PAGE.rows));
    }
    return pages;
}

async function main() {
    if (!fs.existsSync(TXT_PATH)) {
        throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
    }

    const im = imageMagickCmd();
    const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

    fs.rmSync(PAGES_DIR, { recursive: true, force: true });
    ensureDir(PAGES_DIR);

    const total = pages.length;
    const jobs = pages.map((lines, idx) => {
        const num = String(idx + 1).padStart(4, "0");
        const header = `Универсальное пособие • полный исходный код
        Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`)
        `}стр. ${idx + 1} / ${total}`;
        const body = [header, "-".repeat(PAGE.cols), ...lines];
        const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
        const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
        fs.writeFileSync(txtFile, body + "\n", "utf8");
        return { txtFile, pngFile, num };
    });

    const args = (job) => [

```



```
-----  
    console.log(`        страниц: ${total}`);  
    console.log(`        выход:    ${path.relative(ROOT, PAGE)  
}  
  
main().catch((err) => {  
    console.error("Ошибка рендера страниц:", err.message);  
    process.exit(1);  
});
```

=====

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

ФАЙЛ 86/87: `.github/workflows/deploy-pages.yml`
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

name: Deploy app to GitHub Pages

```
# ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ  
# (у бота-интеграции нет admin-прав на них – API отвечает)  
#  
# 1. Settings → Pages → Build and deployment → Source  
#    Если там «Deploy from a branch» (legacy) – шаг D  
# 2. Settings → Environments → github-pages → Deployments  
#    добавьте паттерн «arena/**» и ветку «main» (или  
#    иначе деплой из новой рабочей ветки отклоняется
```

on:

push:

branches:

- main

Позволяет проверить Pages прямо из рабочей ветки

- "arena/**"

workflow_dispatch:

```
deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    # Ранняя и понятная ошибка вместо невнятного «Fail»
    # когда в репозитории Pages не переключён на сбор
    - name: Preflight – Pages собирается через GitHub
      env:
        GH_TOKEN: ${ secrets.GITHUB_TOKEN }
      run: |
        bt=$(gh api "repos/${ github.repository }/pages" |
        echo "Pages build_type: ${bt}")
        if [ "${bt}" != "workflow" ]; then
          echo "::error title=Pages не настроен::Settings не
          те этот запуск. См. комментарий в начале deploy
          exit 1
        fi

    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
```

ФАЙЛ 87/87: .github/workflows/rebuild-pdf.yml
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ

name: Rebuild code PDF

```
# Пайплайн пересборки PDF при каждом коммите:
#
# исходный код → full code all pages.txt (script
```

- name: Checkout
uses: actions/checkout@v6
with:
 fetch-depth: 0
 persist-credentials: true
- name: Setup Node.js
uses: actions/setup-node@v7
with:
 node-version: "24"
 cache: npm
- name: Install ImageMagick + DejaVu fonts
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends
 # На Ubuntu политика ImageMagick иногда запрещает экспорт в pdf
 # разрешаем то, что нужно пайплайну (text -> pdf)
 for policy in /etc/ImageMagick-6/policy.xml /usr/lib/ImageMagick-6/policy.xml; do
 if [-f "\$policy"]; then
 sudo sed -i 's/rights="none" pattern="\(*\)\s*/rights="none" pattern="\1"/g' \$policy
 fi
 done
 convert -version
- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${GITHUB_EVENT_INPUTS.density:-300}
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf

```
git config user.email "41898282+github-action@users.noreply.github.com"
git add -- public/export/full_code_all_pages.*
if git diff --cached --quiet; then
    echo "Экспорт не изменился – коммит не нужен"
    exit 0
fi
git commit -m "chore(export): пересборка full_code_all_pages.*"
git push origin "HEAD:${GITHUB_REF_NAME}"
```